IL1332: Digital Design Using HDL

L7: Combinational Building Blocks

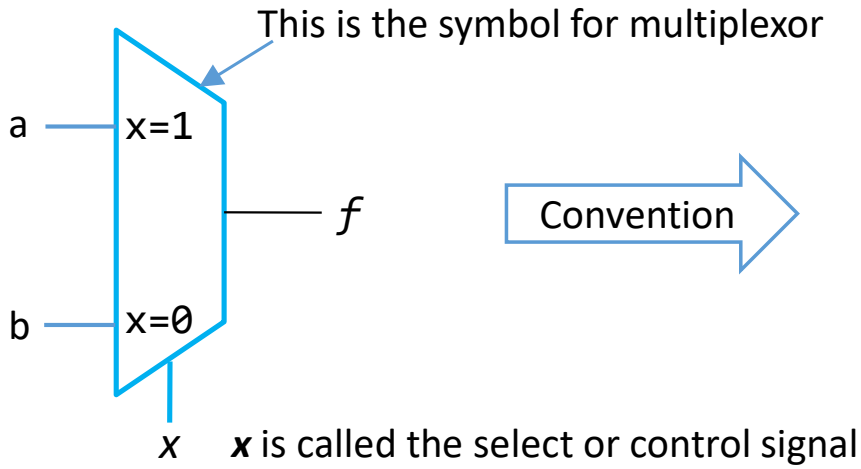
Connecting Lecture to the Textbook

5.2.3 *Comparators* 246
5.2.4 *ALU* 248
5.2.5 *Shifters and Rotators* 250

Not covered in text book

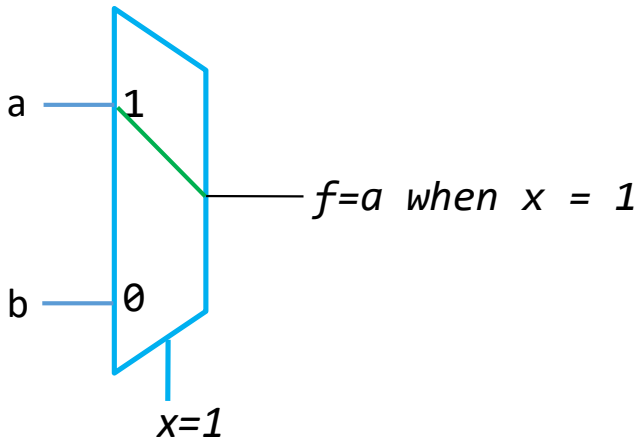
Shannon Decomposition
Multiplexor as the universal building block
Demultiplexor, Encoders and Decoders

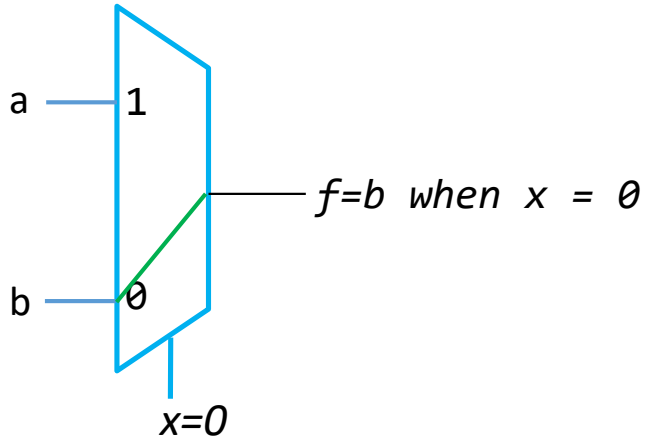
Multiplexor Revision



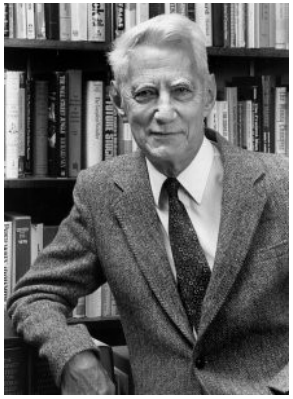
x is called the select or control signal

Multiplexor is also called a Selector





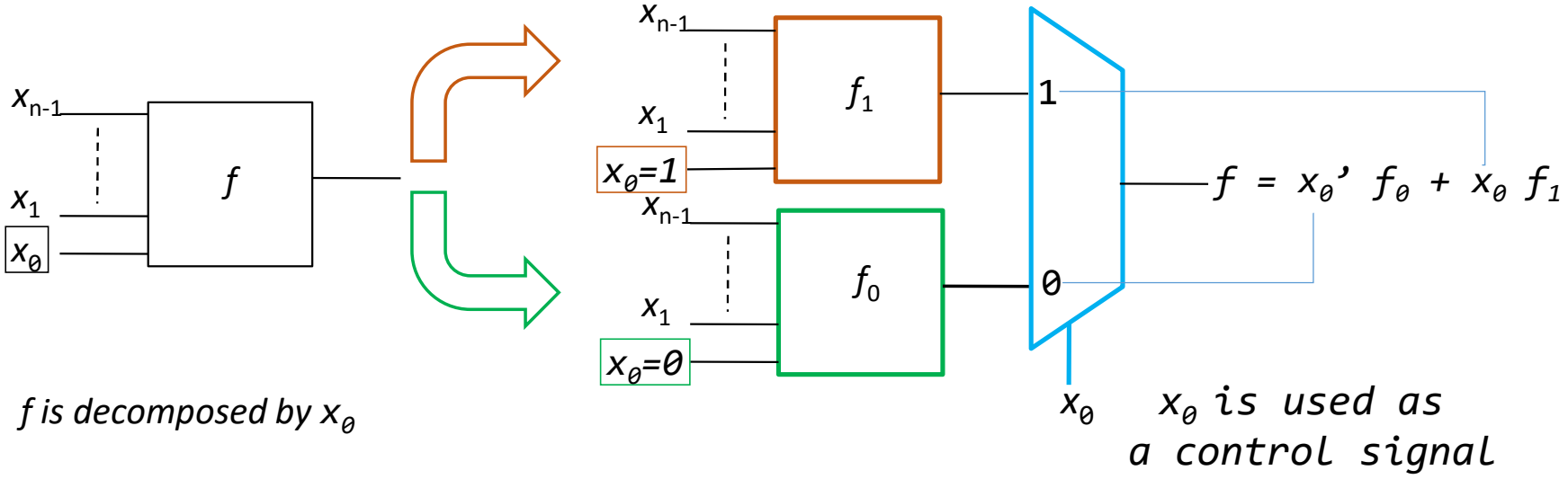
Shannon Decomposition



Claude Shannon

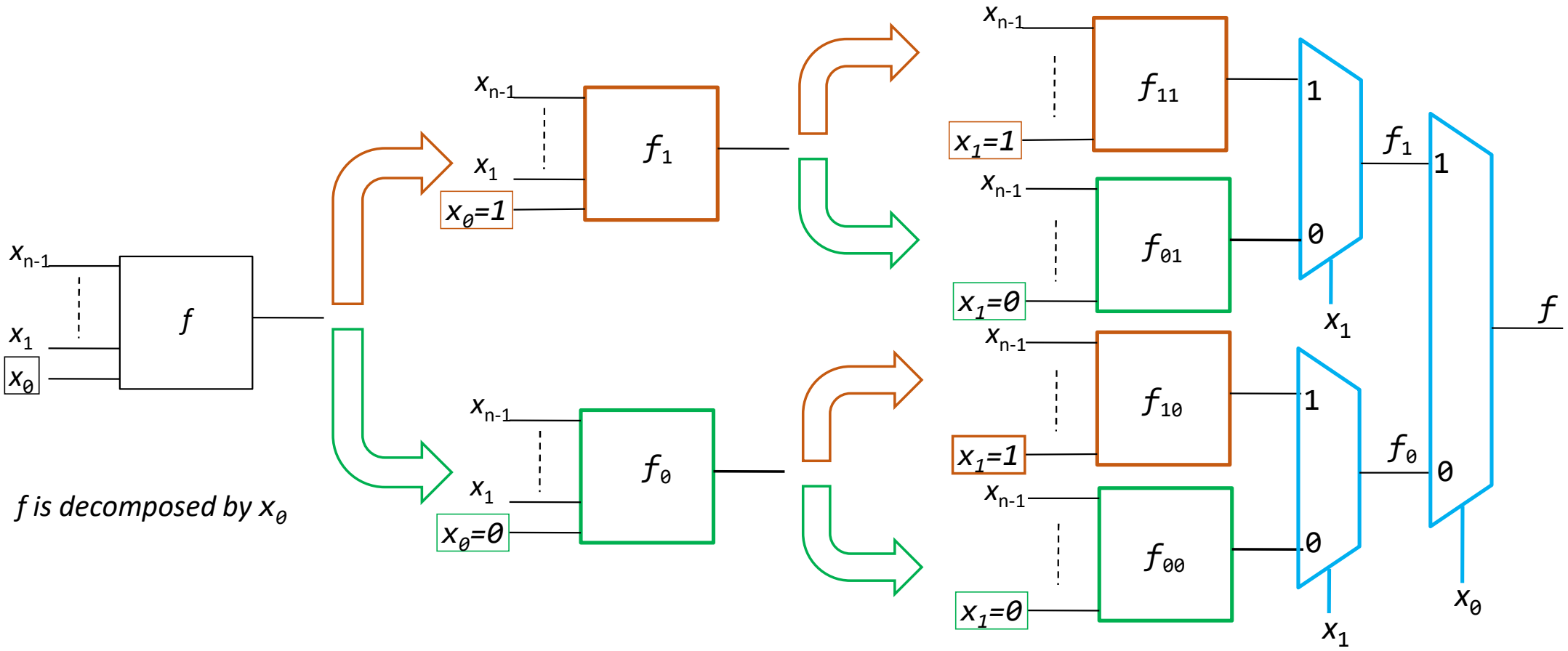
Mathematician and Electrical Engineer (1916 –2001)

Together with Harry Nyquist laid the foundations of Communications Engineering



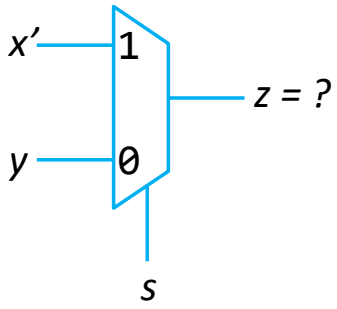
f is decomposed by x_0

Shannon Decomposition is Recursive



We continue decomposition until inputs are constants 1 or 0

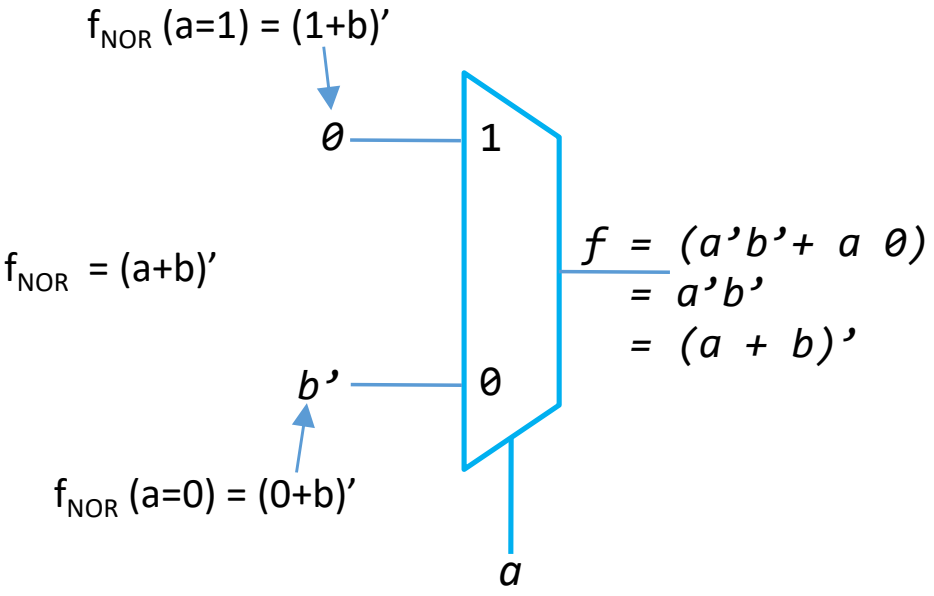
Quiz Time

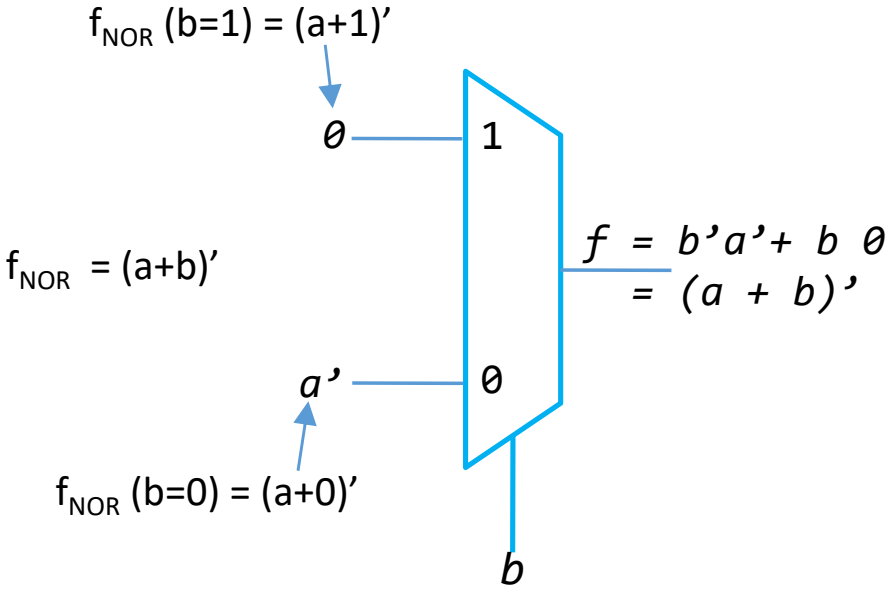


A. xs + y's'
B. xs' + y's
C. x's + ys'
D. x's' + xs

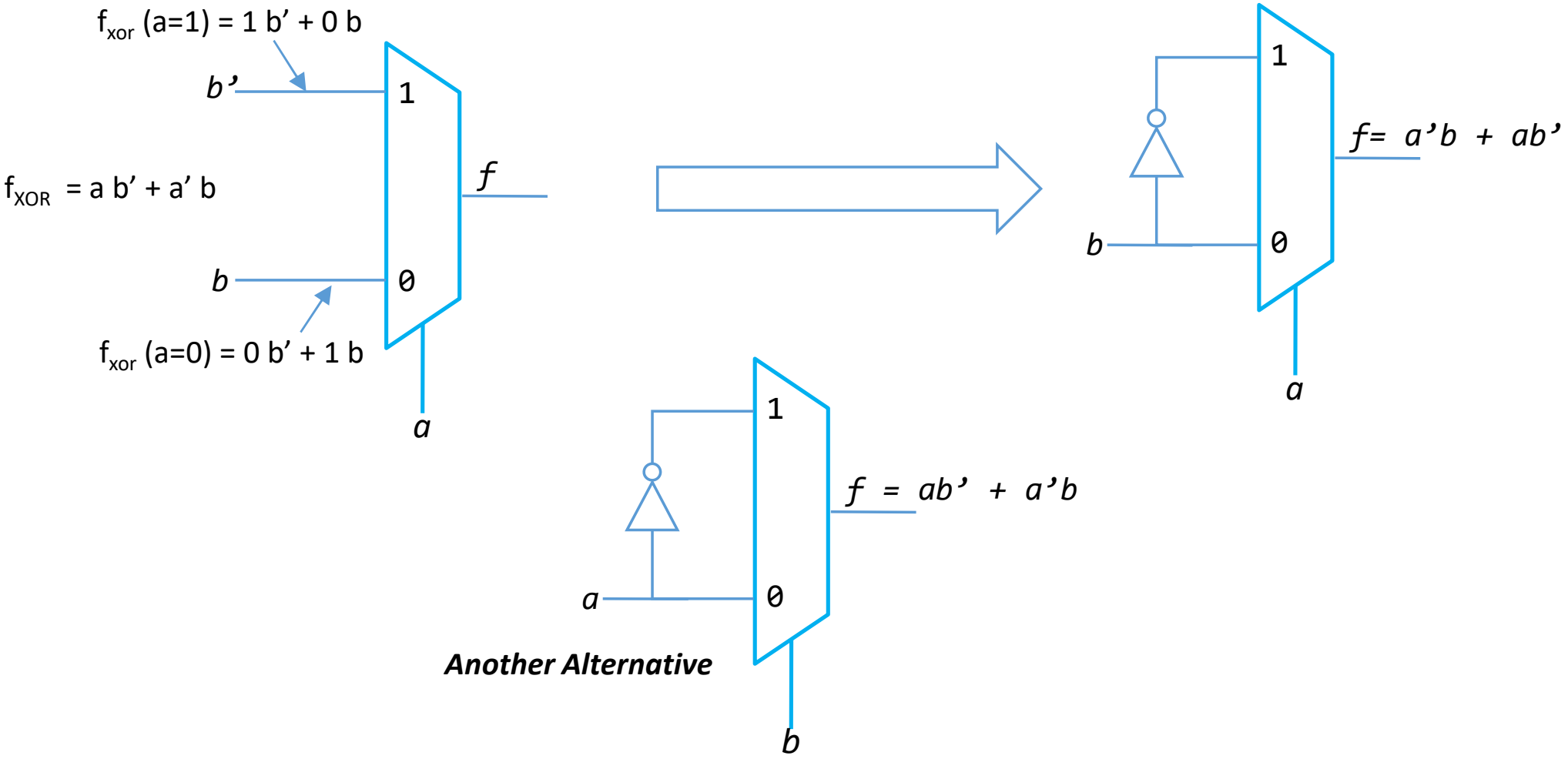
Custom Mux Structures using Shannon Decomposition

NOR gate using 2-to-1 Mux

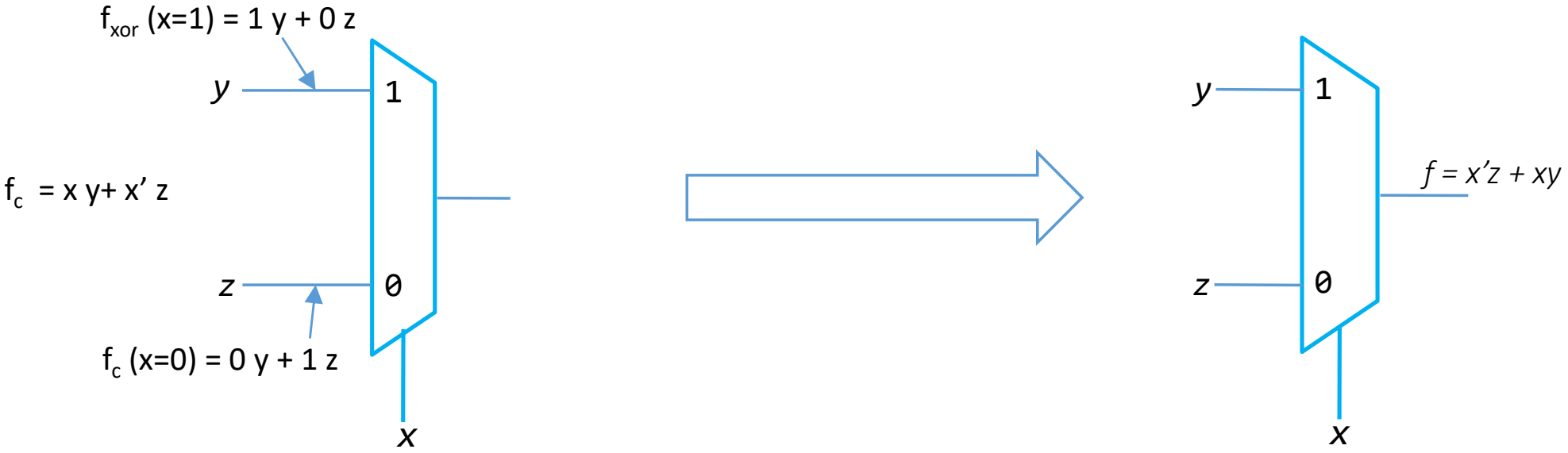




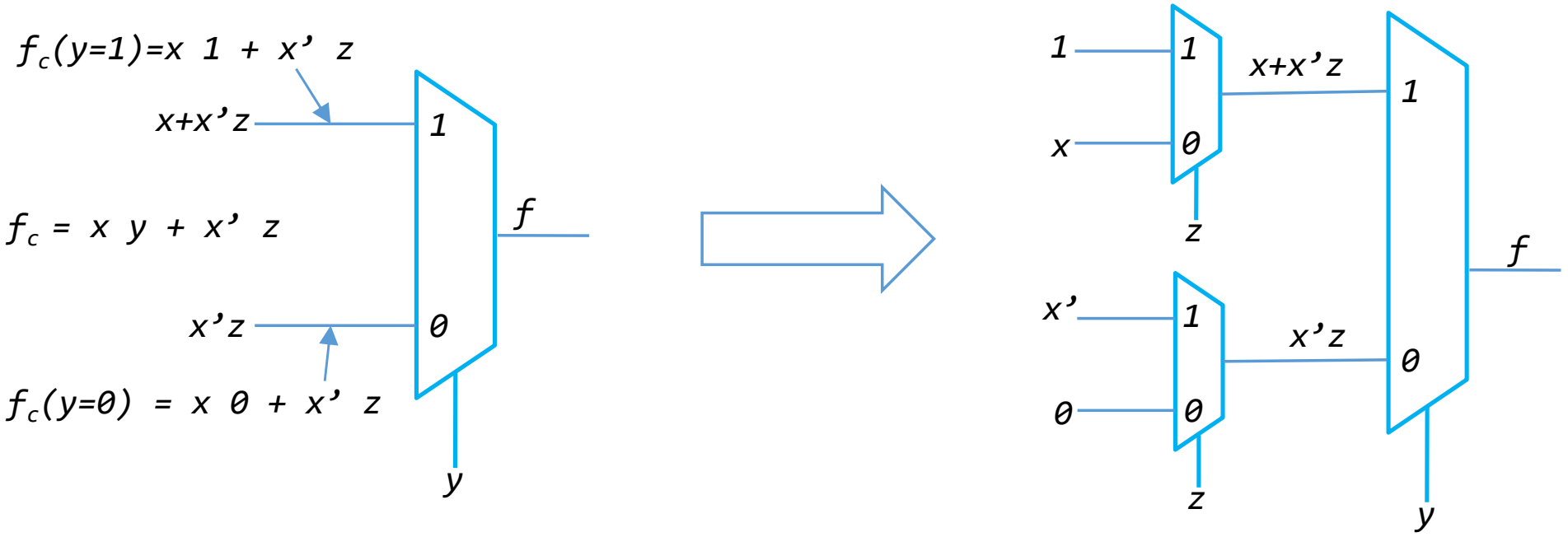
XOR using 2-to-1 Mux



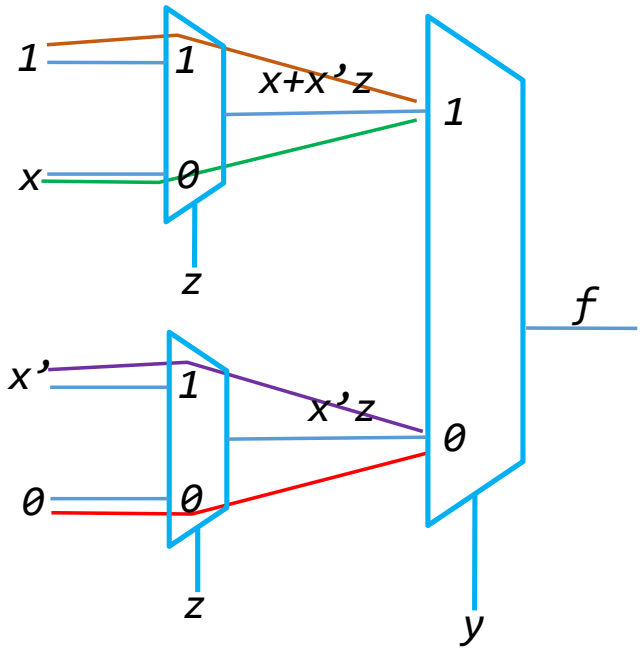
Consensus Theorem using 2-to-1 Muxes

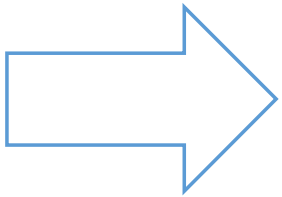


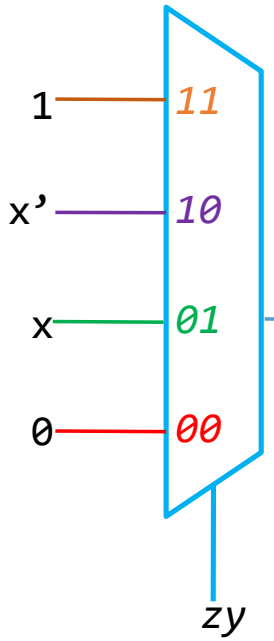
Consensus Theorem using 2-to-1 Muxes



Consensus Theorem using 4-to-1 Muxes







$f = z'y'0 + z'yx + zy'x' + zy$

$= z'yx + zy'x' + zy$

$= z'yx + z(y'x' + y)$

$= z'yx + z(x' + y)$ – 16a

$= z'yx + zx' + zy$

$= (z'x + z)y + zx'$ – 16a

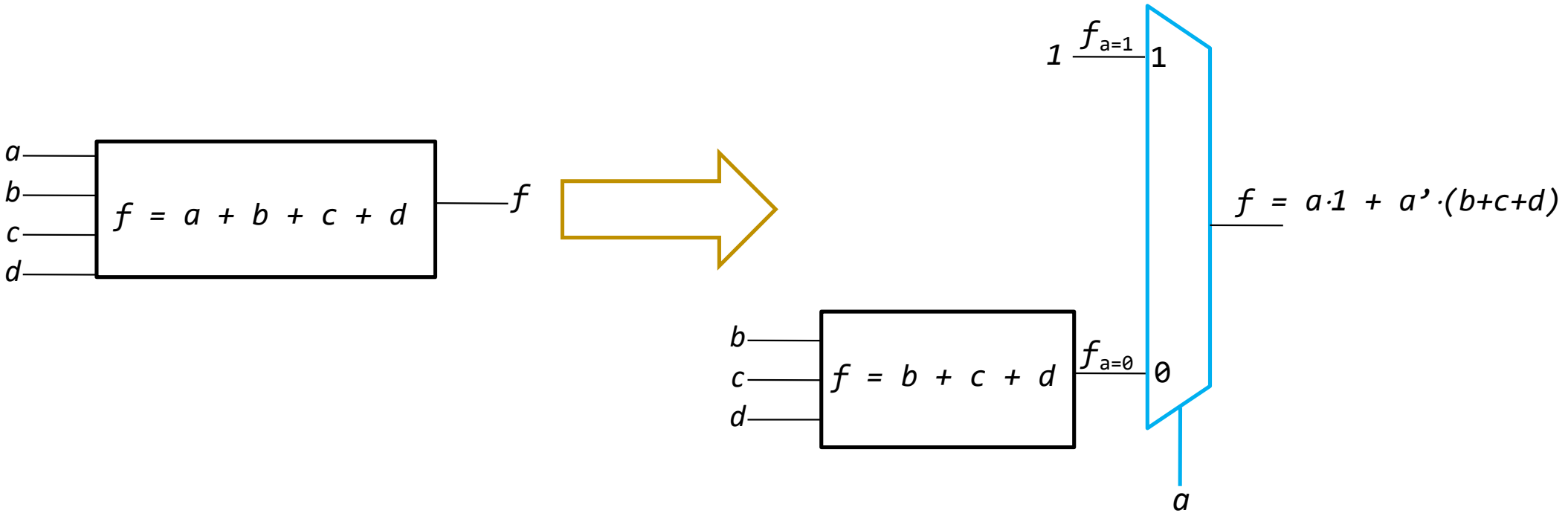
$= (z + x)y + zx'$

$= zy + xy + zx'$

$= xy + x'z$ – 17a

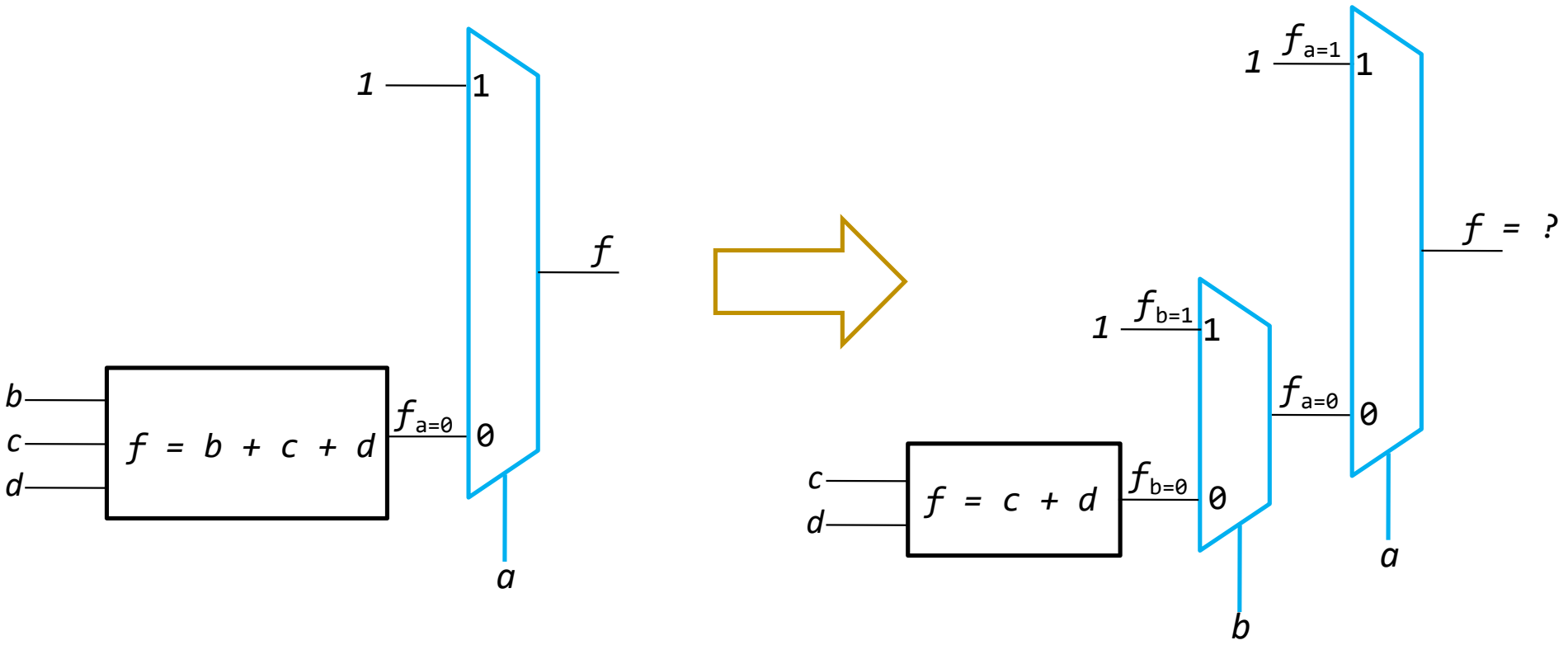
See Boolean Algebra Slides

Example: $f = a + b + c + d$



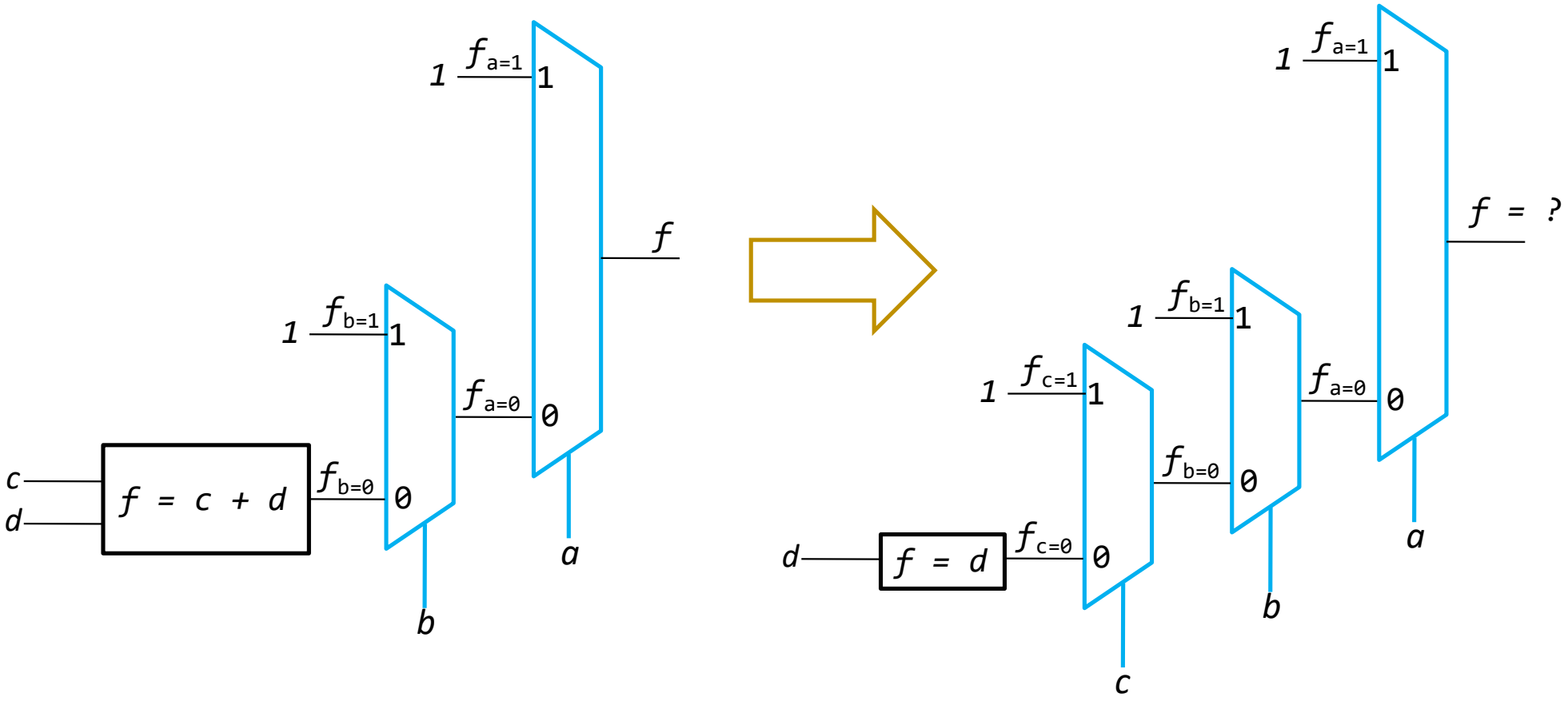
We continue decomposition until inputs are constants 1 or 0

Example: $f = a + b + c + d$



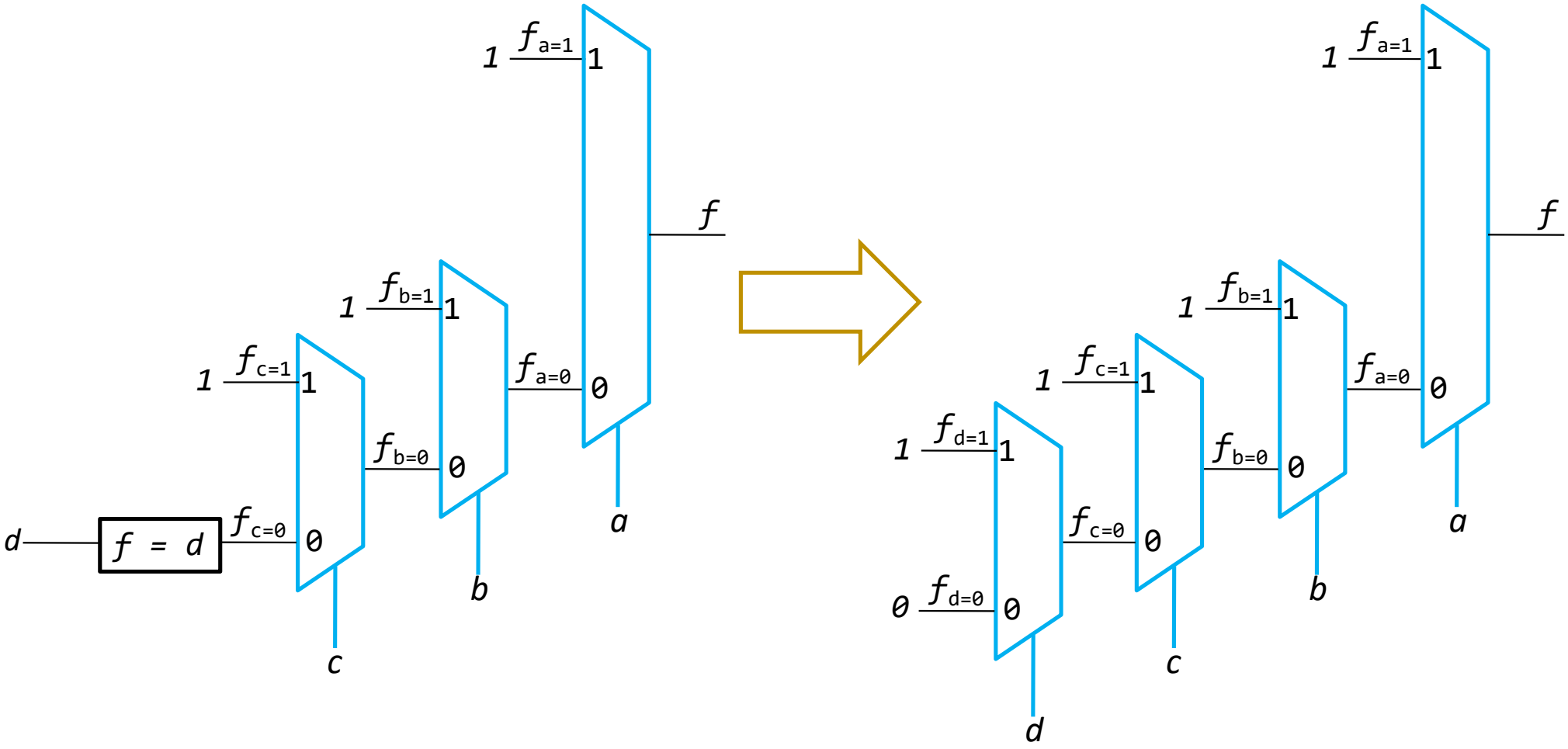
We continue decomposition until inputs are constants 1 or 0

Example: $f = a + b + c + d$

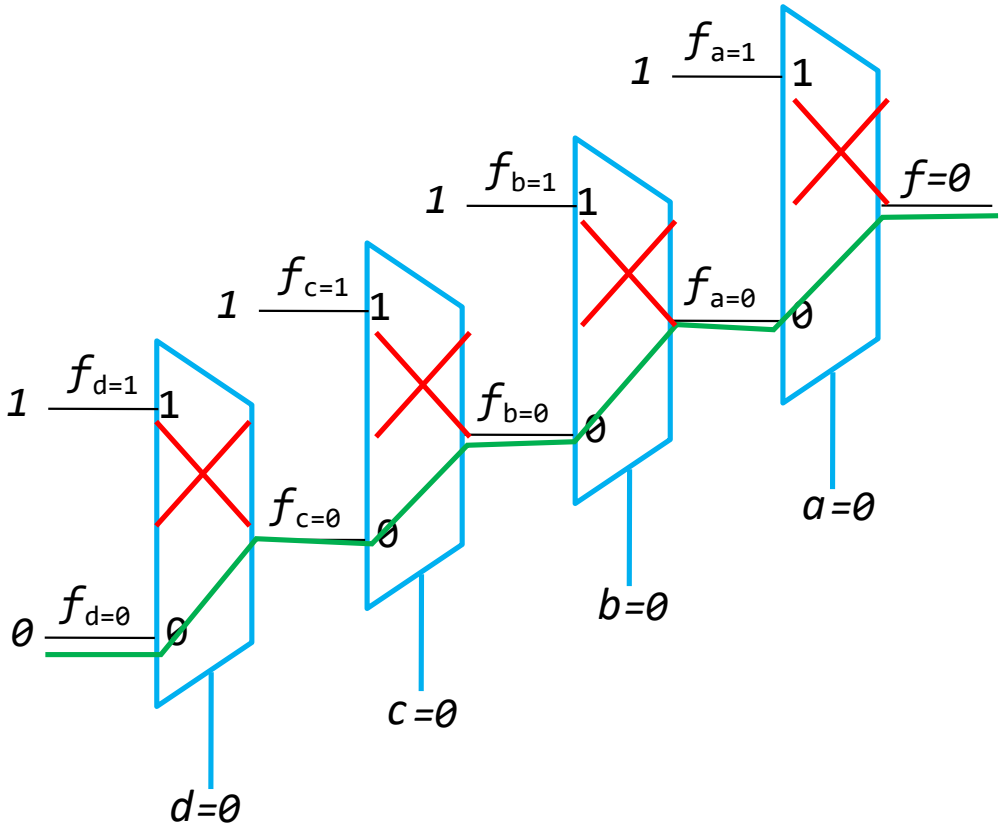


We continue decomposition until inputs are constants 1 or 0

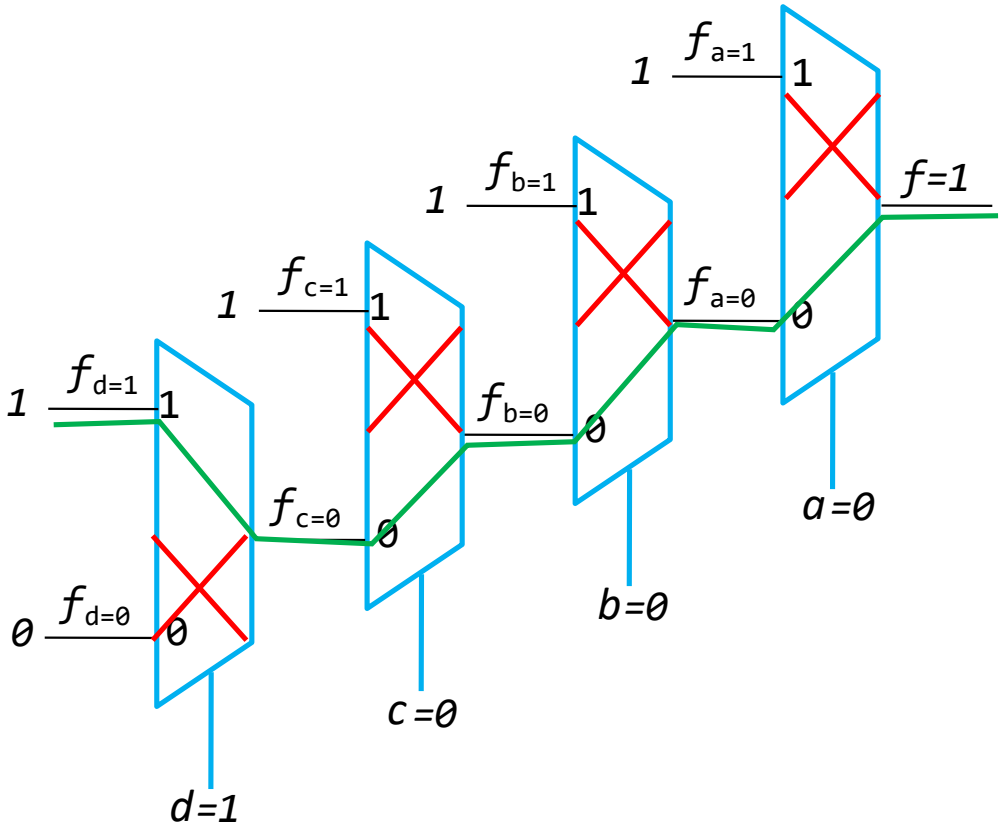
Example: $f = a + b + c + d$



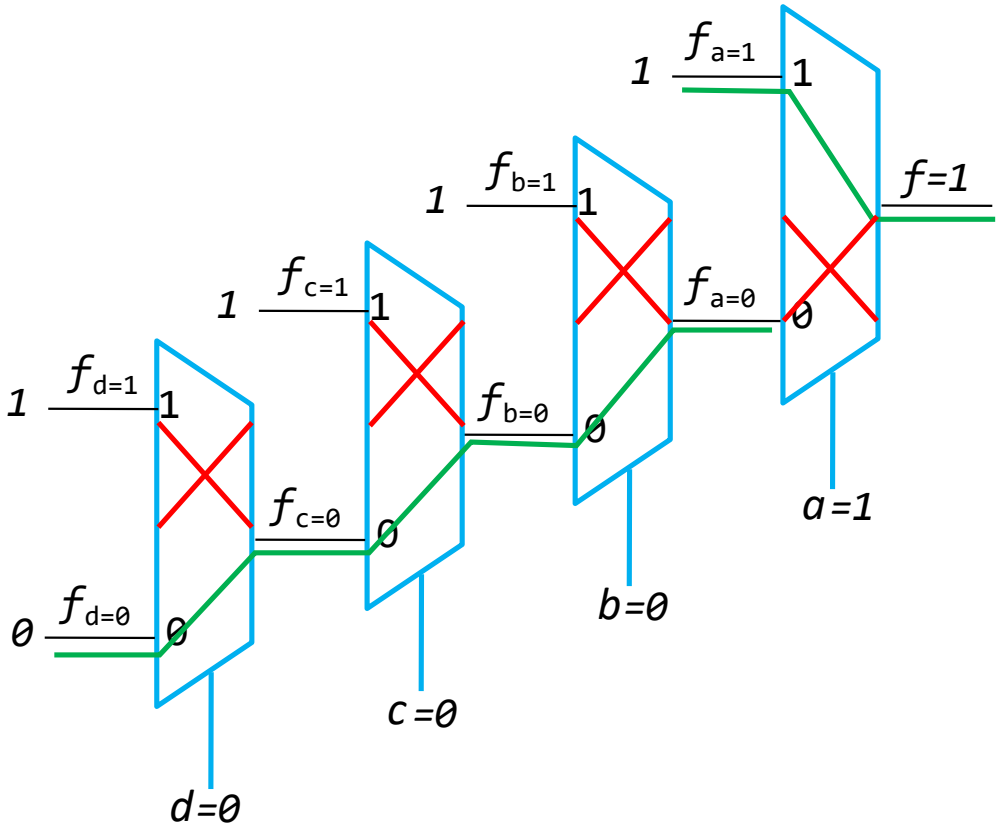
Example: $f = a + b + c + d$



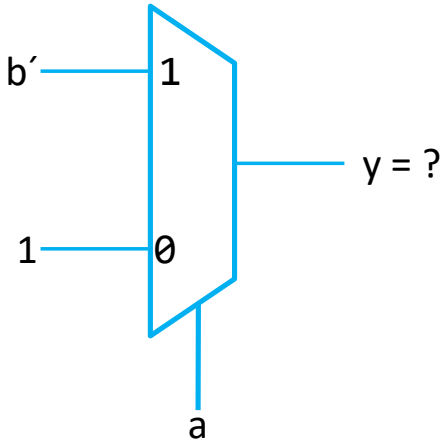
Example: $f = a + b + c + d$



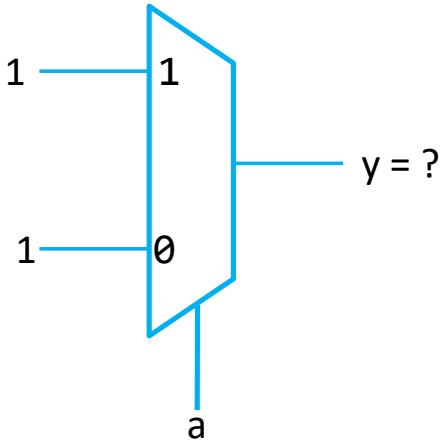
Example: $f = a + b + c + d$



Quiz Time



A. a AND b
B. a NAND b
C. a OR b
D. a NOR b



A. 0
B. 1
C. a
D. a’

Generic Mux Structures

Generic Mux Network

1. The Network of Muxes, we saw is for a specific function f = a + b + c + d

2. We will next see how do we create

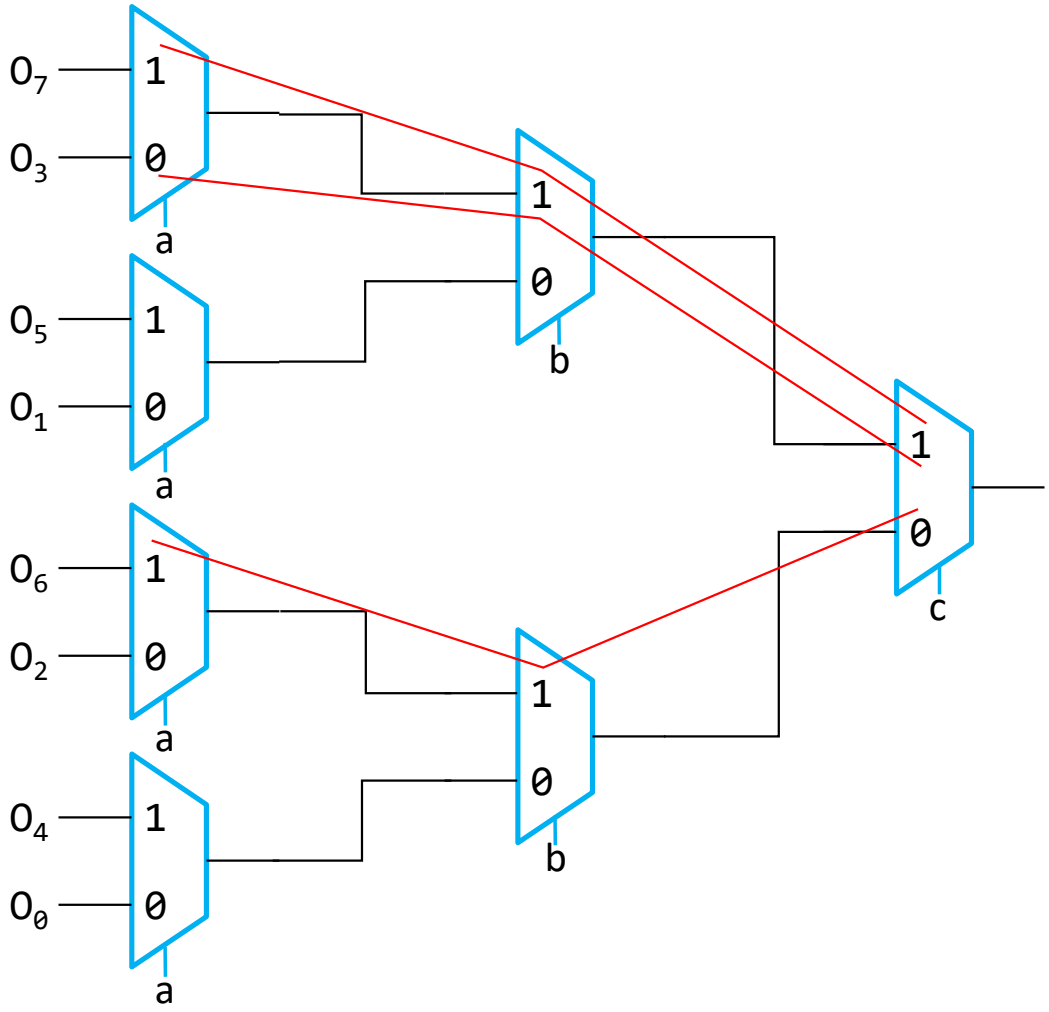
- ***a. A Generic Mux Network***
- ***b. That can implement ANY boolean function***
- ***c. In a certain number of boolean variables***

Generic Mux Network to implement arbitrary 3 variable Boolean Function

What are the literals O_7 *to* O_0 *?*

Any three variable (a, b and c) Boolean Function will have 8 outputs corresponding to 8 minterms

O_7 **to** O_0 *are these eight outputs*



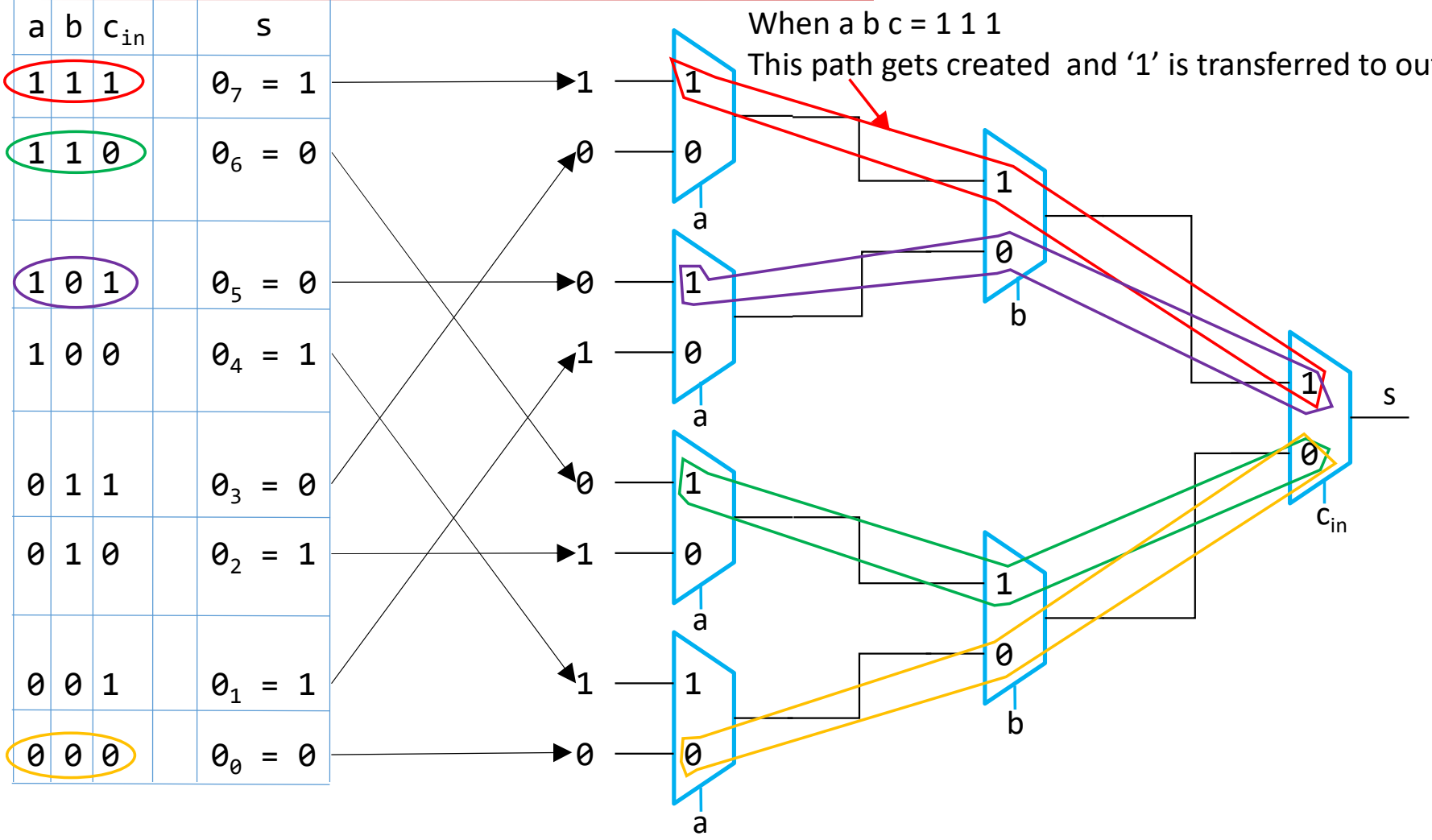
Let us consider the sum and carry-out outputs

a	b	c_{in}		s
1	1	1		$0_7 = 1$
1	1	0		$0_6 = 0$
1	0	1		$0_5 = 0$
1	0	0		$0_4 = 1$
0	1	1		$0_3 = 0$
0	1	0		$0_2 = 1$
0	0	1		$0_1 = 1$
0	0	0		$0_0 = 0$

a	b	c_{in}		co
1	1	1		$0_7 = 1$
1	1	0		$0_6 = 1$
1	0	1		$0_5 = 1$
1	0	0		$0_4 = 0$
0	1	1		$0_3 = 1$
0	1	0		$0_2 = 0$
0	0	1		$0_1 = 0$
0	0	0		$0_0 = 0$

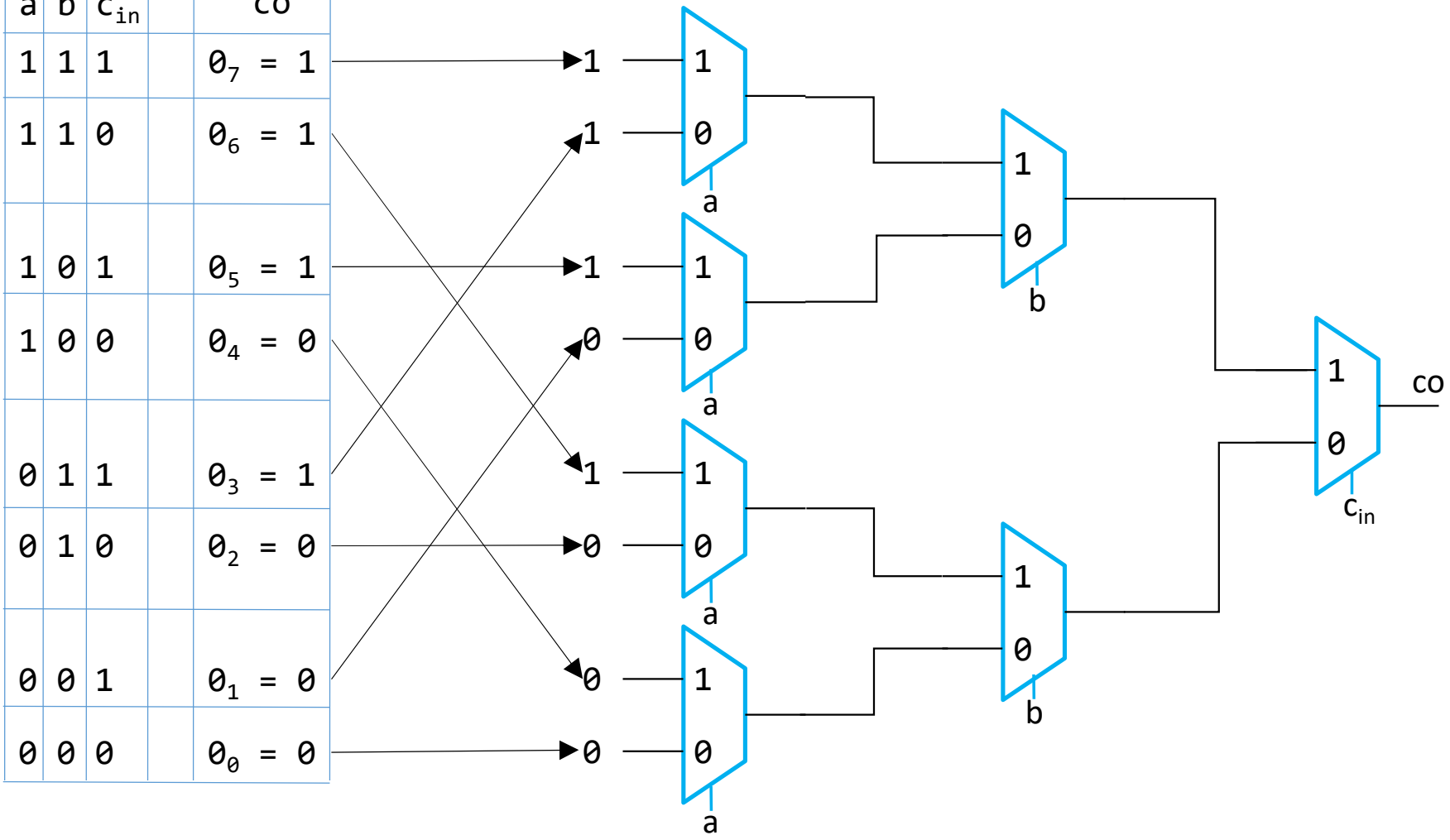
Generic Mux Network Implements Sum function of Full Adder

a	b	c_{in}		s
1	1	1		$0_7 = 1$
1	1	0		$0_6 = 0$
1	0	1		$0_5 = 0$
1	0	0		$0_4 = 1$
0	1	1		$0_3 = 0$
0	1	0		$0_2 = 1$
0	0	1		$0_1 = 1$
0	0	0		$0_0 = 0$



Generic Mux Network Implements Carry-out function of Full Adder

a	b	c_{in}		co
1	1	1		$0_7 = 1$
1	1	0		$0_6 = 1$
1	0	1		$0_5 = 1$
1	0	0		$0_4 = 0$
0	1	1		$0_3 = 1$
0	1	0		$0_2 = 0$
0	0	1		$0_1 = 0$
0	0	0		$0_0 = 0$



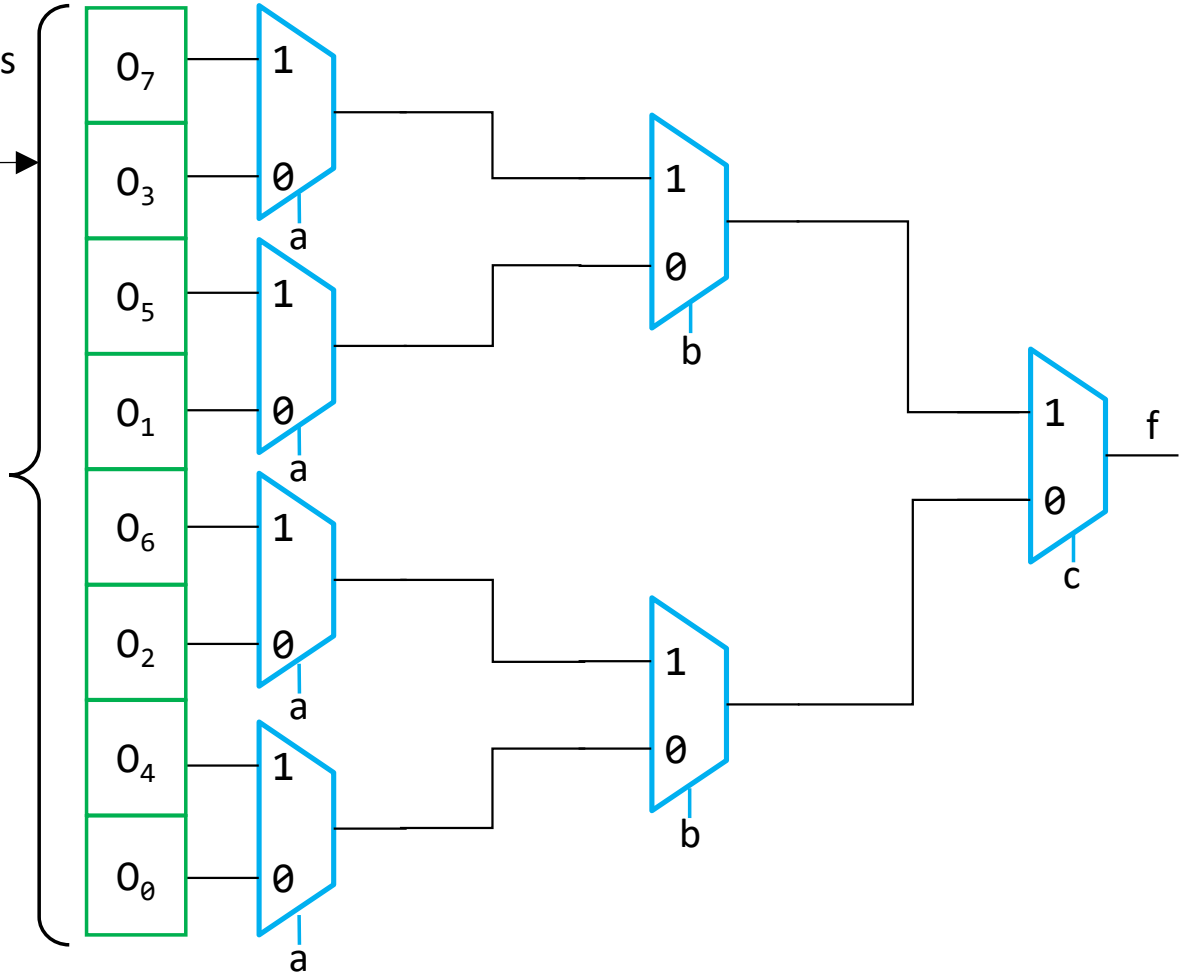
Mux Networks are the building blocks for FPGAs

This construct/design made up of storage elements
That holds the Output Values is called
Look Up Table or LUT

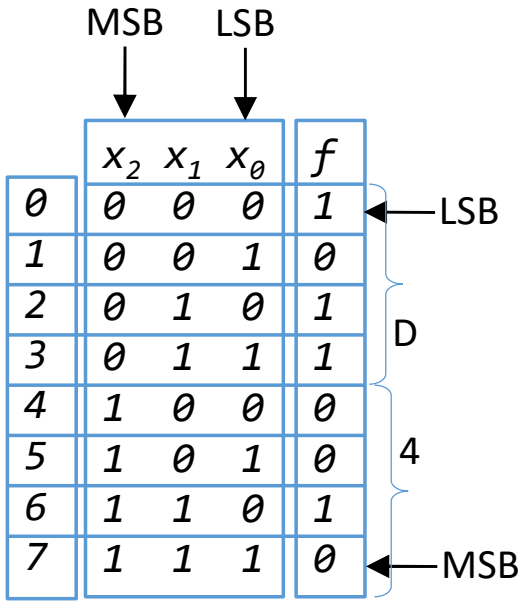
Field Programmable Gate Arrays (FPGAs)
Have millions of LUTs

LUTs can be configured to implement any
3 variable Boolean Functions

A network of wires connect these LUTs to enable implementation of more complex Boolean Functionalities
Some complex FPGAs have sufficient LUTs and wiring resources to implement 1000 processors

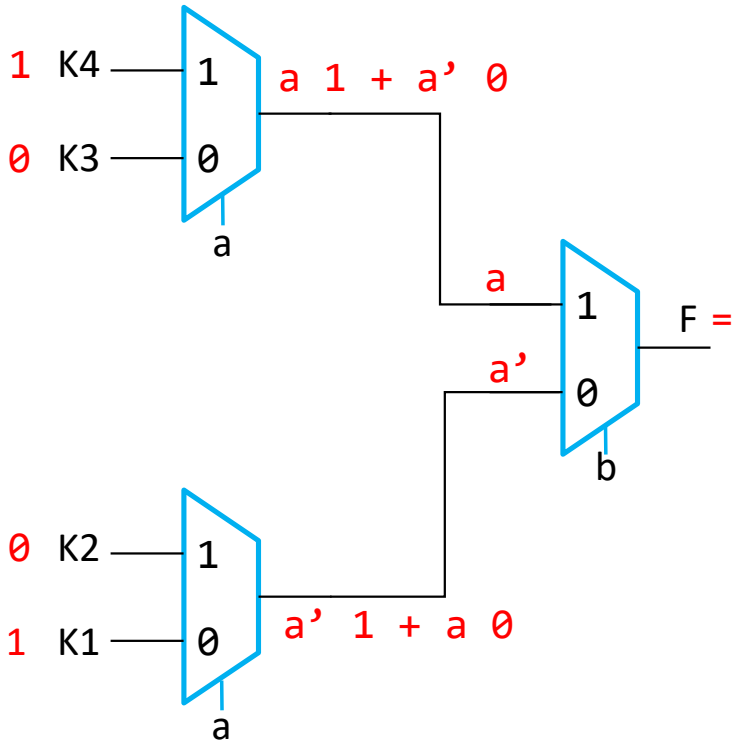


LUT – Order Dependent Function Specificaation



f = 0100 1101
= H'4D

Quiz Time



The generic mux structure on the left can implement arbitrary boolean functions in how many variables ?

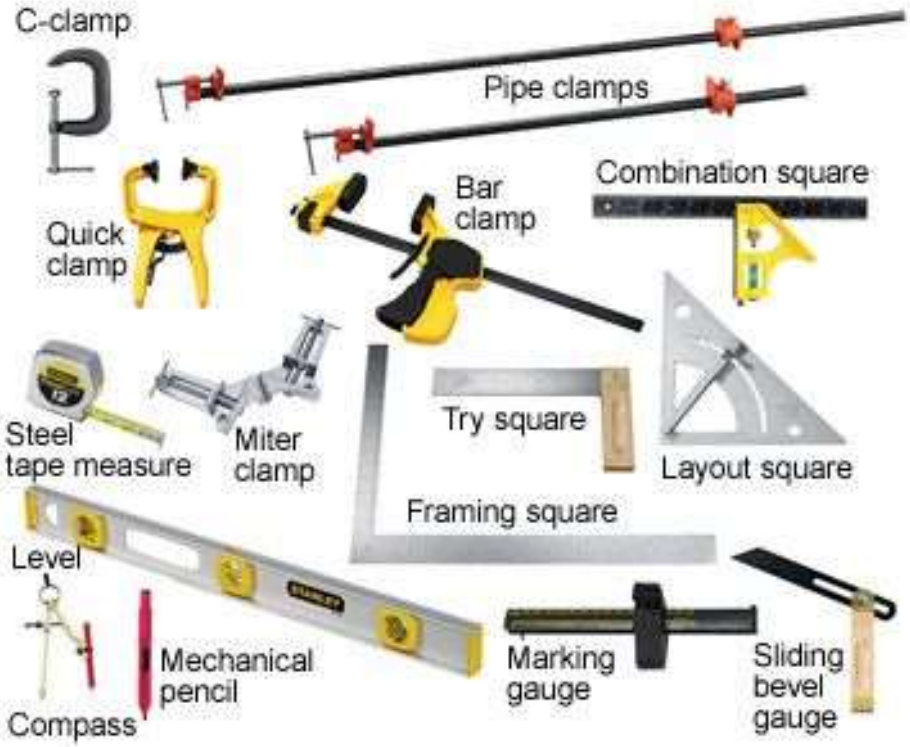
A. 1
B. 2
C. 3

To implement XNOR gate what should be value (0 or 1) at

K1:
K2:
K3:
K4:

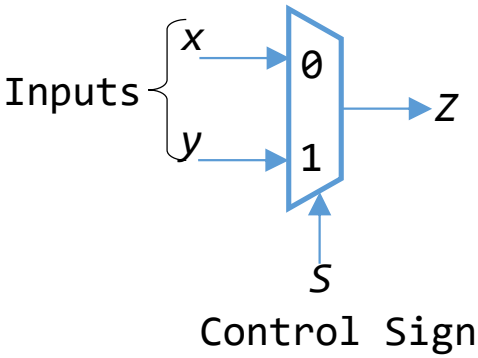
Combinatorial Datapath Building Blocks

1. ***Multiplexors – Muxes – Selectors***
2. ***Demultiplexors***
3. ***Decoders***
4. ***Encoders***
5. ***Comparators***
6. ***Shifters and Rotators***

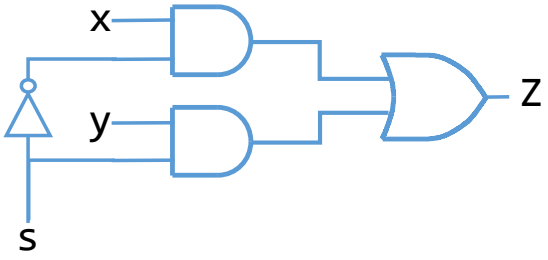


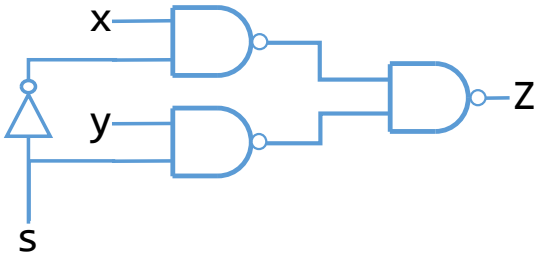
Multiplexors - MUXes

2-to-1 Multiplexor – MUX - Selector

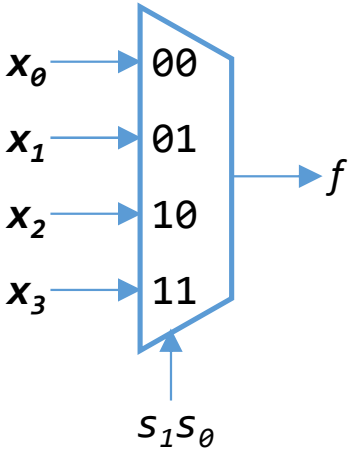


X	Y	S	Z
x	-	0	x
-	y	1	y



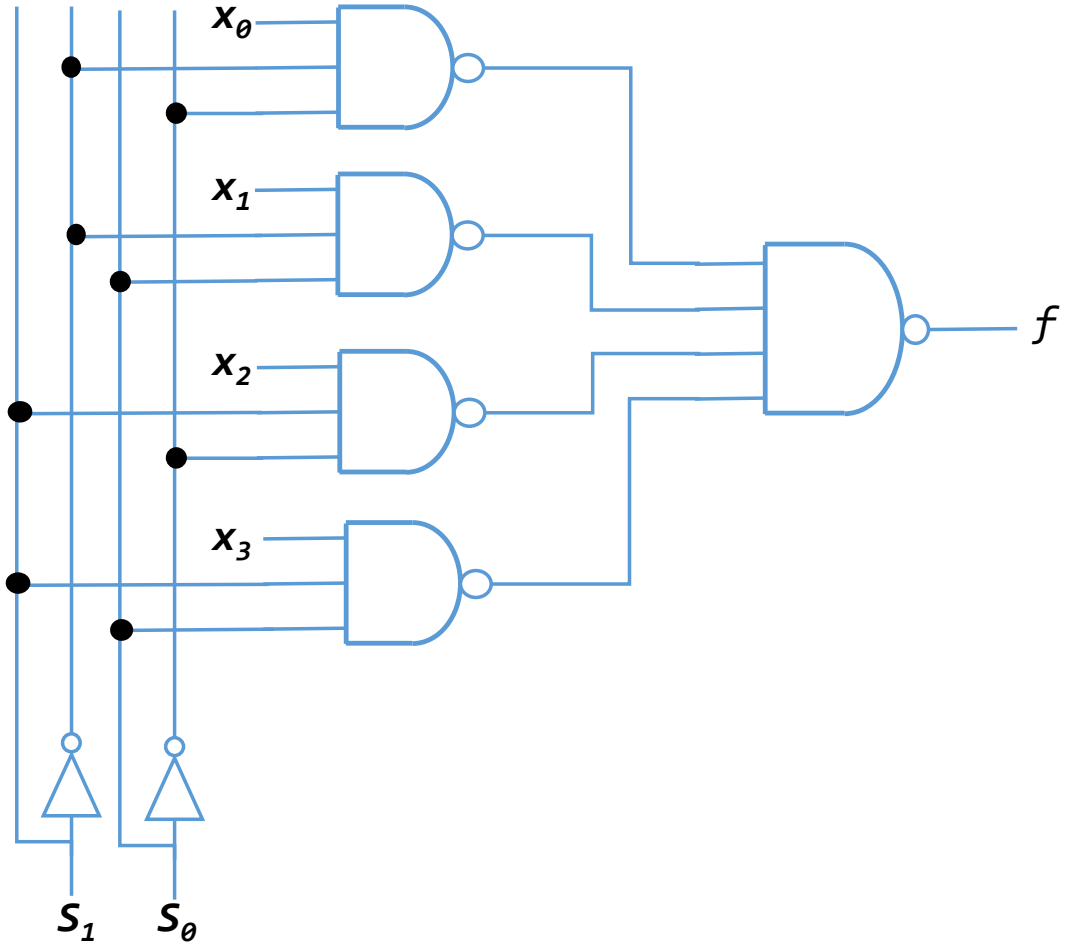


4-to-1 Multiplexor – MUX - Selector

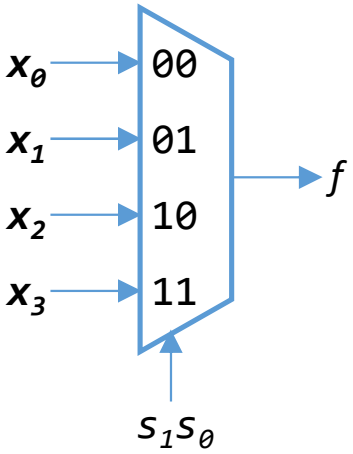


Control Signals

x_0	x_1	x_2	x_3	S_1S_0	f
x_0	-	-	-	0 0	x_0
-	x_1	-	-	0 1	x_1
-	-	x_2	-	1 0	x_2
-	-		x_3	1 1	x_3

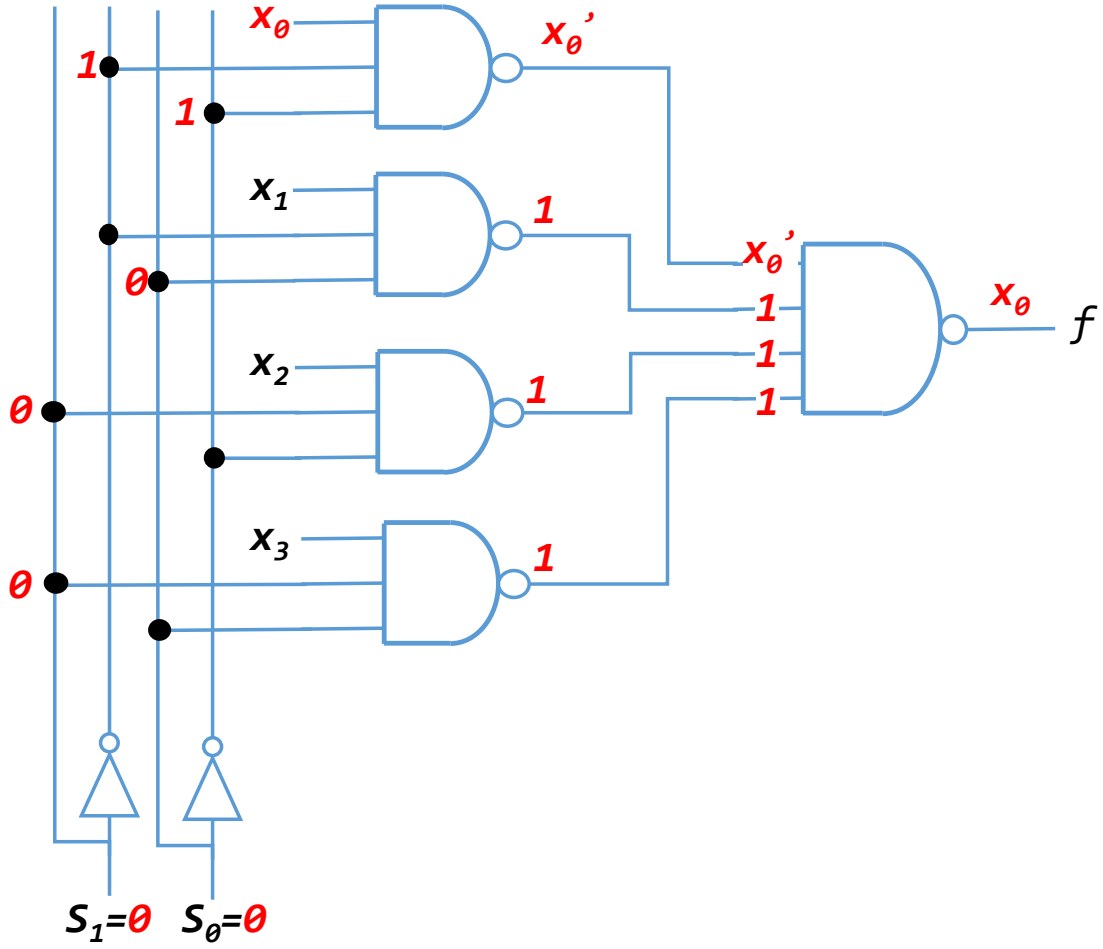


4-to-1 Multiplexor – MUX - Selector

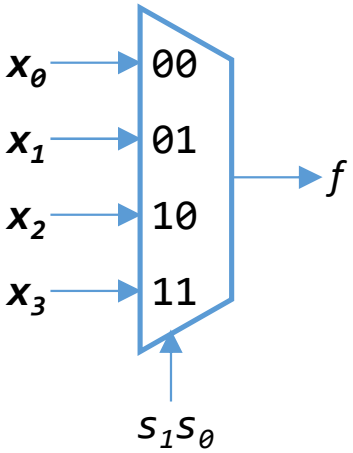


Control Signals

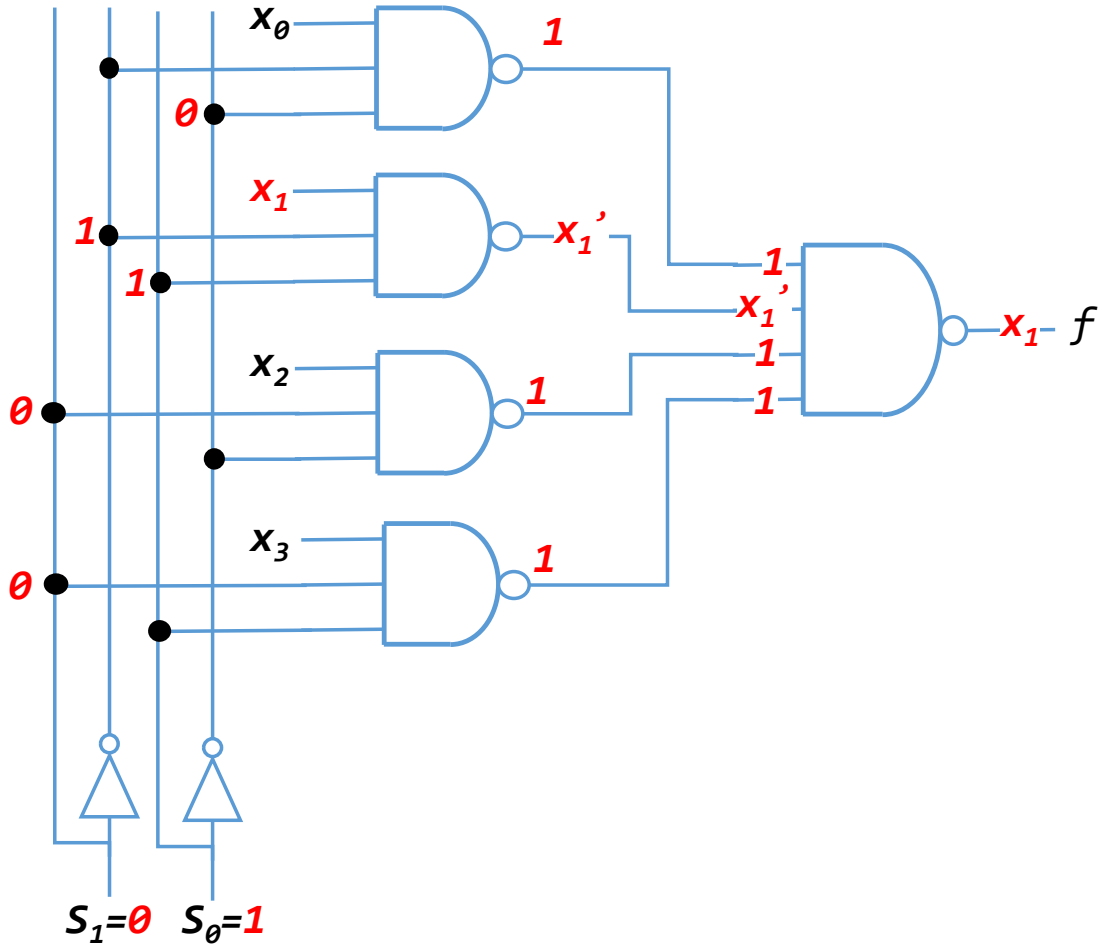
x_0	x_1	x_2	x_3	S_1S_0	f
x_0	-	-	-	0 0	x_0
-	x_1	-	-	0 1	x_1
-	-	x_2	-	1 0	x_2
-	-		x_3	1 1	x_3



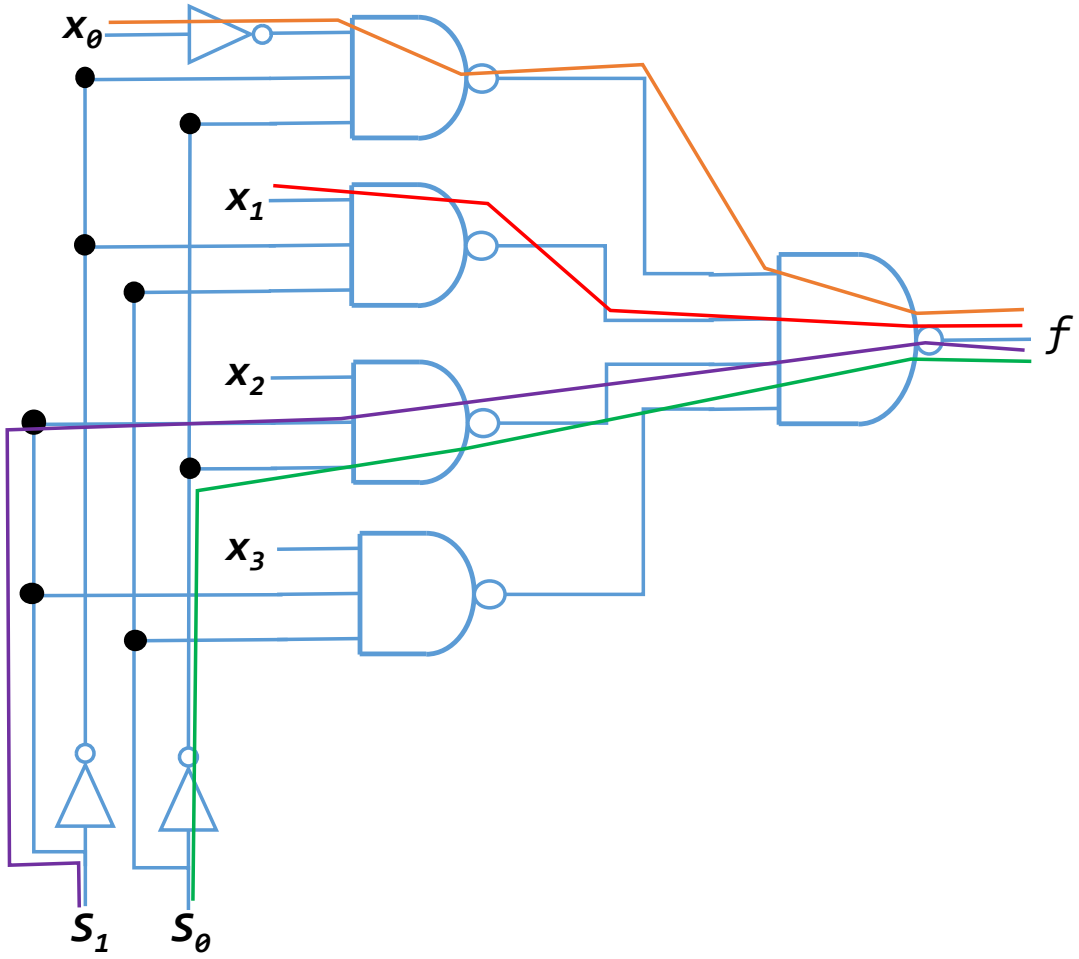
4-to-1 Multiplexor – MUX - Selector



x_0	x_1	x_2	x_3	S_1S_0	f
x_0	-	-	-	0 0	x_0
-	x_1	-	-	0 1	x_1
-	-	x_2	-	1 0	x_2
-	-		x_3	1 1	x_3



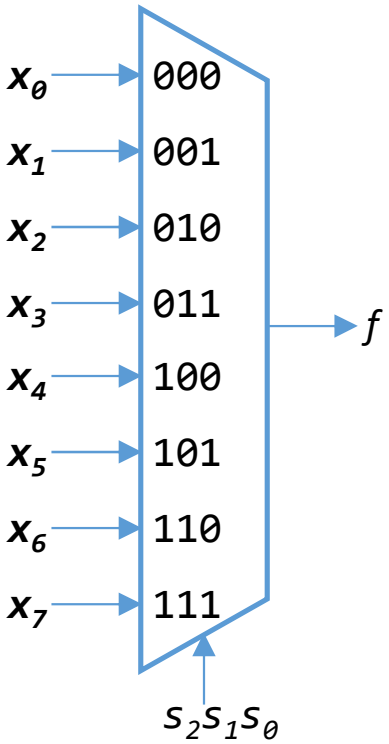
Quiz Time



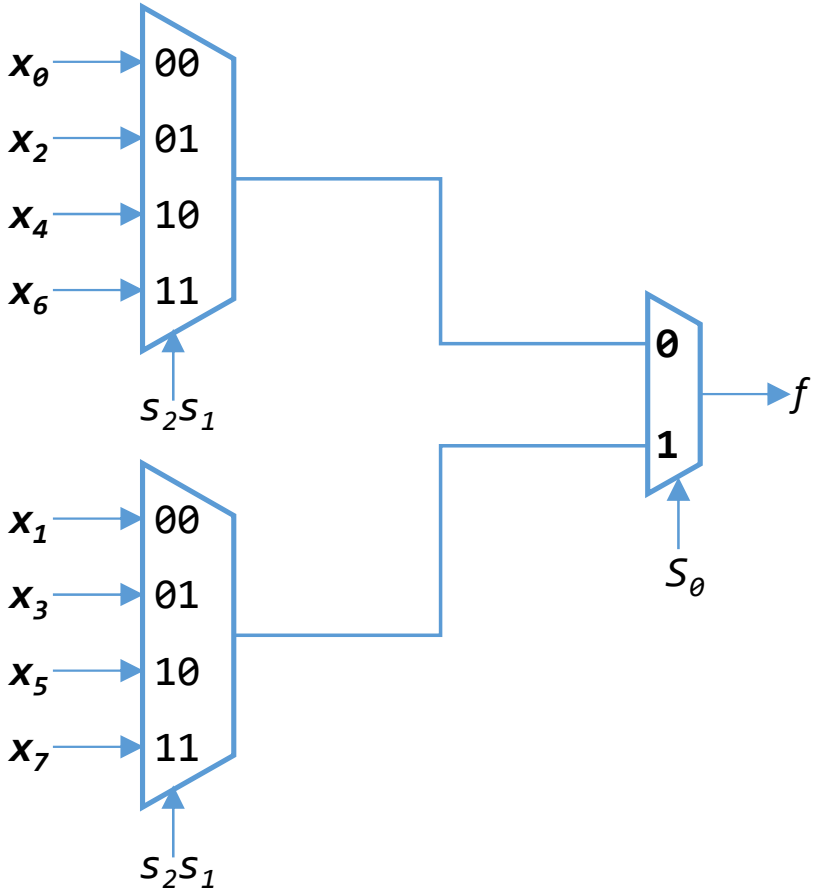
Order the four paths in terms of their delay

	Fastest			Slowest
A				
B				
C				
D				

8-to-1 Mux in terms of smaller Muxes



Control Signals



Implementing Multiplexors in SystemVerilog

```
module mux2to1_slice
  (   input  logic a,
      input  logic b,
      input  logic s,
      output logic y);

   logic    u1_out;
   logic    u2_out;
   logic    s_inv;

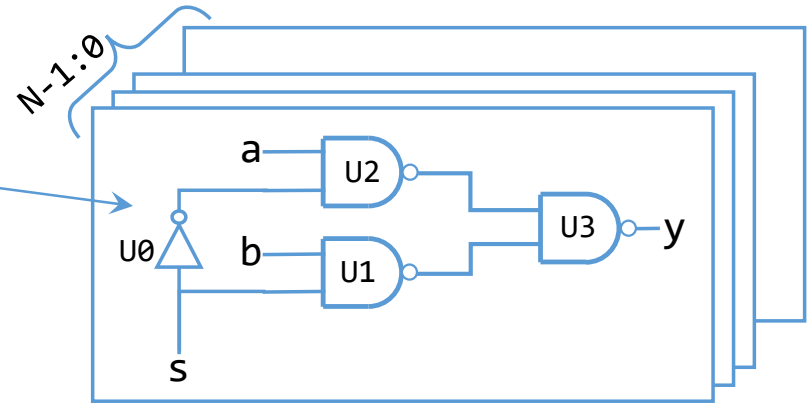
   nand2_delay U0 (.a(s),
                   .b(s),
                   .y(s_inv));

   nand2_delay U1 (.a(a),
                   .b(s),
                   .y(u1_out) );

   nand2_delay U2 (.a(b),
                   .b(s_inv),
                   .y(u2_out));

   nand2_delay U3 (.a(u1_out),
                   .b(u2_out),
                   .y(y) );

endmodule
```




```
module mux2to1_Nbit #(parameter N=1)
                  (   input  logic [N-1:0] A,
                      input  logic [N-1:0] B,
                      input  logic s,
                      output logic [N-1:0] Y);

    // Generate N bit 2-1 mux
    genvar i;
    generate
        for (i = 0; i < N; i++) begin : gen_2to1_mux
            mux2to1_slice ms (.a(A[i]),
                              .b(B[i]),
                              .s(s),
                              .y(Y[i]));
        end
    endgenerate
endmodule
```

```
module mux4to1_slice
                (   input  logic a,
                    input  logic b,
                    input  logic c,
                    input  logic d,
                    input  logic [1:0] s,
                    output logic y);

logic u1_out, u2_out, u3_out, u4_out;
logic s0, s1, s0_inv, s1_inv;

   assign s0 = s[0];
   assign s1 = s[1];

   nand2_delay inv0 (.a(s0),.b(s0),.y(s0_inv));
   nand2_delay inv1 (.a(s[1]),.b(s[1]),.y(s1_inv));

   nand3_delay U1 (.a(a),.b(s0_inv),.c(s1_inv),.y(u1_out));

   nand3_delay U2 (.a(b),.b(s0),.c(s1_inv),.y(u2_out));

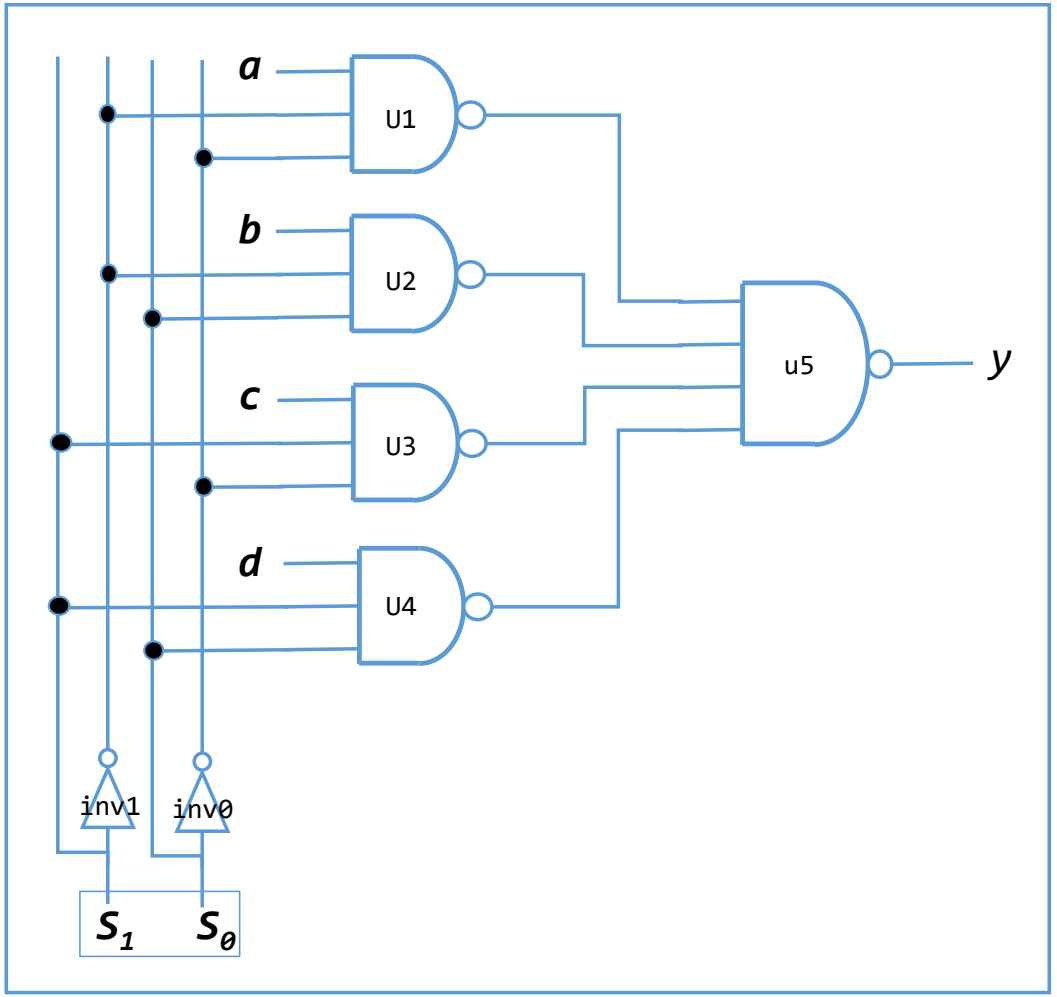
   nand3_delay U3 (.a(c),.b(s0_inv),.c(s1),.y(u3_out));

   nand3_delay U4 (.a(d),.b(s0),.c(s1),.y(u4_out));

   nand4_delay U5 (.a(u1_out),.b(u2_out),
                   .c(u3_out),.d(u4_out),.y(y));

endmodule
```

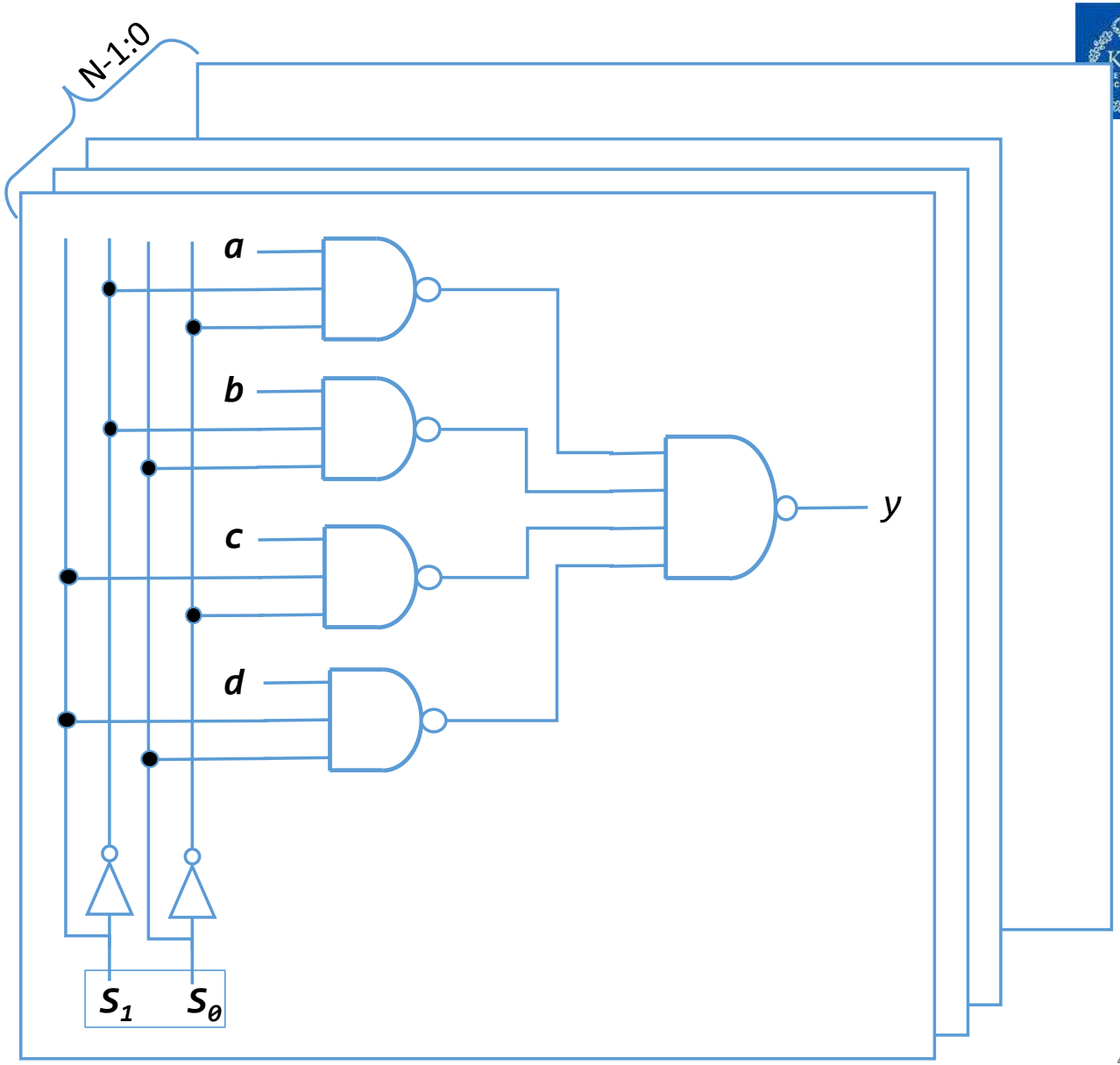
mux4to1_slice



```
module mux4to1_N_bit #(parameter N=4)
                (input  logic [N-1:0] A,
                 input  logic [N-1:0] B,
                 input  logic [N-1:0] C,
                 input  logic [N-1:0] D,
                 input  logic [1:0] sel,
                 output logic [N-1:0] Y
                );


    // Generate N bit 2-1 mux
    genvar i;
    generate
        for (i = 0; i < N; i++)
         begin : gen_4to1_mux
            mux4to1_slice ms (
                .a(A[i]),
                .b(B[i]),
                .c(C[i]),
                .d(D[i]),
                .s(sel),
                .y(Y[i])
            );
        end
    endgenerate

endmodule
```

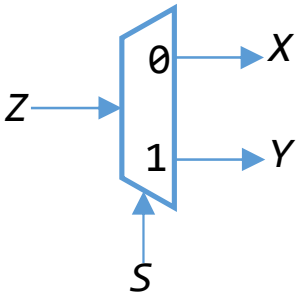


Suggestion for Lab Exercise

- Implement 8to1 Mux using 4to1 and 2to1 muxes
- Implement an N-bit RCA using muxes to Sum and Carry Out
- Compare area and delay with N-bit RCA created using NAND gates

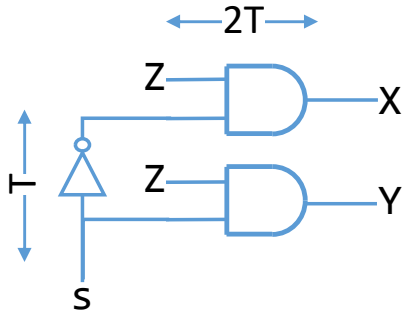
Demultiplexors - DEMUXes

2-to-1 Demultiplexor – DEMUX - Selector



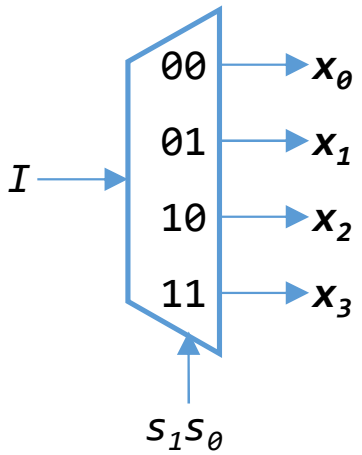
Select /Control Signal

S	Z	X	Y
0	X	X	0
1	Y	0	Y



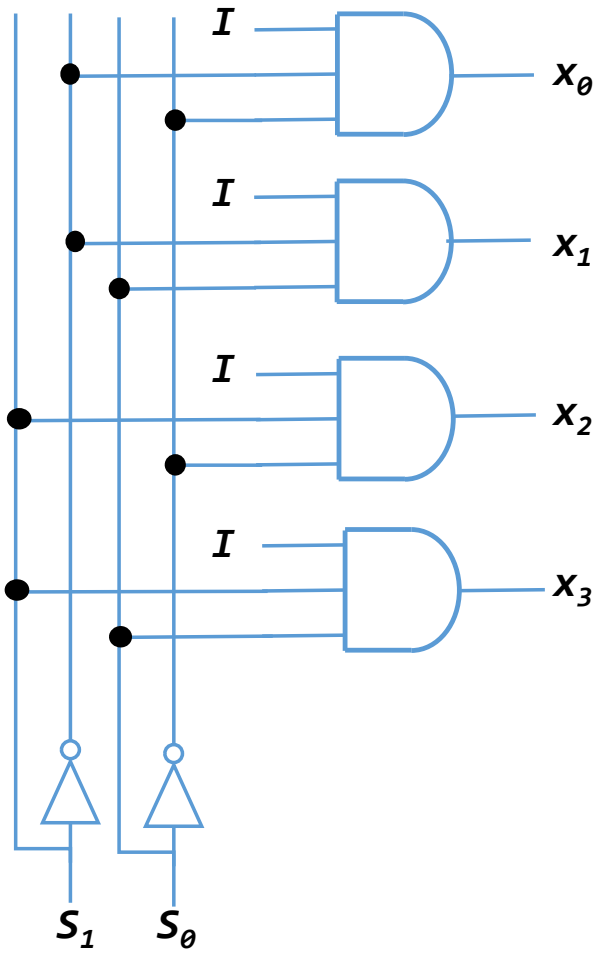
1. T is the delay of a two input NAND gate
2. 2T because a NAND gate will be followed by another NAND gate as an inverter
3. Area of each AND gate is 2A

4-to-1 Demultiplexor – DEMUX - Selector

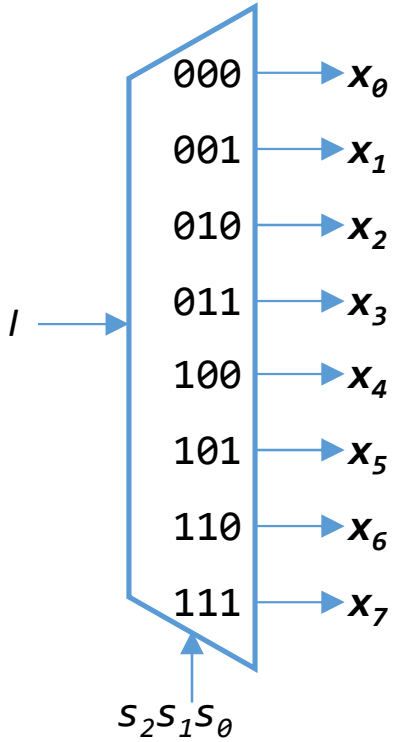


Control Signals

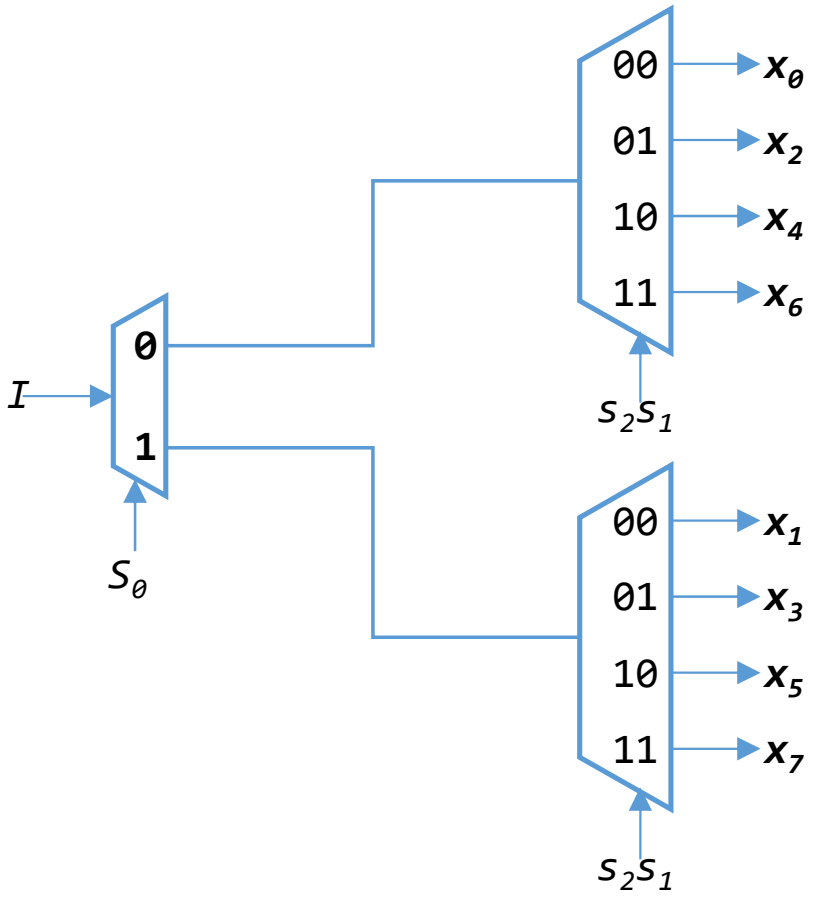
S_1S_0	I	x_0	x_1	x_2	x_3
0 0	I	I	0	0	0
0 1	I	0	I	0	0
1 0	I	0	0	I	0
1 1	I	0	0	0	I



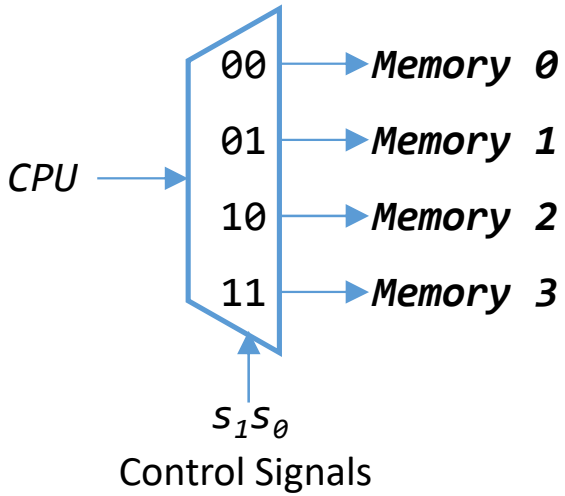
8-to-1 DEMUX in terms of smaller DEMUXes



Control Signals



What are Demux used for ?



Mux and DeMux are widely used when ever we need to serialize/parallelize



Telephone exchanges are in principle large scale mux and demux structures

In digital design within a chip, Mux and Demux are widely used to exchange data among many parallel computing and storage structures

We will also see Demux playing a role in Serial in Parallel Out (SIPO) Structures when we study sequential circuits

Lab Exercise Suggestion

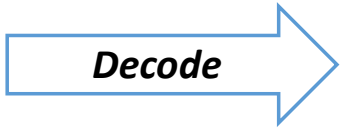
- Modify the Mux design to create the following Demux design
- N-bit 2to1 demux; first design a single bit 2to1 demux_slice
- Implement 4to1 demux_slice and N-bit 4to1 demux
- Implement N bit 8to1 demux using 4to1 and 2to1 muxes

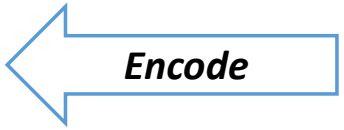
Decoders

Binary Patterns vs. One-hot Patterns

Binary Patterns

```
0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1
```





One-hot Pattern

```
0 0 0 0 0 0 0 1
0 0 0 0 0 0 1 0
0 0 0 0 0 1 0 0
0 0 0 0 1 0 0 0
0 0 0 1 0 0 0 0
0 0 1 0 0 0 0 0
0 1 0 0 0 0 0 0
1 0 0 0 0 0 0 0
```

N Patterns require $\lceil Log_2(N) \rceil$ bits

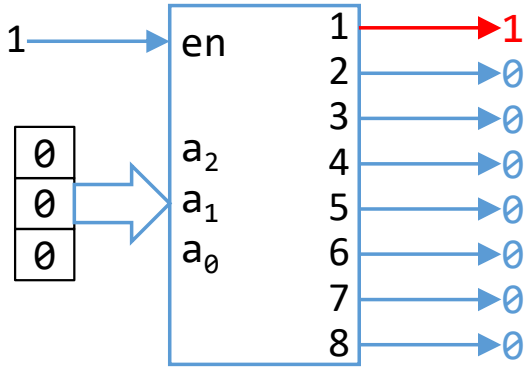
N Patterns require ***N*** bits

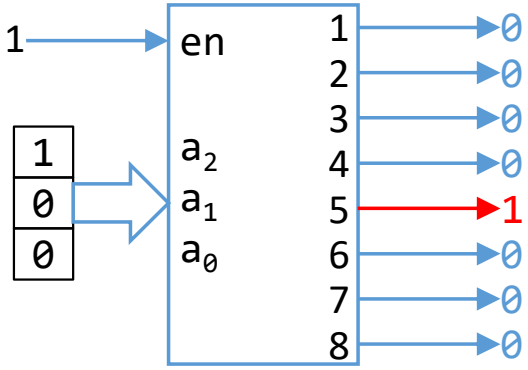
K bits = $\lceil Log_2(N) \rceil$

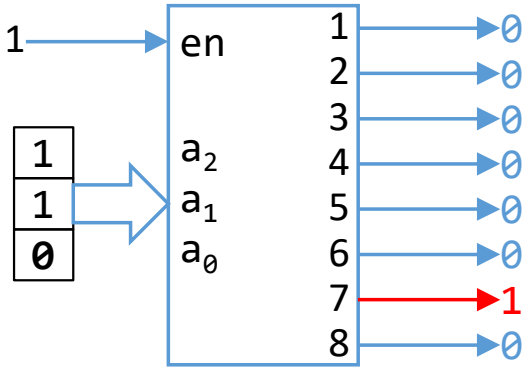


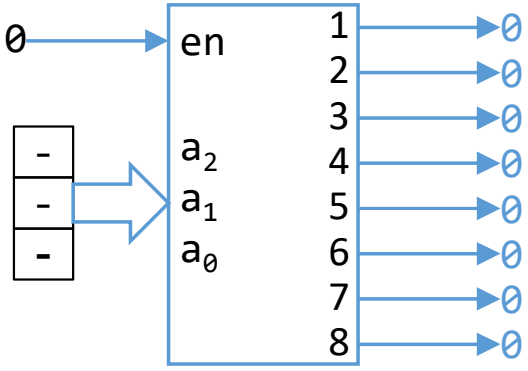
2^K one-hot patterns

3 to 8 decoder different operational modes



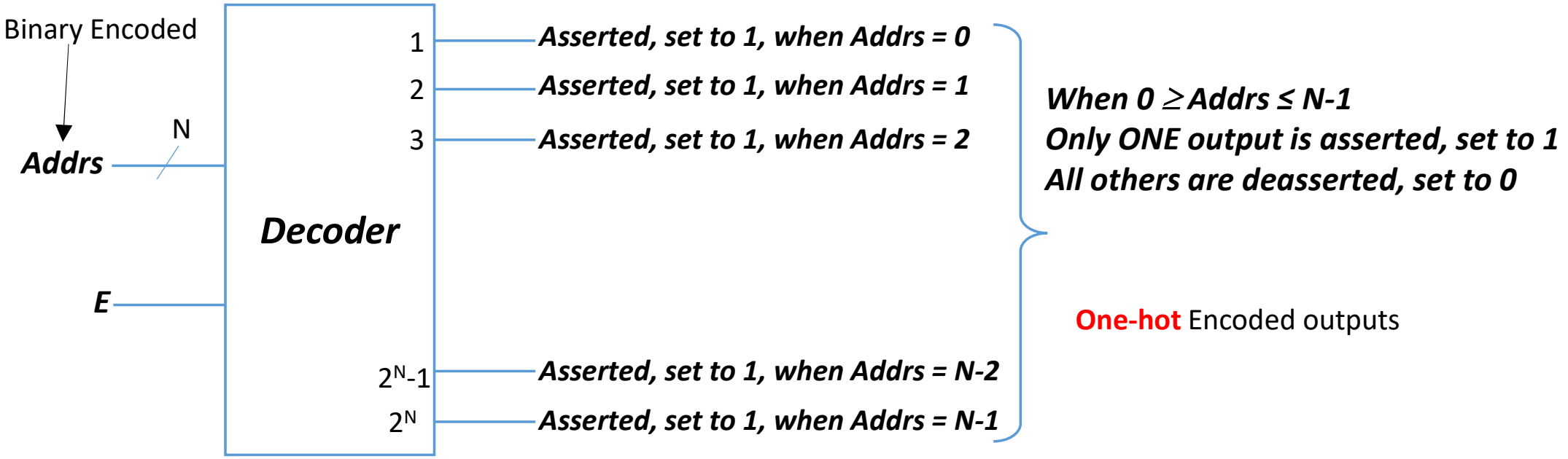






Decoders

Decoders decode a binary input to assert one-hot encoded outputs



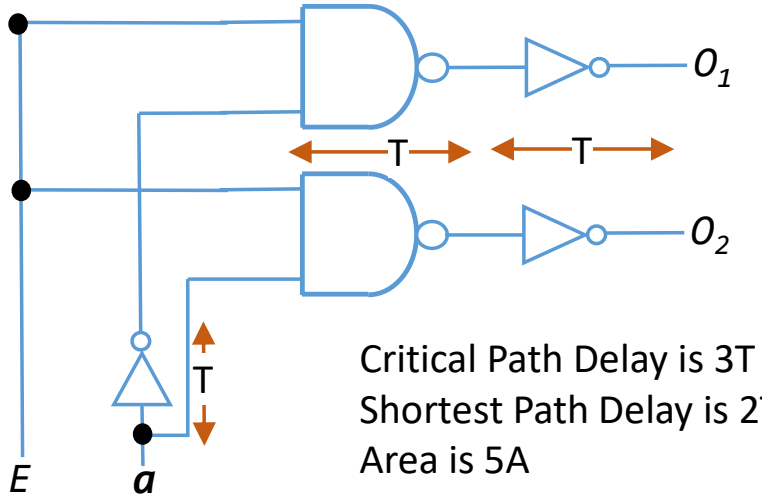
What do we do, when we want to deassert all the outputs ?
We have an extra input called enable signal (E)
When E = 1, the output corresponding to Addrs is asserted (set to 1)
When E = 0, all outputs are deasserted (set to 0)

1-to-2 Decoder

Binary Input

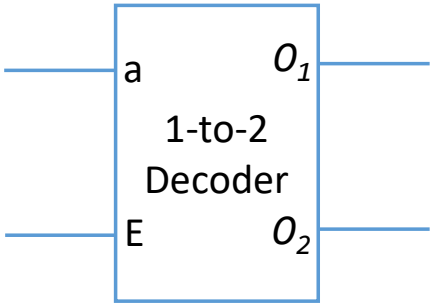
One-hot outputs

E	a	O_2	O_1
1	0	0	1
1	1	1	0
0	-	0	0



Critical Path Delay is 3T
Shortest Path Delay is 2T
Area is 5A

Symbol Diagram



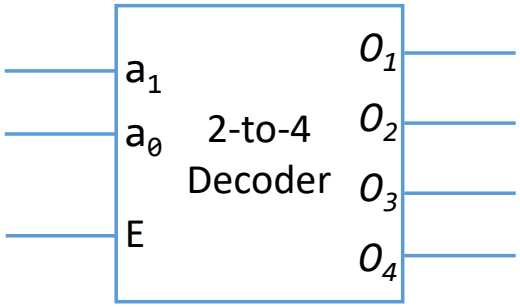
Pin	Direction	Type	Function
a	Input	Bit	The input whose value decides which output will be asserted
E	Input	Bit	The Enable signal that decides if any output will be asserted at all. Overrides the input signal. If ***E = 0***, no output is asserted. If ***E = 1***, the value of ***a*** decides which output will be asserted
O_1	Output	Bit	The output signal that is asserted when ***a = 0*** and ***E = 1***
O_2	Output	Bit	The output signal that is asserted when ***a = 1*** and ***E = 1***

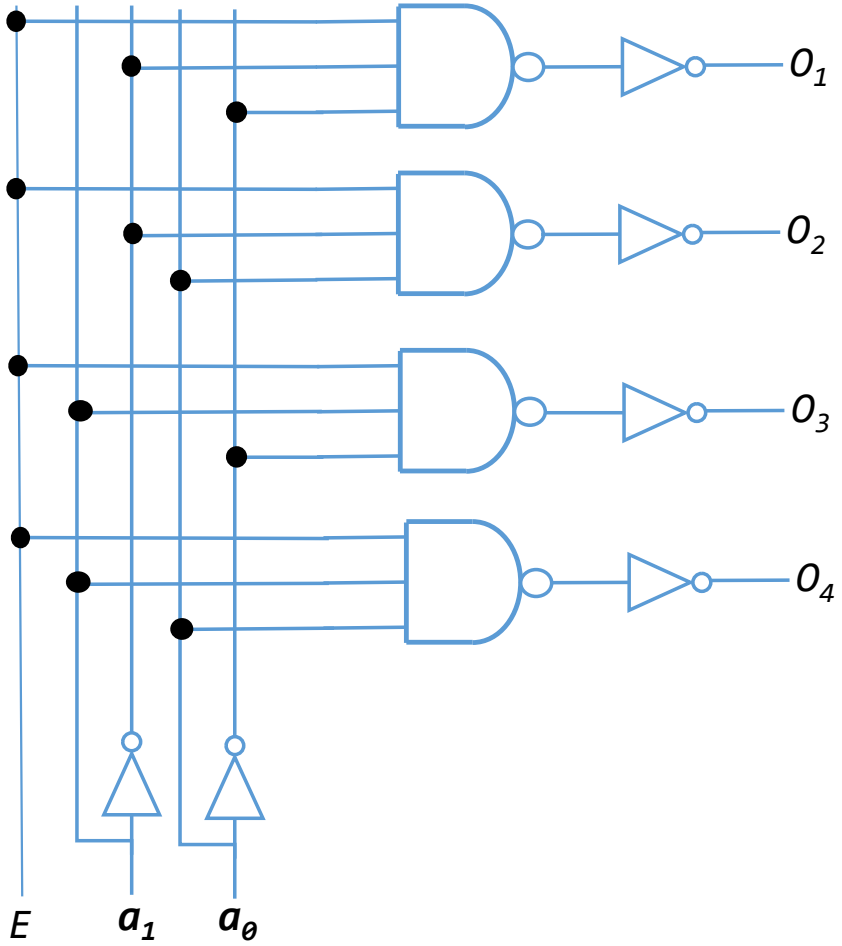
2-to-4 Decoder

Binary inputs

One-hot outputs

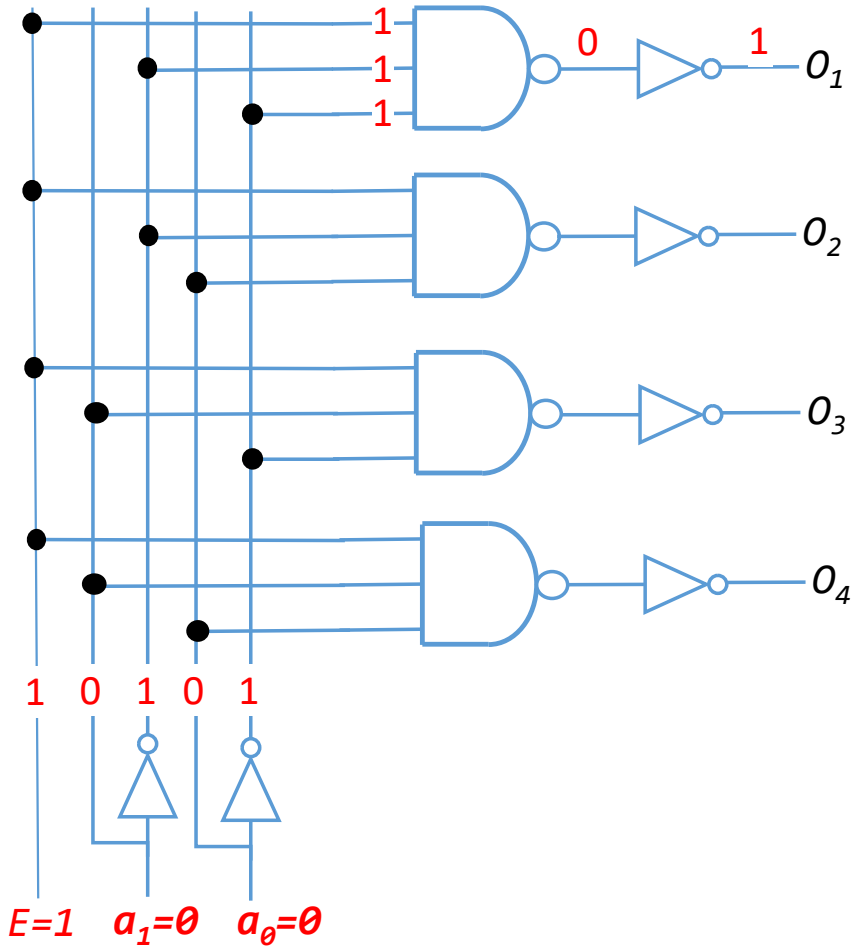
E	a_1	a_0	O_4	O_3	O_2	O_1
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0
0	-	-	0	0	0	0





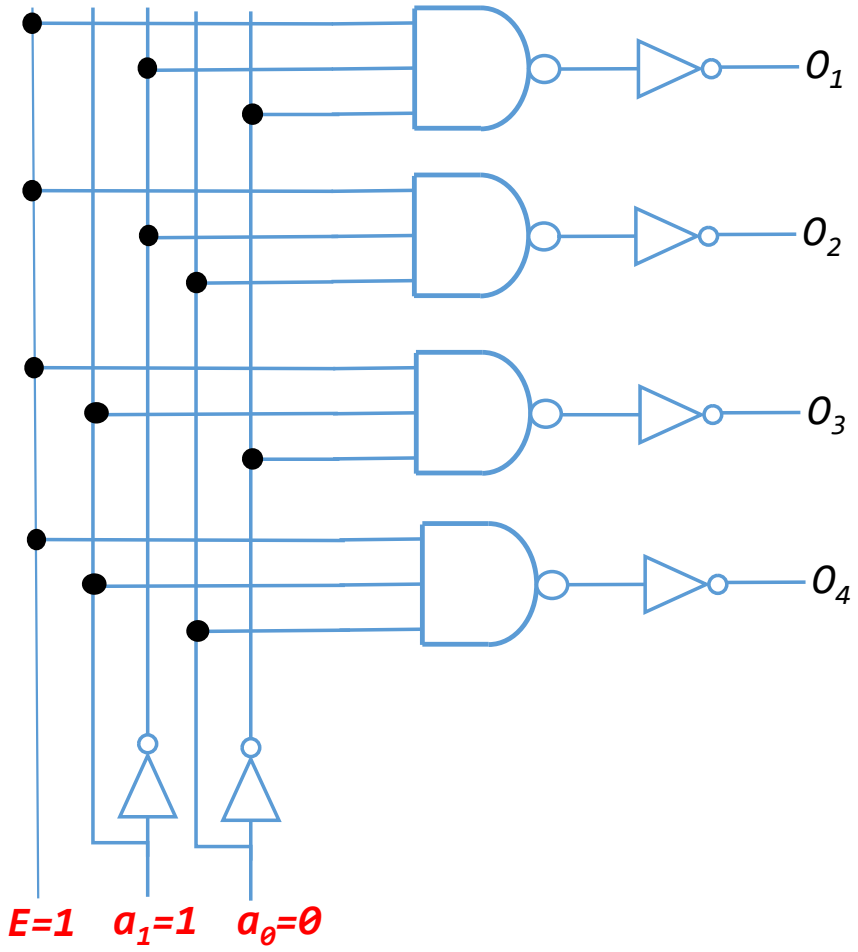
2-to-4 Decoder – Example 1

E	a_1	a_0	O_4	O_3	O_2	O_1
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0
0	-	-	0	0	0	0



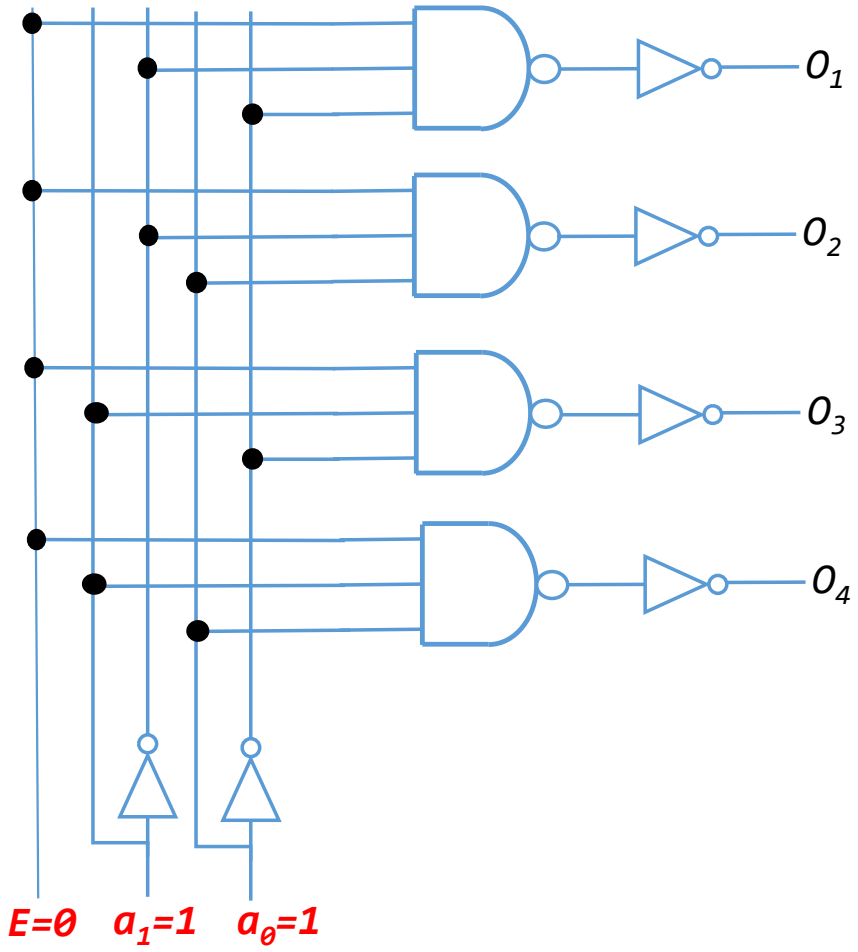
2-to-4 Decoder – Example 2

E	a_1	a_0	O_4	O_3	O_2	O_1
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0
0	-	-	0	0	0	0



2-to-4 Decoder – Example 3

E	a_1	a_0	O_4	O_3	O_2	O_1
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0
0	-	-	0	0	0	0



```
module decoder2to4 (
    input logic [1:0] A,    // 2-bit input
    input logic       en,   // Enable input
    output logic [3:0] Y    // 4-bit output
);

  // Intermediate signals for the NOT gates
  logic a0_inv, a1_inv;
  logic a0, a1;

    assign a0 = A[0];
    assign a1 = A[1];

    // Inverting the inputs
    nand2_delay inv0 (.a (A[0]), .b (A[0]), .y (a0_inv));

    nand2_delay inv1 (.a(A[1]),  .b(A[1]),  .y (a1_inv));

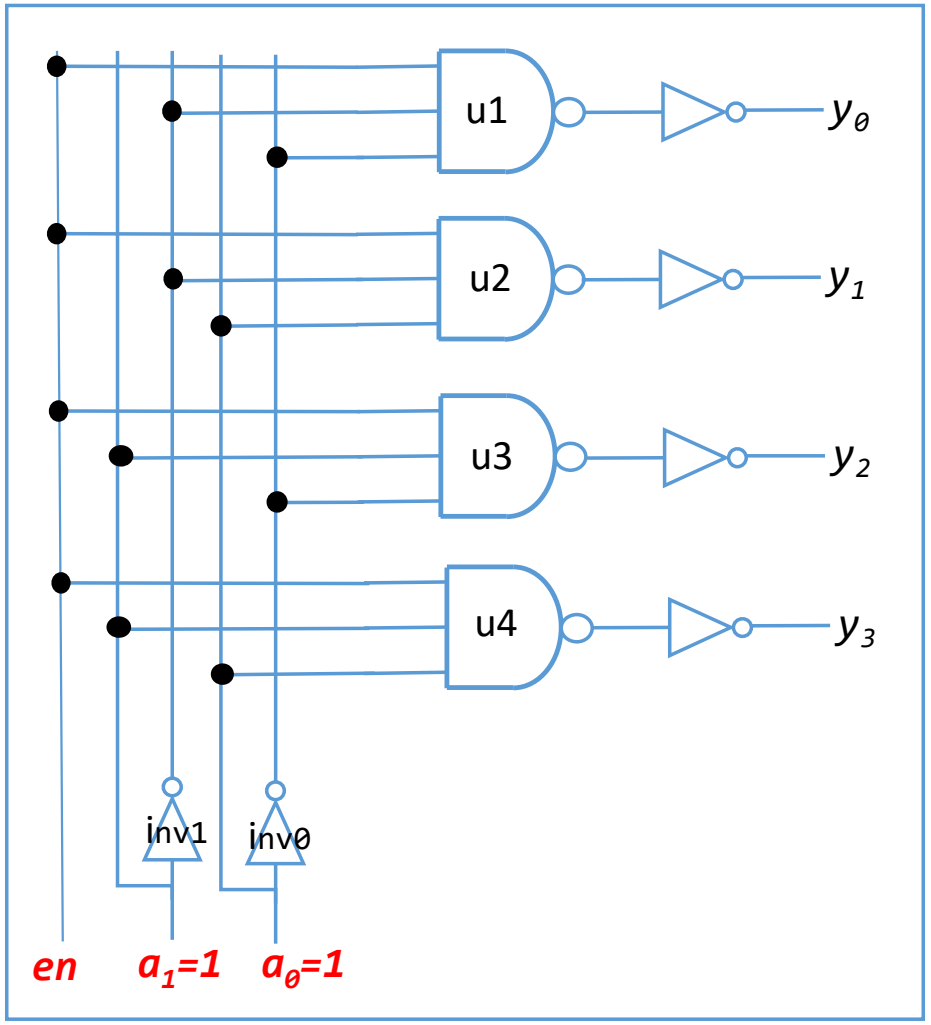
  // AND gates to implement the 2-to-4 decoder logic with enable
    and3_delay U1 (.a(en), .b(a1_inv),  .c(a0_inv),  .y(Y[0]));

    and3_delay U2 (.a(en), .b(a1_inv),  .c(a0),      .y(Y[1]));

    and3_delay U3 (.a(en), .b(a1),      .c(a0_inv),  .y(Y[2]));

    and3_delay U4 (.a(en), .b(a1),      .c(a0),      .y(Y[3]));

endmodule
```



E	a_1	a_0	O_4	O_3	O_2	O_1
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0
0	-	-	0	0	0	0

```
`timescale 1ns / 1ps

module tb_decoder2to4;

  logic [1:0] A;
  logic en;
  logic [3:0] Y;

  // Instantiate the decoder
  decoder2to4 uut (.A(A), .en(en), .Y(Y));

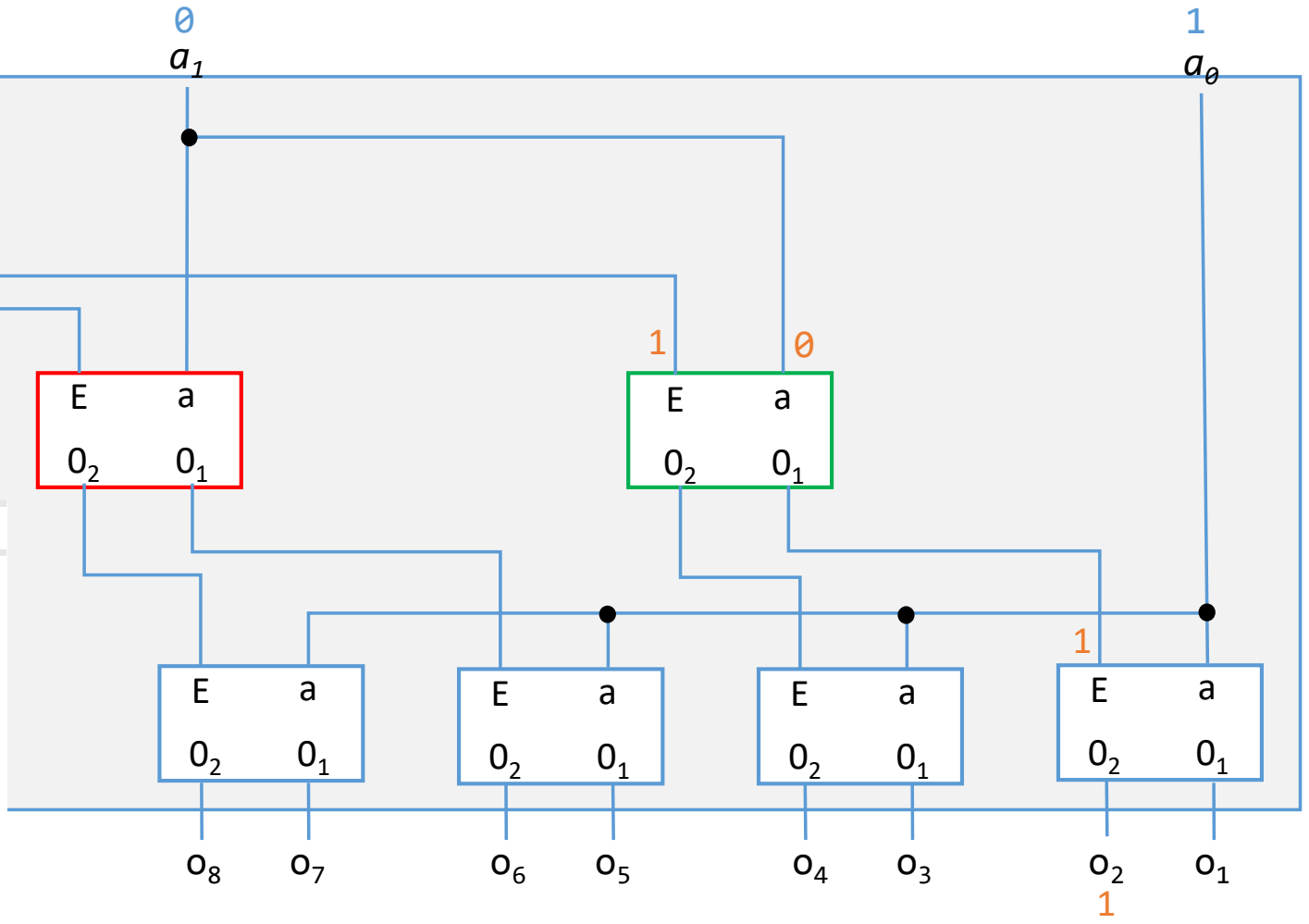
  initial begin
    // Test when enable is 0 (all outputs should be 0)
    en = 0;
    for (int i = 0; i < 4; i++) begin
      A = i;
      #10;
    end

    // Test when enable is 1 (outputs should reflect the decoded input)
    en = 1;
    for (int i = 0; i < 4; i++) begin
      A = i;
      #10;
    end

    $stop;
  end

endmodule
```

3-to-8 Decoder using 1-to-2 Decoders



E	a_2	a_1	a_0	o_8	o_7	o_6	o_5	o_4	o_3	o_2	o_1
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0
0	-	-	-	0	0	0	0	0	0	0	0

Quiz Time

If inputs are
E =1, a_2 = 1, a_1 = 0 and a_0 =1

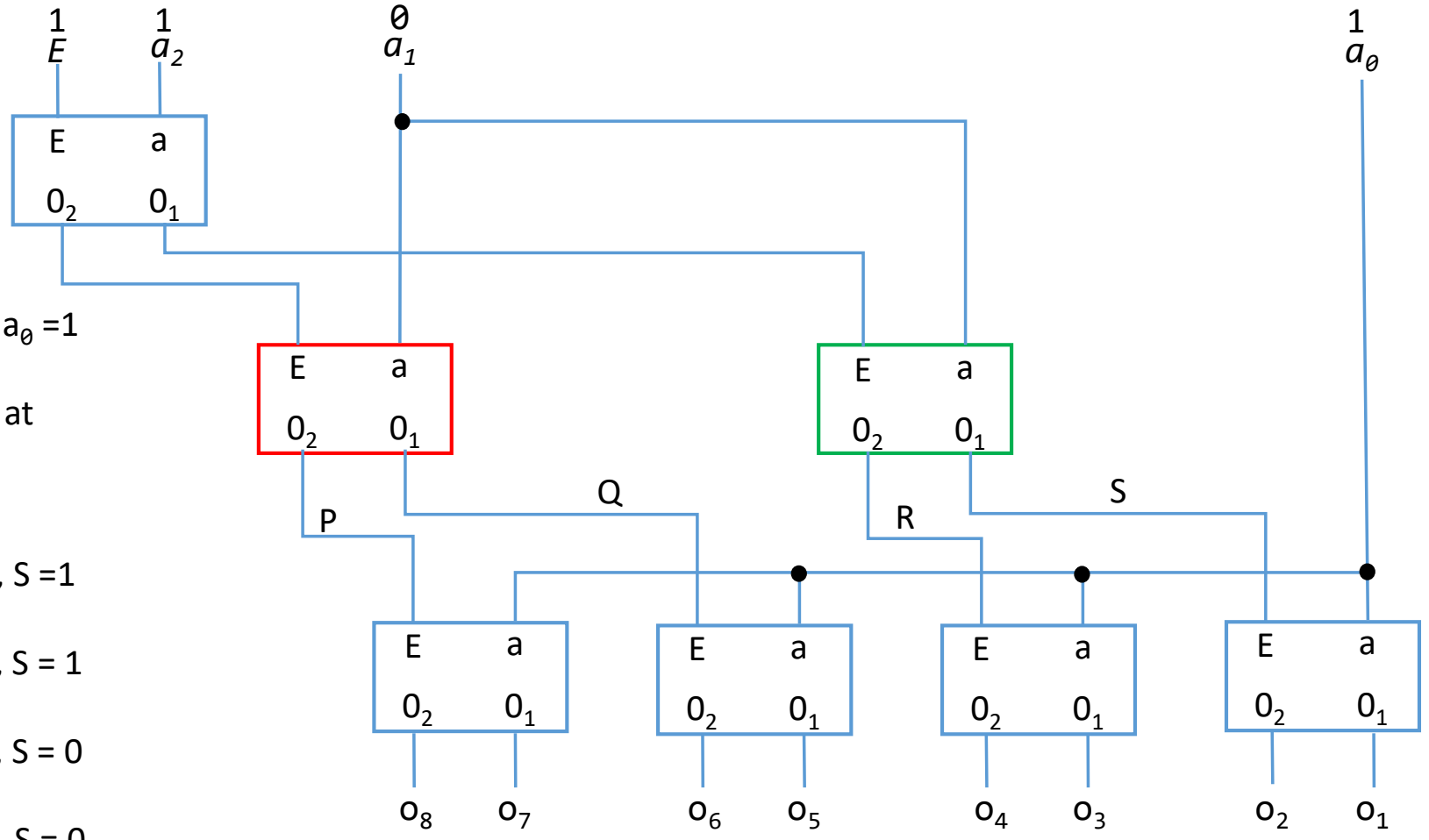
What will be the value at
P, Q, R and S

A: P = 1, Q = 1, R = 0, S =1

B: P = 0, Q = 1, R = 0, S = 1

C: P = 1, Q = 0, R = 0, S = 0

D: P = 0, Q = 1, R = 0, S = 0



Lab Exercise Suggestion

- Implement a 1-to-2 decoder with an enable signal
- Implement a 3-to-8 decoder with an enable signal using 1-to-2 decoders as the building blocks as shown in the slides; adapt the TB used for 2-to-4 decoder

Tri State

Tri State

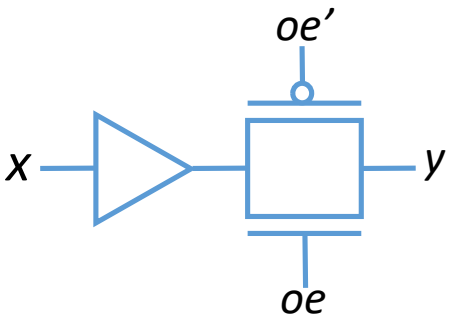
1. All nodes (input and output) of digital logic occupy, ideally logic 1 or logic 0 state.
2. These states imply that the node in logic is driven by V_{DD} or connected to GND (logic 0)
3. However, in some cases, a node in the logic may be not connected to either V_{DD} or GND.
4. This may sound as an anomaly (abnormality), however in reality this is desirable and is very useful.
5. This *third state is often called the high impedance state* because there is a high impedance between the node and the V_{DD} / GND
6. This third state is represented by *Z*
7. Logic gates that can intentionally drive its output to the *Z* state are called tri-stated

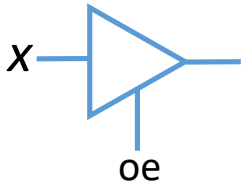
oe: output enable

x	*oe*	*y*
x	*1*	*x*
-	*0*	*Z*

y = x when oe = 1

y = Z when oe = 0; y is disconnected from x





y = x when oe = 1
else Z

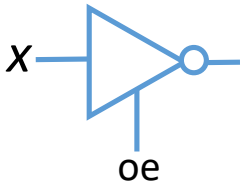
Inverting Tri State

oe: output enable

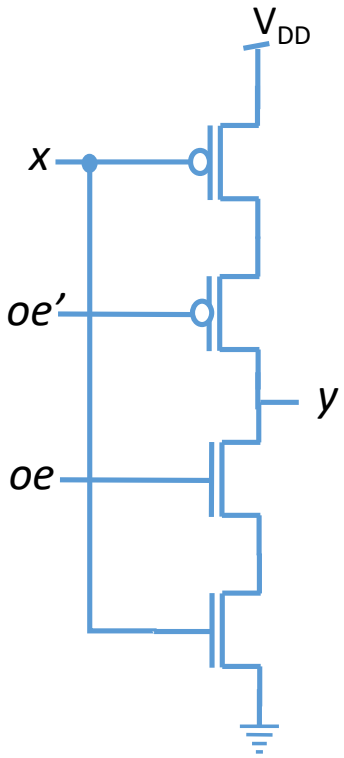
x	oe	y
x	1	x’
-	0	Z

y = x’ when oe = 1

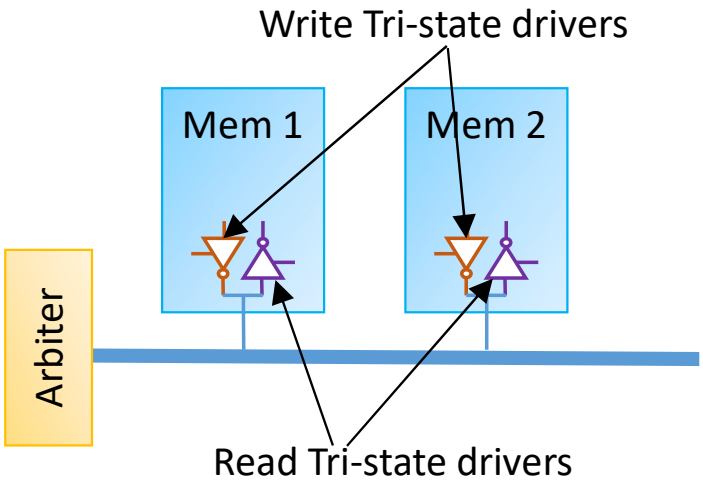
y = Z when oe = 0; y is disconnected from x



y = x’ when oe = 1
else Z



What are tri state buffers/drivers used for ?

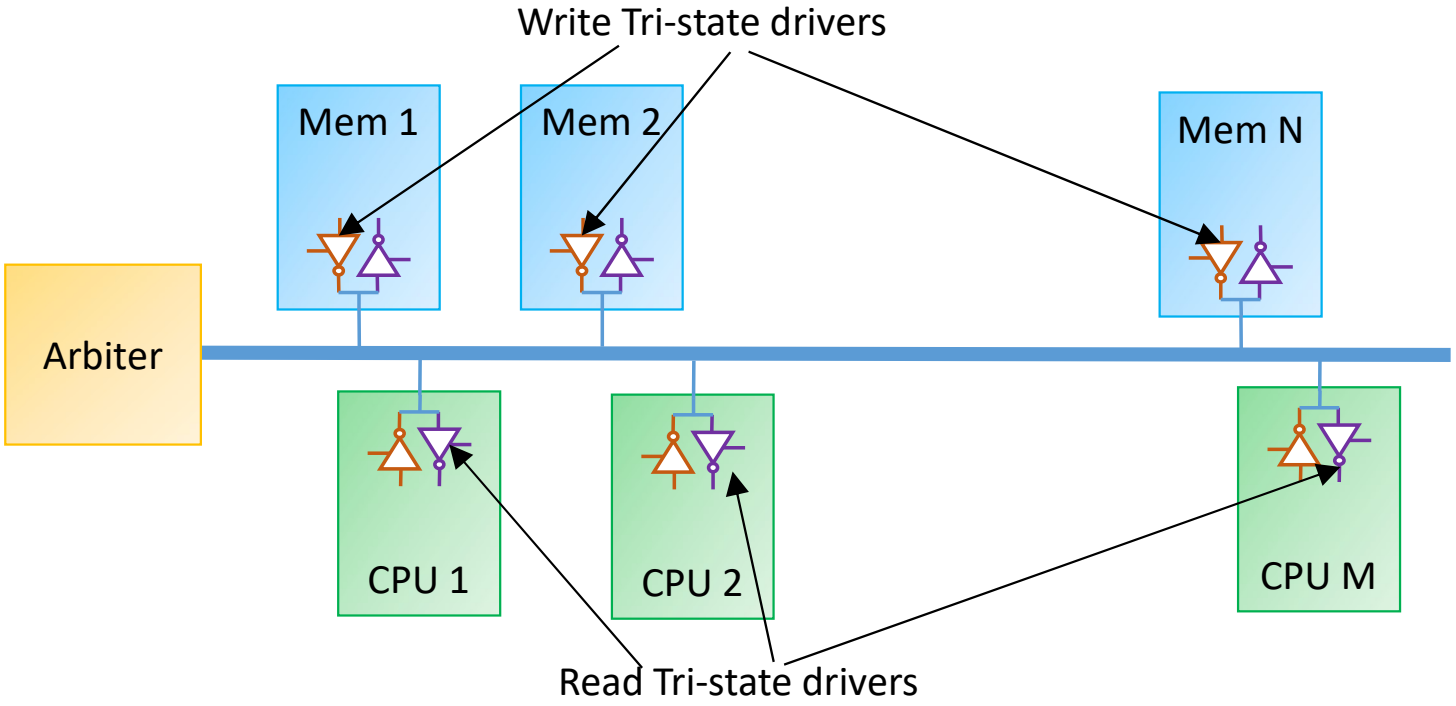


1. Let us say two Memory Devices share a bus (a bundle of wires)
2. Both memory devices can read from the bus at the same time
3. Both memory devices cannot however write to the bus at the same time

   One could be wanting to write logic 1 and ther other could be wanting write logic 0. This would create a short circuit between V_{DD} and GND
4. Tri-state buffers / drivers help because we can disable one of the driver, while the other is writing
5. Usually there is a device called arbiter that grants access to the bus before it can write to it.
6. If all devices can read, why do we need ***read*** tri-state drivers ?
7. This is because if Mem 1 device is writing to the bus the read tri-state driver in Mem 1 is disabled. What Mem 1 is writing is meant for external devices.

What are tri states used for ?

Tri States are used when we need bi-directional connections
When a device or a sub-system what to read and write from the same port

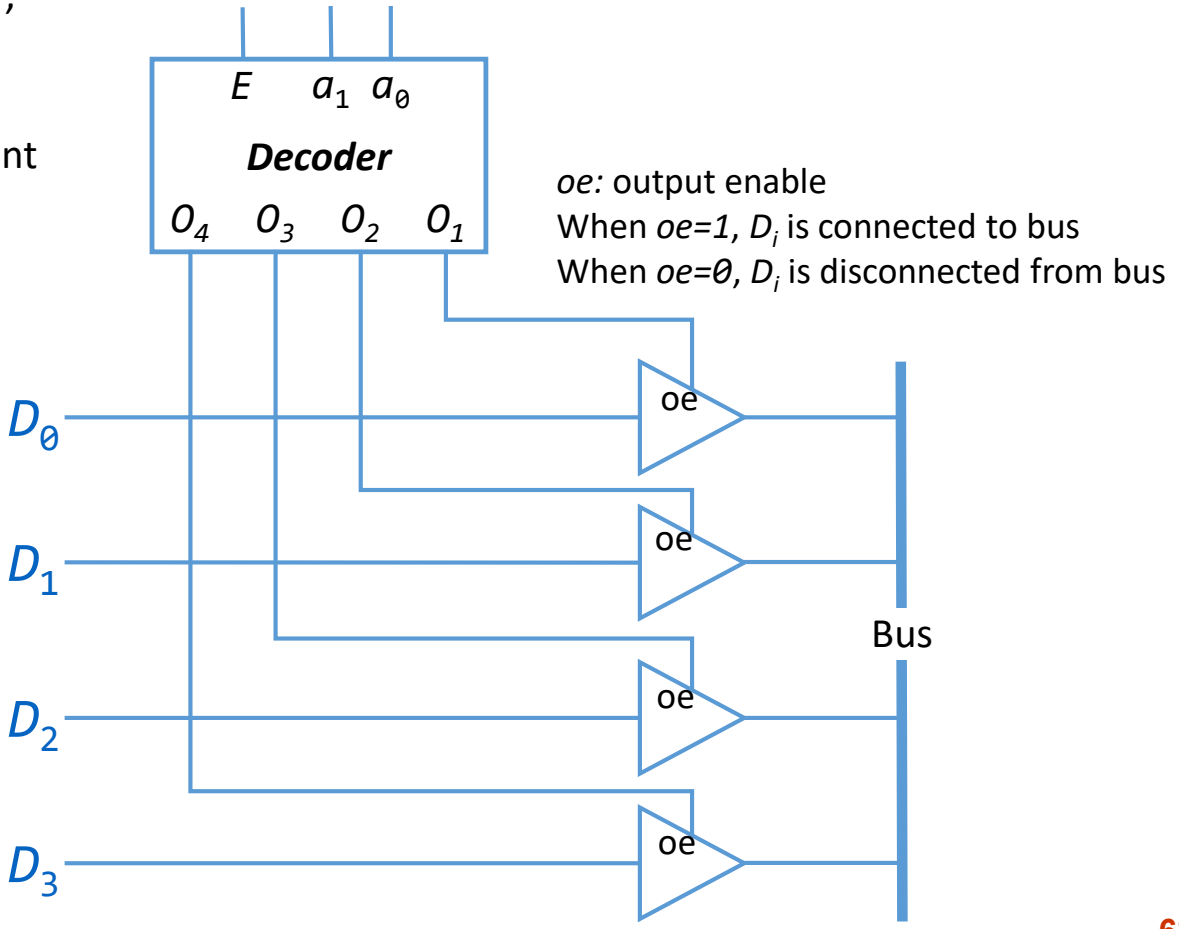


Only one device is allowed to drive, i.e., write to the bus. The output enable of only one output tri-state driver is activated at a time.
All others are inactivated
Multiple devices can read.

Bus Implementation – Write Logic Using Decoder

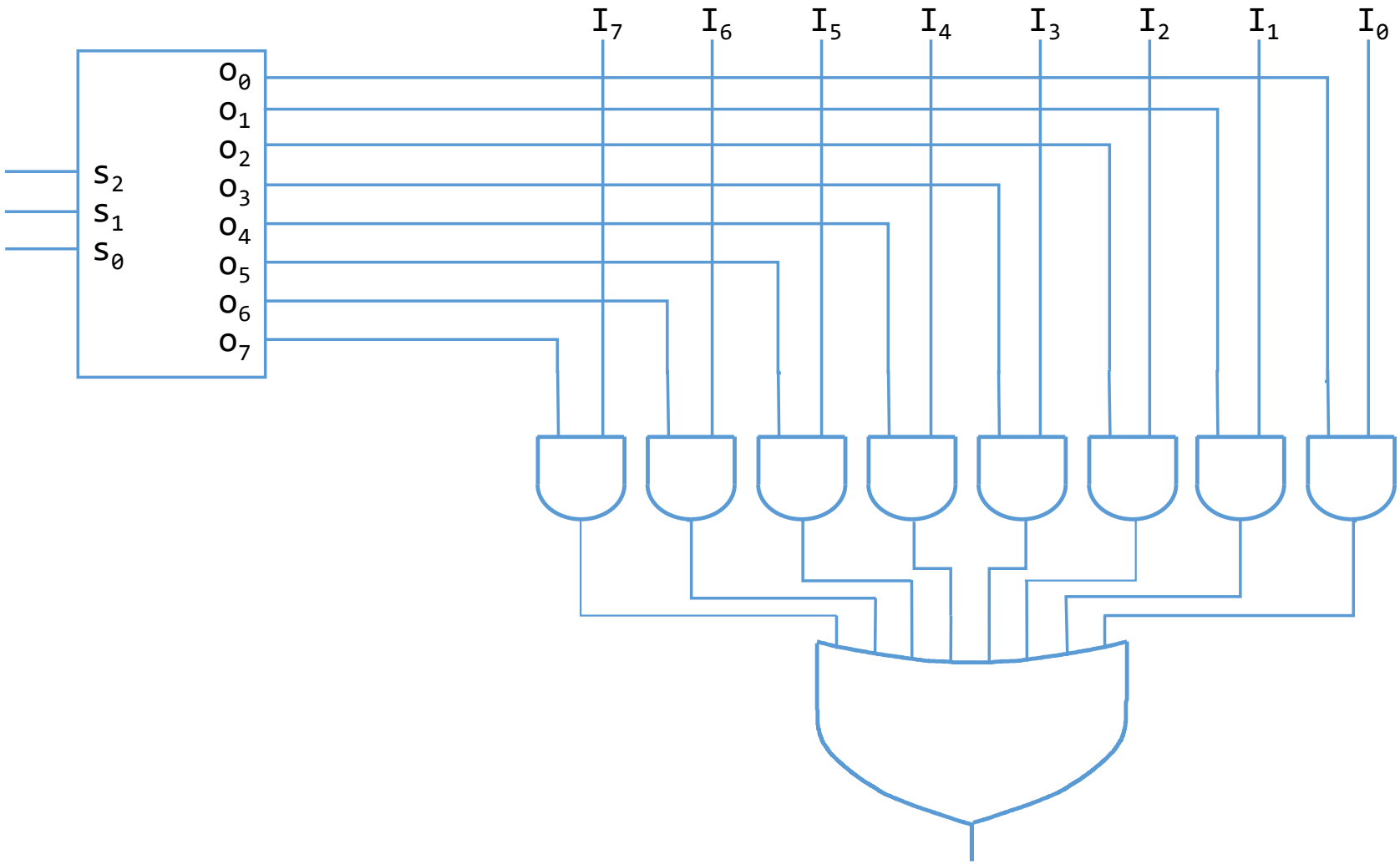
- Bus drivers have three possible output values: 0, 1, and Z (High Impedance ≈ Disconnection)
- Tri-stated buses provide similar functionality as multiplexors. The two solutions are used in different scenarios.

E	a_1	a_0	O_4	O_3	O_2	O_1
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0
0	-	-	0	0	0	0

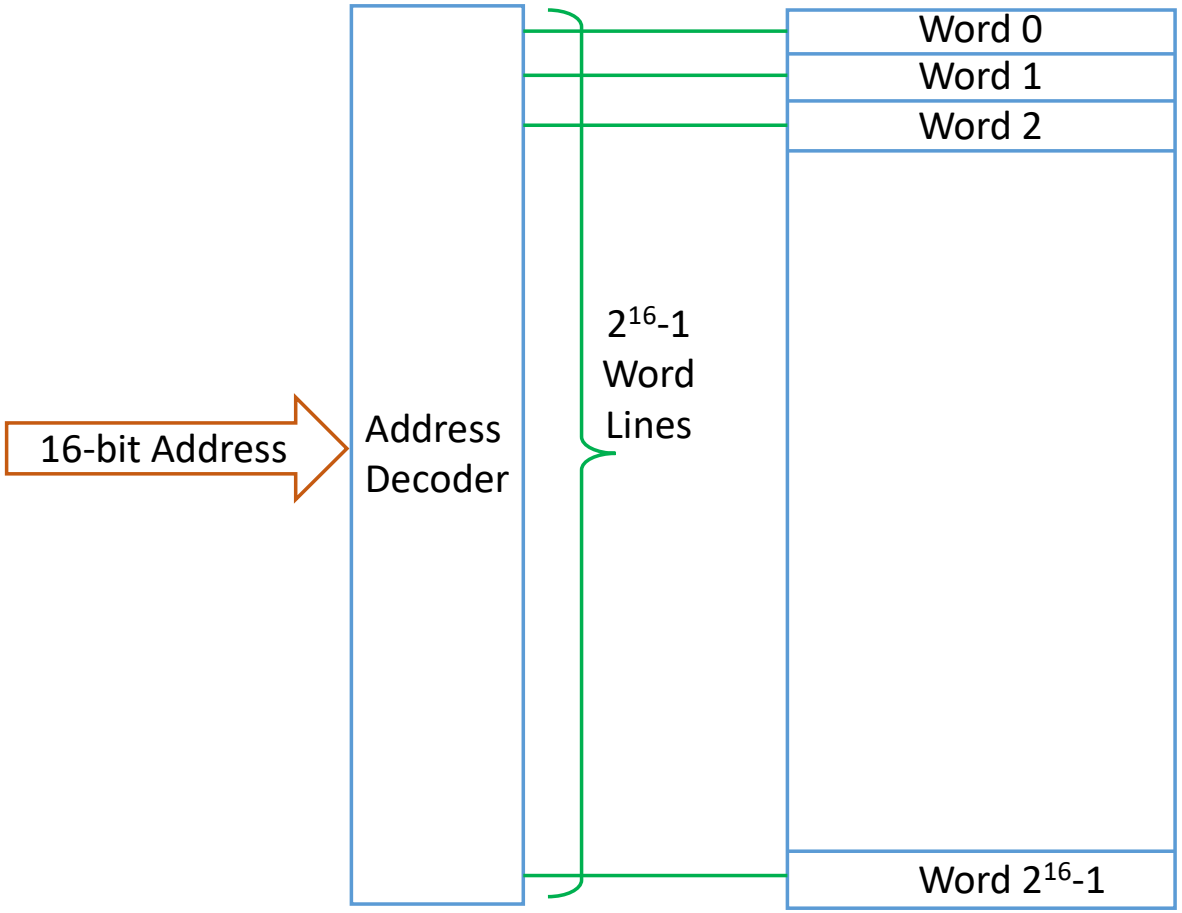


Suggest Covering in the tutorial

8-to-1 Multiplexor/Selector using Decoder

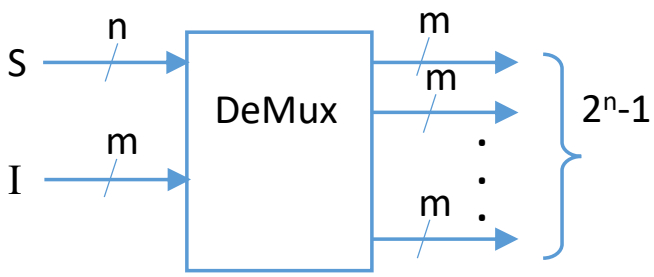


Decoders are an essential part of all Memories

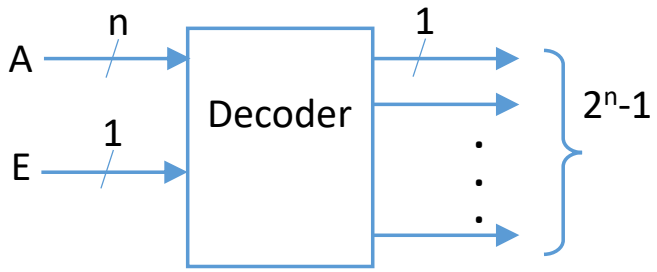


1. Memories are organized in terms of words
2. Computer sends the address of the word to be read/written to the address decoder
3. The address decoder turns on the appropriate word line; the other word lines are turned off
4. The active word line selects the word to be read or written to

What is the difference between DeMux and Decoder ?



DeMux connects input *I* to one of 2^n-1 outputs



Decoder turns on, i.e., sets to 1, one of 2^n-1 outputs selected by the address A, when the Enable signal is 1

1. Both have *n* bit select/address signals
2. Both have 2^n-1 outputs – only one of them is selected/turned on

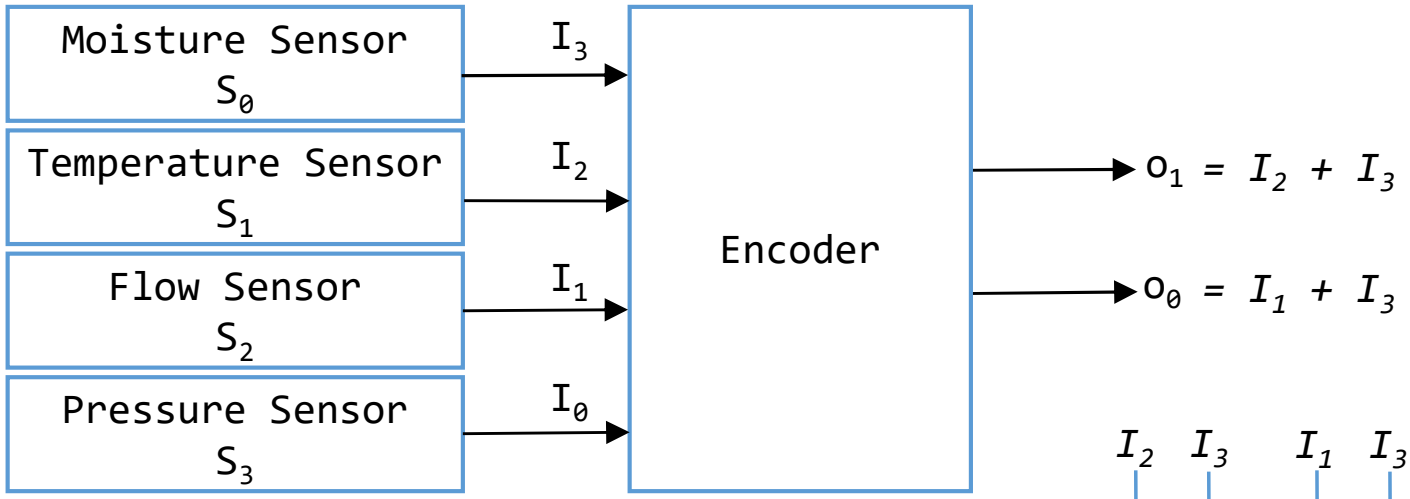
1. But the demux outputs are *m* bit wide, same width as the input *I*
   Demux transfers the m bits of input I to m bits of the selected output
2. And the decoder outputs are *1* bit wide that is either turned on or off
   Decoders do not have any input to transfer to the output
3. DeMux has no Enable signal, Decoder has an enable signal

Encoders and Priority Encoders

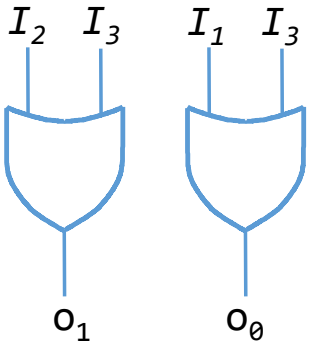
Encoder Introduction

Encoder implements the inverse function of Decoder

It maps one-hot encoded inputs to a binary encoded outputs



Inputs				Outputs	
I_3	I_2	I_1	I_0	o_1	o_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

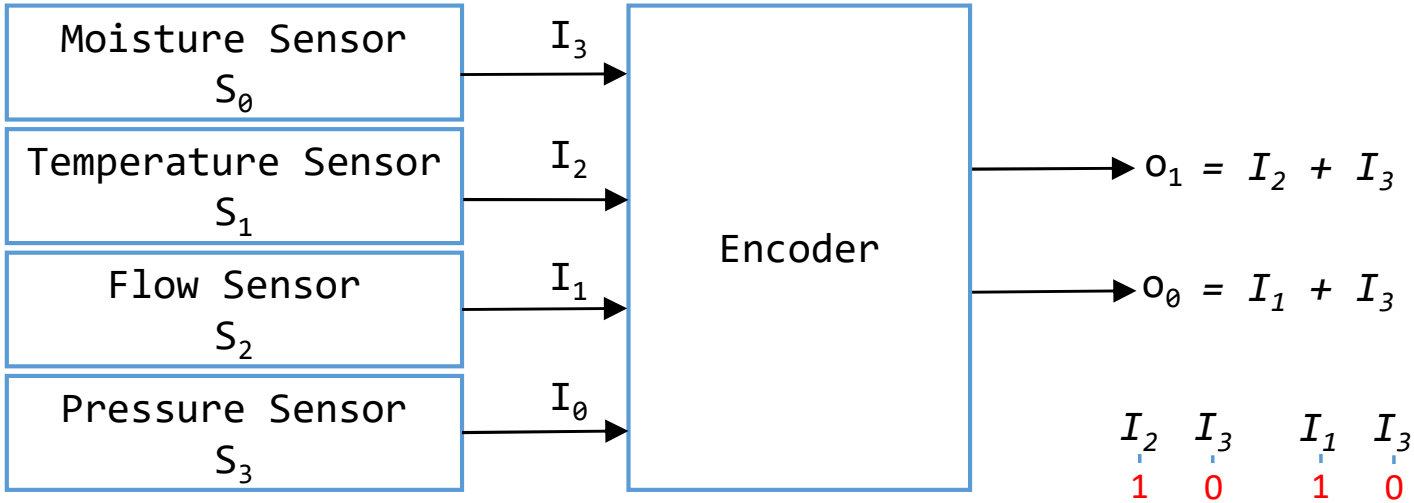


Encoder

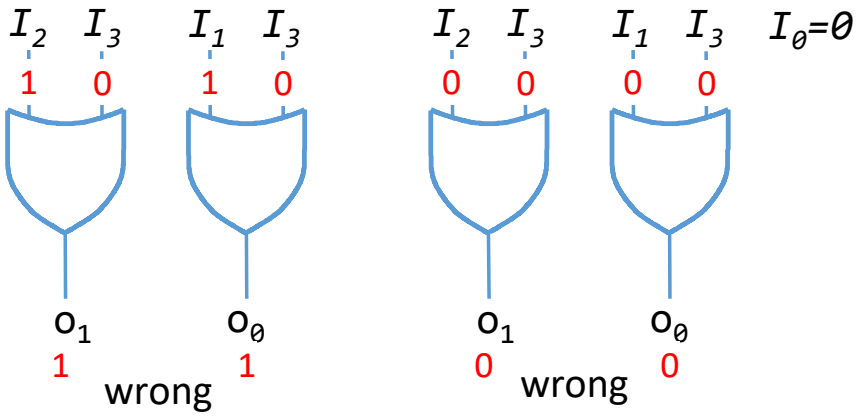
What happens when the inputs do not respect the one-hot requirement ?

For instance, what if inputs I_1 and I_2 are ON ?

What happens when the none of the input sensors are hot ?



I_3	I_2	I_1	I_0	o_1	o_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1



Solution: Priority Encoder

4-to-2 Priority Encoder

1. **Priority Encoder priortizes the most significant input**
2. **Produces an output Any that is asserted when atleast one of the one-hot inputs is asserted**

I_3	I_2	I_1	I_0	o_1	o_0	Any
0	0	0	1	0	0	1
0	0	1	-	0	1	1
0	1	-	-	1	0	1
1	-	-	-	1	1	1
0	0	0	0	0	0	0

I_3	I_2	I_1	I_0	o_1	o_0	Any
0	0	0	1	0	0	1
0	0	1	-	0	1	1
0	1	-	-	1	0	1
1	-	-	-	1	1	1
0	0	0	0	0	0	0

$f_O_0 = I_3'I_2'I_1 + I3$

I_3 I_2 \ I_1 I_0	00	01	11	10
00			1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$f_O_1 = I_3'I_2 + I3$

I_3 I_2 \ I_1 I_0	00	01	11	10
00			1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$f_Any = I_1 + I_2 + I_3 + I_4$

You are expected to be able to write the Priority Encoder Truth table based on understanding of its functionality

```
module priority_encoder4to2 (
    input logic [3:0] I,    // 4-bit input
    output logic [1:0] Y,   // 2-bit output
    output logic any
);

  // Intermediate signals for the NOT gates
  logic i0, i1, i2, i3;
  logic i0_inv, i1_inv, i2_inv, i3_inv;
  logic u1_out, u2_out, u3_out, u4_out,
        u5_out, u7_out, u8_out, u9_out, u10_out;

    assign i0 = I[0];
    assign i1 = I[1];
    assign i2 = I[2];
    assign i3 = I[3];

    // Inverting the inputs
    nand2_delay inv0 (.a (i0), .b (i0), .y(i0_inv));
    nand2_delay inv1 (.a (i1), .b (i1), .y(i1_inv));
    nand2_delay inv2 (.a (i2), .b (i2), .y(i2_inv));
    nand2_delay inv3 (.a (i3), .b (i3), .y(i3_inv));
```

```
// Y[0] = (i3_inv AND i2_inv AND i1) OR i3
nand3_delay u1 (.a(i1), .b(i2_inv),
                .c(i3_inv), .y(u1_out));
nand2_delay u2 (.a(i3), .b(i3), .y(u2_out));
nand2_delay u3 (.a(u1_out), .b(u2_out), .y(Y[0]));

// Y[1] = (i3_inv AND i2) OR i3
nand2_delay u4 (.a(i2),     .b(i3_inv), .y(u4_out));
nand2_delay u5 (.a(i3),     .b(i3),     .y(u5_out));
nand2_delay u6 (.a(u4_out), .b(u5_out), .y(Y[1]));

// any = i1 OR i2 OR i3 OR i4
nand2_delay u7  (.a(i0), .b(i0), .y(u7_out));
nand2_delay u8  (.a(i1), .b(i1), .y(u8_out));
nand2_delay u9  (.a(i2), .b(i2), .y(u9_out));
nand2_delay u10 (.a(i3), .b(i3), .y(u10_out));

nand4_delay u11 (.a(u7_out), .b(u8_out), .c(u9_out),
                 .d(u10_out),.y(any));

endmodule
```

```
`timescale 1ns / 1ps

module tb_priority_encoder_4to2;

  logic [3:0] I;
  logic [1:0] Y;
  logic any;

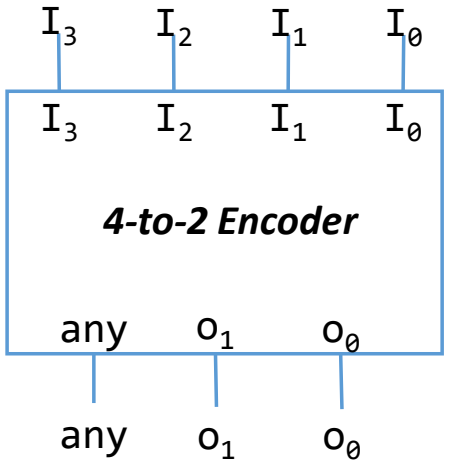
  // Instantiate the priority encoder
  priority_encoder4to2 uut (
    .I(I),
    .Y(Y),
    .any(any)
  );

  initial begin
    // Test all possible input combinations
    for (int i = 0; i < 16; i++) begin
      I = i;
      #10;
    end
    $stop;
  end

endmodule
```

4-to-2 Priority Encoder - Variations

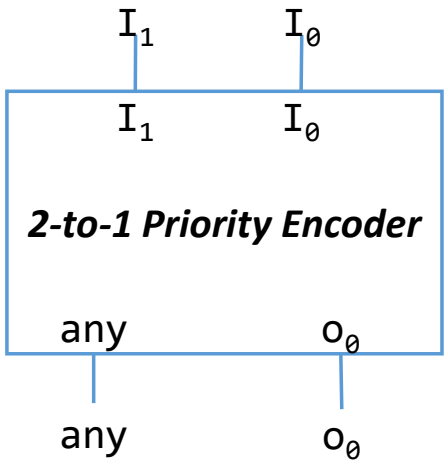
I_3	I_2	I_1	I_0	o_1	o_0	Any
0	0	0	1	0	0	1
0	0	1	-	0	1	1
0	1	-	-	1	0	1
1	-	-	-	1	1	1
0	0	0	0	0	0	0



Real life and Exams:

1. *In this example the priority is $I_3 > I_2 > I_1 > I_0$*
2. *In real life and in exams the priority could be different*
3. *The output encoding is binary in this example.*
4. *In real life the encoding could be two's complement or fixed point*

Quiz Time



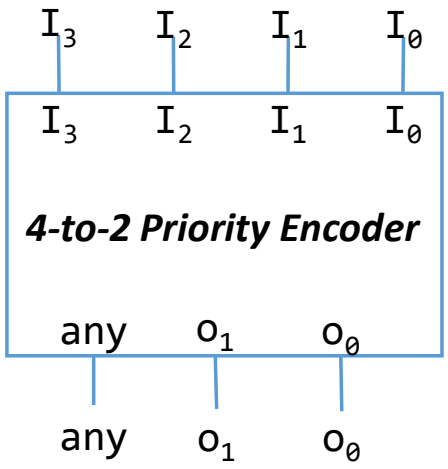
If the inputs are $(I_1, I_0) = (11)$
What would be the outputs (any, O_0)

A. any = 0, O_0 = 0
B. any = 0, O_0 = 1
C. any = 1, O_0 = 0
D. any = 1, O_0 = 1

If the inputs are $(I_1, I_0) = (01)$
What would be the outputs (any, O_0)

A. any = 0, O_0 = 0
B. any = 0, O_0 = 1
C. any = 1, O_0 = 0
D. any = 1, O_0 = 1

Quiz Time



If the inputs are $(I_3, I_2, I_1, I_0) = (1011)$
What would be the outputs (any, O_1, O_0)

A. any = 1, O_1 = 0, O_0 = 1
B. any = 1, O_1 = 1, O_0 = 1
C. any = 1, O_1 = 1, O_0 = 0
D. any = 0, O_1 = 1, O_0 = 1

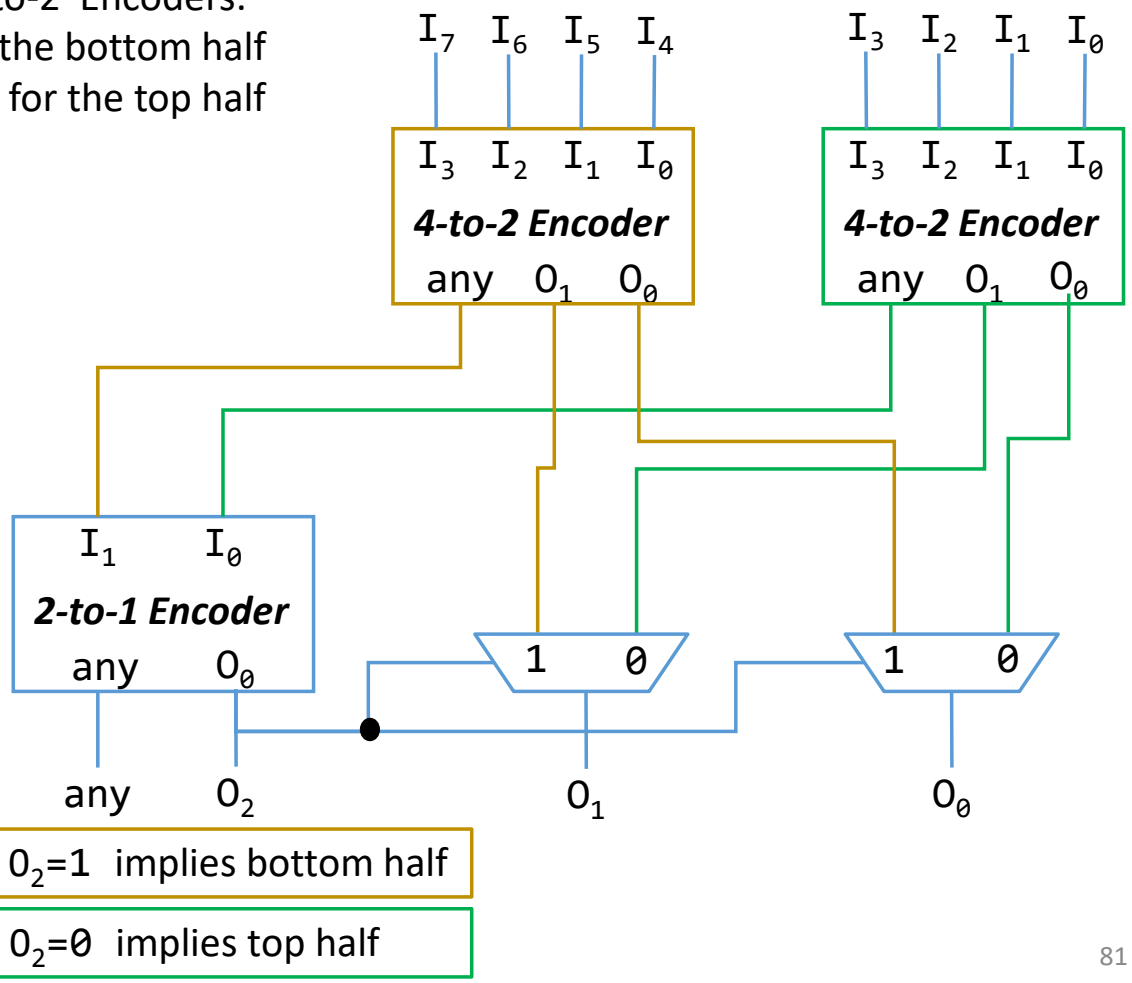
If the inputs are $(I_3, I_2, I_1, I_0) = (0000)$
What would be the outputs (any, O_1, O_0)

A. any = 0, O_1 = 0, O_0 = 0
B. any = 1, O_1 = 1, O_0 = 1
C. any = 1, O_1 = 0, O_0 = 1
D. any = 1, O_1 = 1, O_0 = 0

8-to-3 Priority Encoder using smaller Encoders

These two bits are encoded by the two 4-to-2 Encoders.
One for the bottom half
One for the top half

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	O_2	O_1	O_0	Any
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	1	-	0	0	1	1
0	0	0	0	0	1	-	-	0	1	0	1
0	0	0	0	1	-	-	-	0	1	1	1
0	0	0	1	-	-	-	-	1	0	0	1
0	0	1	-	-	-	-	-	1	0	1	1
0	1	-	-	-	-	-	-	1	1	0	1
1	-	-	-	-	-	-	-	1	1	1	1



You should be able to

Implement 4-to-2 priority encoder using 2-to-1 priority encoder

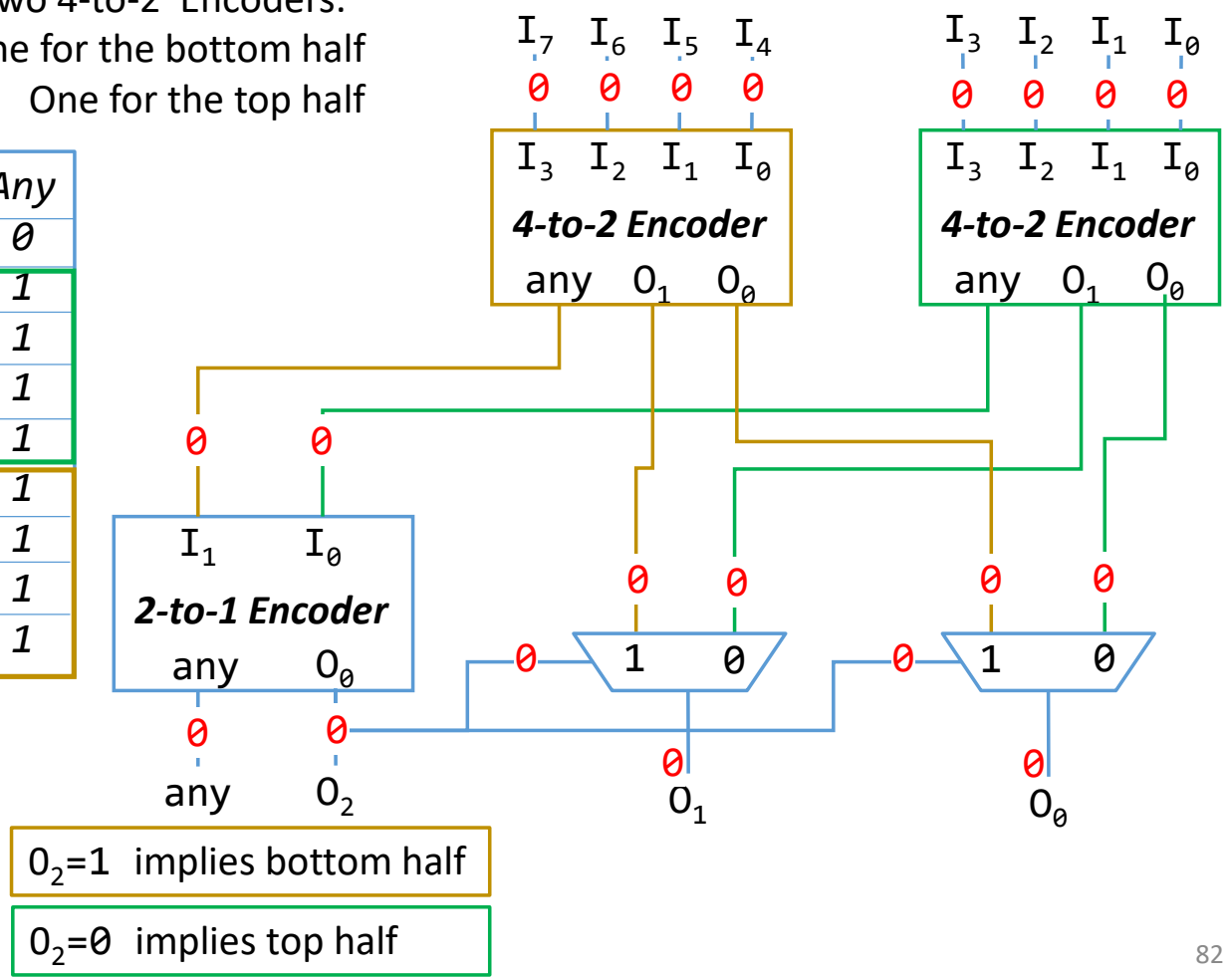
Know the delay and gate count

Draw timing diagram of outputs for a given input sequence

8-to-3 Priority Encoder using smaller Encoders

These two bits are encoded by the two 4-to-2 Encoders.
One for the bottom half
One for the top half

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	O_2	O_1	O_0	Any
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	1	-	0	0	1	1
0	0	0	0	0	1	-	-	0	1	0	1
0	0	0	0	1	-	-	-	0	1	1	1
0	0	0	1	-	-	-	-	1	0	0	1
0	0	1	-	-	-	-	-	1	0	1	1
0	1	-	-	-	-	-	-	1	1	0	1
1	-	-	-	-	-	-	-	1	1	1	1

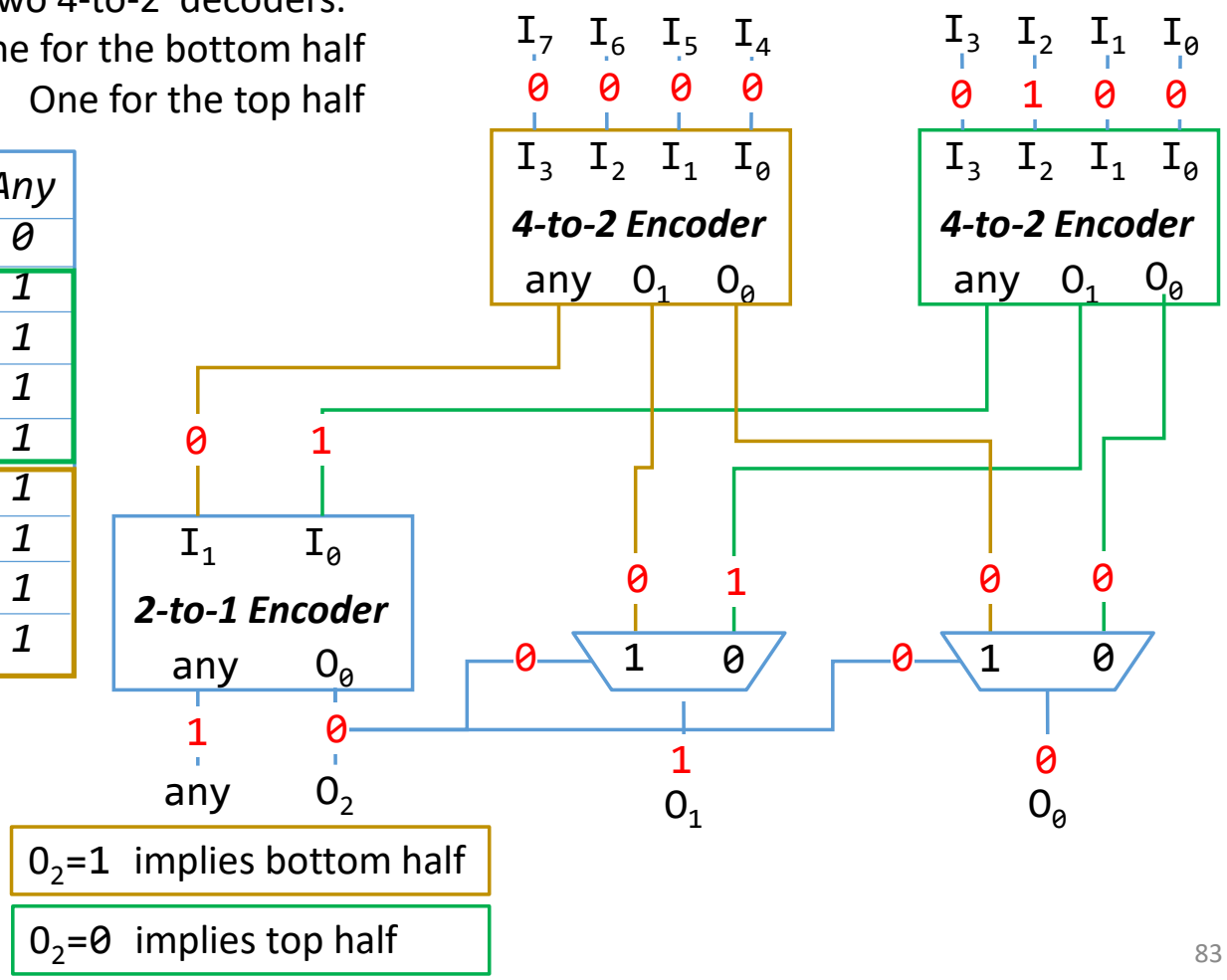


Homework Exercise:
Implement 4-to-2 priority encoder using 2-to-1 priority encoder

8-to-3 Priority Encoder using smaller Encoders

These two bits are encoded by the two 4-to-2 decoders.
One for the bottom half
One for the top half

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	O_2	O_1	O_0	Any
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	1	-	0	0	1	1
0	0	0	0	0	1	-	-	0	1	0	1
0	0	0	0	1	-	-	-	0	1	1	1
0	0	0	1	-	-	-	-	1	0	0	1
0	0	1	-	-	-	-	-	1	0	1	1
0	1	-	-	-	-	-	-	1	1	0	1
1	-	-	-	-	-	-	-	1	1	1	1



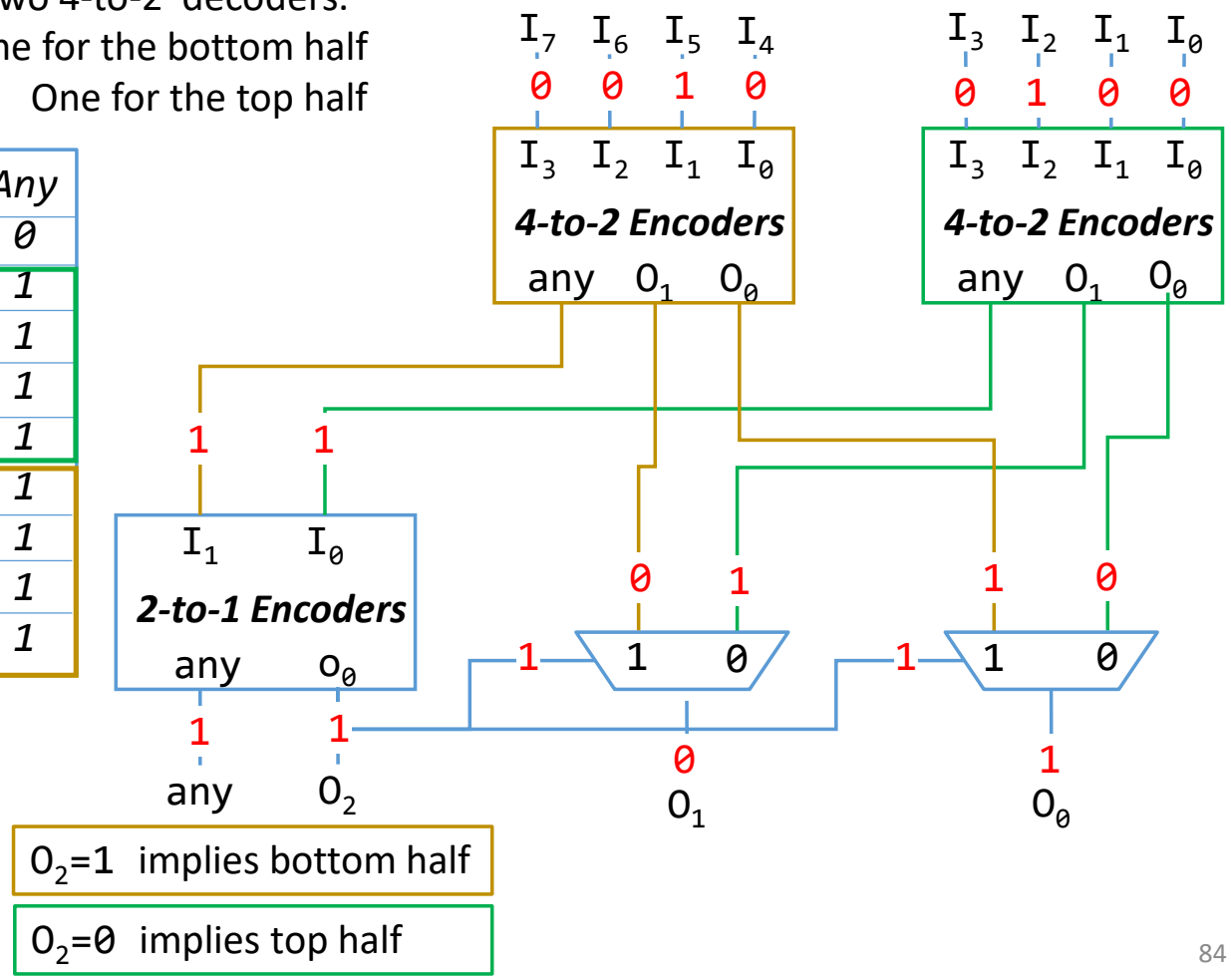
O_2=1 implies bottom half

O_2=0 implies top half

8-to-3 Priority Encoder using smaller Encoders

These two bits are encoded by the two 4-to-2 decoders.
One for the bottom half
One for the top half

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	O_2	O_1	O_0	Any
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	1	-	0	0	1	1
0	0	0	0	0	1	-	-	0	1	0	1
0	0	0	0	1	-	-	-	0	1	1	1
0	0	0	1	-	-	-	-	1	0	0	1
0	0	1	-	-	-	-	-	1	0	1	1
0	1	-	-	-	-	-	-	1	1	0	1
1	-	-	-	-	-	-	-	1	1	1	1

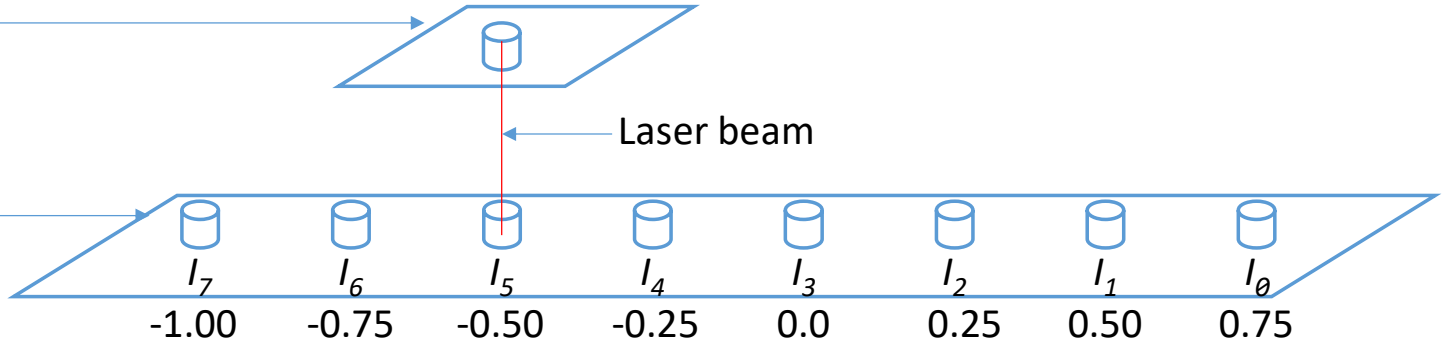


Real Life Scenario

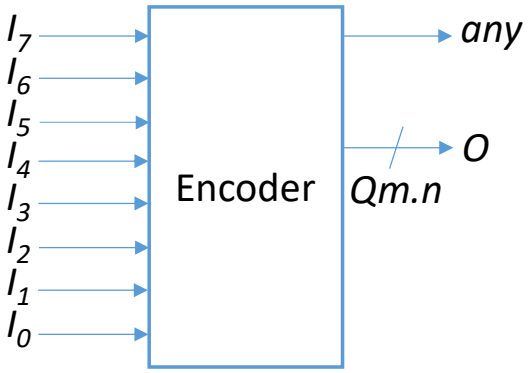
Robotic Arm

1. Moves horizontally Left or right
2. Beams a laser vertically down

Measurement plate
with sensors at fixed positions
at an interval of 0.25 mm



1. When the laser beam and the sensor align, as shown, one and only one of the inputs to the decoder from I_7 to I_0 is activated, i.e., set to 1 and the distance measurement shown in the diagram is output by the encoder. The any signal is also set to 1
2. When the laser beam and the sensor do not align the inputs I_7 to I_0 are deactivated, i.e., set to 0. Also the *any* signal is also set to 0
3. Select an appropriate fixed point format (*Qm.n*) for output of the encoder that can represent -1.00 to 0.75
4. Create the truth table and write the output in terms of implicants. *Minimization using K Map is not necessary*.



Lab Exercise Suggestions

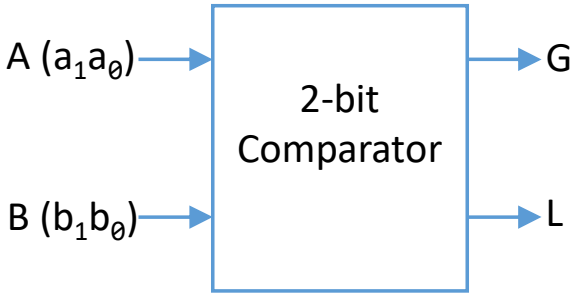
- Implement a 2-to-1 priority encoder using the 4-to-2 priority encoder as a guide
- Use the schematic in the slides to implement an 8-to-3 priority encoder using the 2-to-1 and 4-to-2

Comparator

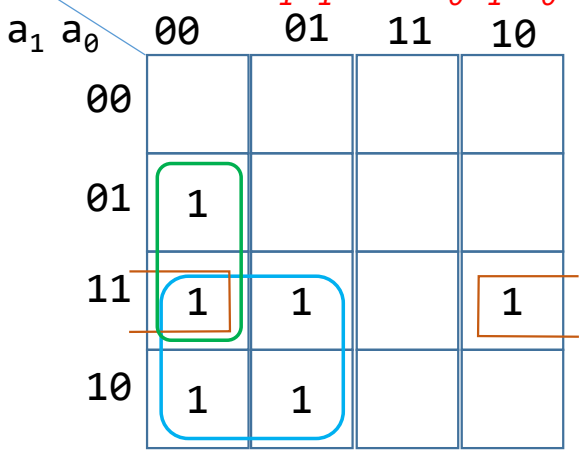
2-bit Comparator

a_1	a_0	b_1	b_0	G	L
0	0	0	0	0	0
0	0	0	1	0	1
0	0	1	0	0	1
0	0	1	1	0	1
0	1	0	0	1	0
0	1	0	1	0	0
0	1	1	0	0	1
0	1	1	1	0	1
1	0	0	0	1	0
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	1	0	1
1	1	0	0	1	0
1	1	0	1	1	0
1	1	1	0	1	0
1	1	1	1	0	0

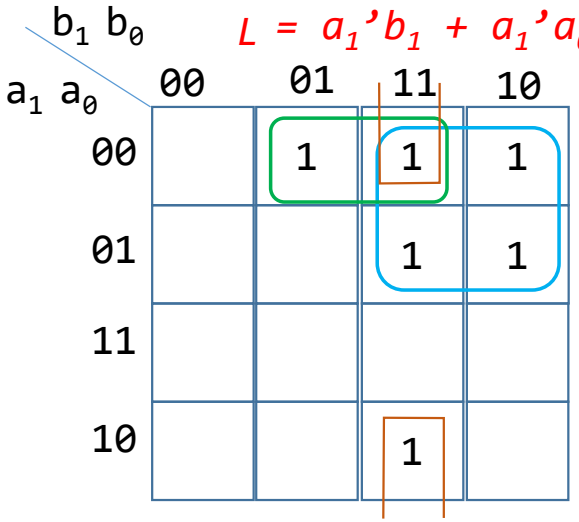
$A = a_1a_0 \quad B = b_1b_0$
$G = 1$ when $A > B$
$L = 1$ when $A < B$
$G = L = 0$ when $A = B$



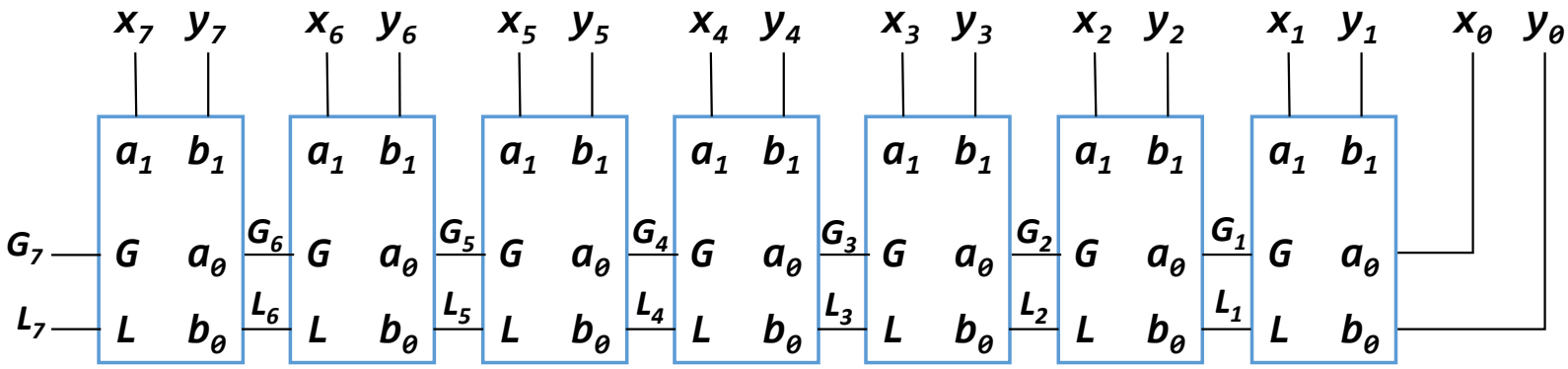
$G = a_1b_1' + a_0b_1'b_0' + a_1a_0b_0'$



$L = a_1'b_1 + a_1'a_0'b_0 + b_1b_0a_0'$



Building 8-bit Comparator from 2-bit comparators



If there are 128 bits, how many stages ?

Embedded Quiz

Let us compare

$x_3\ x_2\ x_1\ x_0 = 0\ 1 \mid 1\ 0$

$y_3\ y_2\ y_1\ y_0 = 1\ 1 \mid 0\ 1$

Stage1

$G_1 = (x_1\ x_0) > (y_1\ y_0) = 1$

$L_1 = (x_1\ x_0) < (y_1\ y_0) = 0$

X is winning

Comparision reduces to

$x_3\ x_2\ G_1 = 0 \mid 1\ 1$

$y_3\ y_2\ L_1 = 1 \mid 1\ 0$

Stage 2:

$G_2 = (x_2\ G_1) > (y_2\ L_1) = 1$

$L_2 = (x_2\ G_1) < (y_2\ L_1) = 0$

X is still winning

Comparision reduces to

$x_3\ G_2 = 0\ 1$

$y_3\ L_2 = 1\ 0$

Stage 3:

$G_3 = (x_3\ G_2) > (y_3\ L_2) = 0$

$L_3 = (x_3\ G_2) < (y_3\ L_2) = 1$

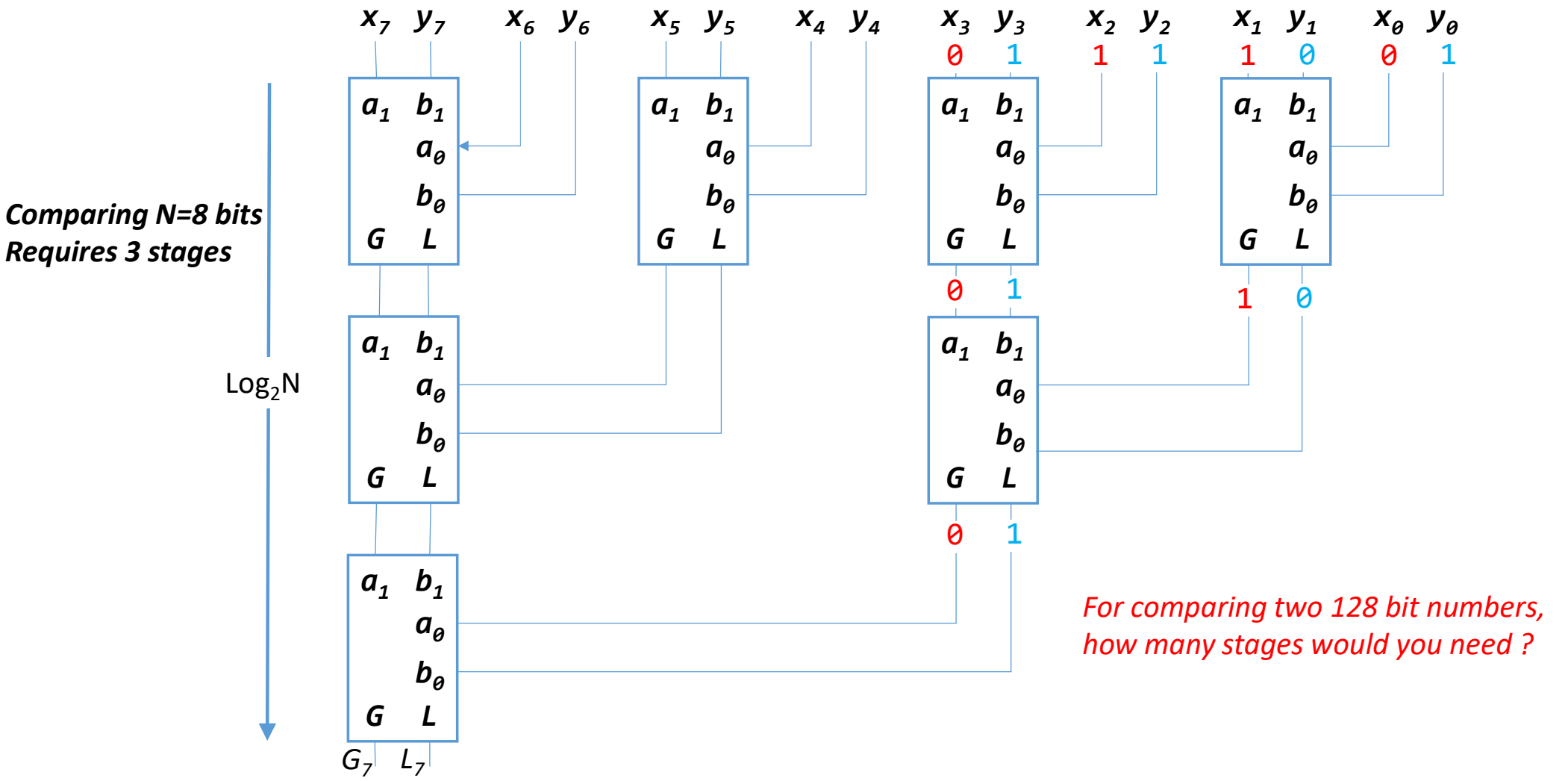
Y turns the tables and wins

$$G_i = (x_i > y_i)\ \text{OR}\ ((x_i = y_i)\ \text{AND}\ (G_{i-1} > L_{i-1}))$$

$$L_i = (x_i < y_i)\ \text{OR}\ ((x_i = y_i)\ \text{AND}\ (G_{i-1} < L_{i-1}))$$

Parallel – Log_2N delay Comparator

$(x_3x_2x_1x_0=0110)$ $(y_3y_2y_1y_0=1101)$



For comparing two 128 bit numbers, how many stages would you need ?

```
module comparator_slice (
                         input logic  a0,
                         input logic  b0,
                         input logic  a1,
                         input logic  b1,
                         output logic g,
                         output logic l
);


  // Intermediate signals for the NOT gates
  logic a0_inv, b0_inv, a1_inv, b1_inv;
  logic u1_out, u2_out, u3_out, u4_out,
        u5_out, u7_out, u8_out;

    // Inverting the inputs
    nand2_delay inv0 (.a (a0), .b (a0), .y(a0_inv));
    nand2_delay inv1 (.a (a1), .b (a1), .y(a1_inv));
    nand2_delay inv2 (.a (b0), .b (b0), .y(b0_inv));
    nand2_delay inv3 (.a (b1), .b (b1), .y(b1_inv));

 // g = a1 AND b1_inv OR a0 AND b1_inv AND b0_inv OR a1 AND a0 and b0_inv
   nand2_delay u1 (.a(a1), .b(b1_inv), .y(u1_out));
   nand3_delay u2 (.a(a0), .b(b1_inv), .c(b0_inv), .y(u2_out));
   nand3_delay u3 (.a(a0), .b(a0), .c(b0_inv), .y(u3_out));
   nand3_delay u4 (.a(u1_out), .b(u2_out), .c(u3_out), .y(g));

   // l = a1_inv AND b1 OR a0_inv AND a1_inv AND b0 OR b1 AND b0 and a0_inv
   nand2_delay u5 (.a(a1_inv), .b(b1), .y(u5_out));
   nand3_delay u6 (.a(a0_inv), .b(a1_inv), .c(b0), .y(u6_out));
   nand3_delay u7 (.a(b1), .b(b0), .c(a0_inv), .y(u7_out));
   nand3_delay u8 (.a(u5_out), .b(u6_out), .c(u7_out), .y(l));

endmodule
```

$g = a_1 b_1' + a_0 b_1' b_0' + a_1 a_0 b_0'$

$l = a_1' b_1 + a_1' a_0' b_0 + b_1 b_0 a_0'$

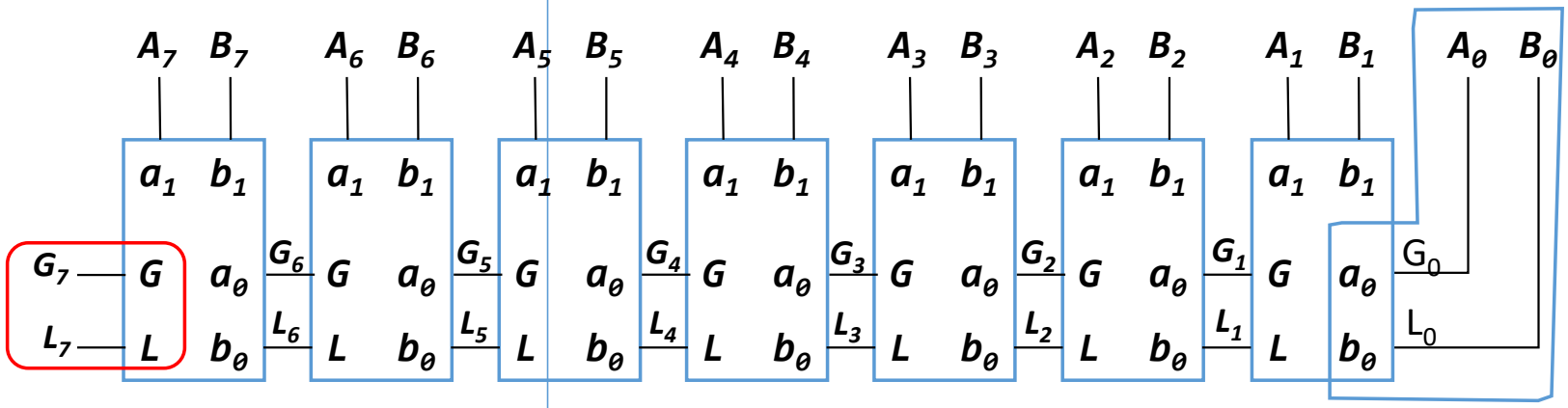
```
module comparator_Nbit #(parameter N=4)
                (   input logic [N-1:0] A,
                    input logic [N-1:0] B,
                    output logic        g,
                    output logic        l);

   logic [N-1:0] G, L;

   assign G[0] = A[0];
   assign L[0] = B[0];

   assign g = G[N-1];
   assign l = L[N-1];

    // Generate N bit 2-1 mux
    genvar i;
    generate
        for (i = 0; i < N-1; i++) begin : gen_comp
            comparator_slice cs (
                .a0(G[i]),
                .b0(L[i]),
                .a1(A[i+1]),
                .b1(B[i+1]),
                .g(G[i+1]),
                .l(L[i+1])
            );
        end
    endgenerate
endmodule
```



```
`timescale 1ns / 1ps

module tb_comparator_Nbit;

   parameter N = 4;

   logic [N-1:0] A;
   logic [N-1:0] B;
   logic g, l;

  // Instantiate the comparator
   comparator_Nbit #(N) uut (
    .A(A),
    .B(B),
    .g(g),
    .l(l)
  );

  initial begin
     A = 10; B = 11; #30;

     A = 10; B = 8; #30;

     A = 10; B = 10; #30;

     A = 0; B = 1; #30;

    $stop;
  end

endmodule
```

Labs Exercises

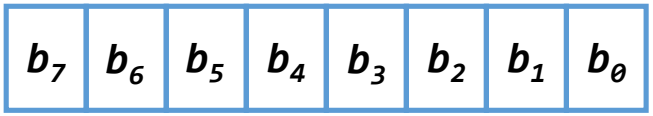
- Implement the $Log_2(N)$ parallel comparator to compare two 4-bit unsigned numbers

Shifters and Rotators

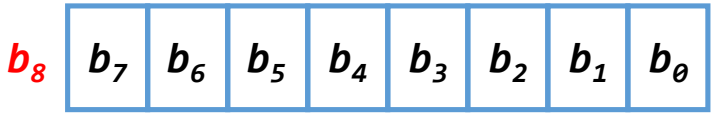
Logical Right Shift

Logical Right Shift

The Bit Patterns are not interpreted as a number

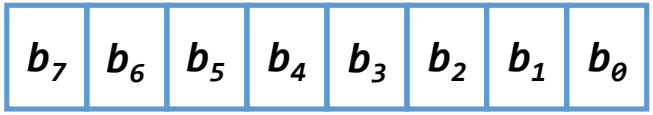


An external bit, b_8, is shifted into the pattern from MSB side
All bits shift right.
The LSB, b_0 , is dropped

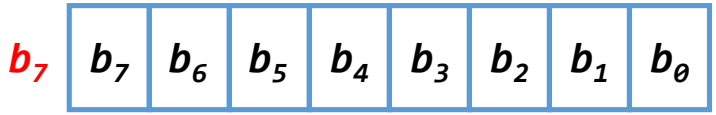


Arithmetic Right Shift

The Bit pattern is interpreted as a number
Arithmetic Right Shift Divides by Two



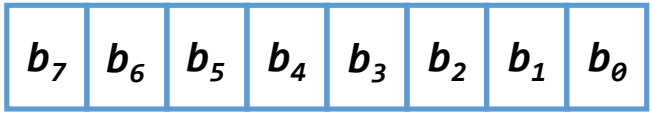
An external bit, b_7, is the sign extension bit
The LSB, b_0 , is dropped



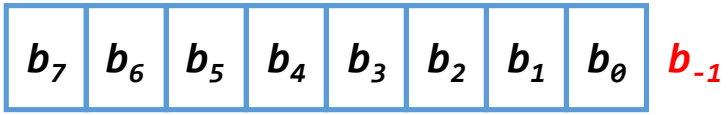
Logical Left Shift

Logical Left Shift

The Bit Patterns are not interpreted as a number



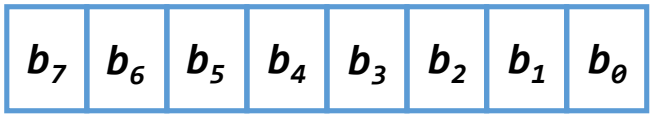
An external bit, b_{-1} , is shifted into the pattern from LSB side
All bits shift Left.
The MSB, b_7 , that is shifted out is dropped



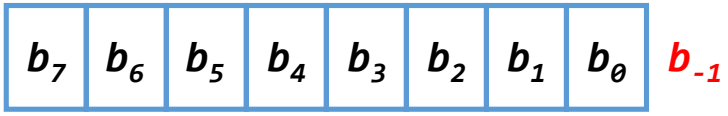
Arithmetic Left Shift

Arithmetic Left Shift

The Bit pattern is interpreted as an unsigned number.
Arithmetic Left Shift Multiplies by Two.
Overflow and Underflow can happen.

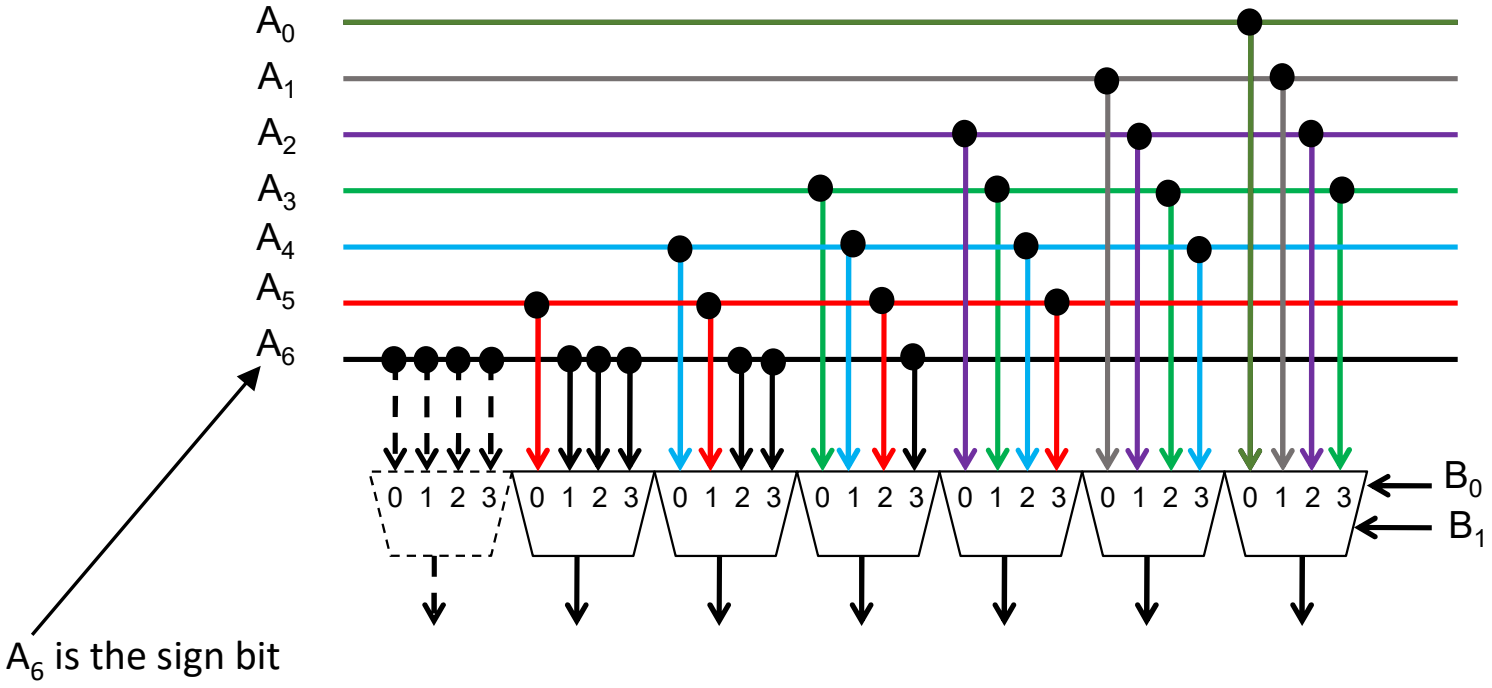


An external bit, b_{-1} , is shifted into the pattern from LSB side
All bits shift Left.
The MSB, b_7 , that is shifted out is dropped



Arithmetic Right Shift

Division by (2^0, 2^1, 2^2, 2^3) – 1, 2, 4, 8

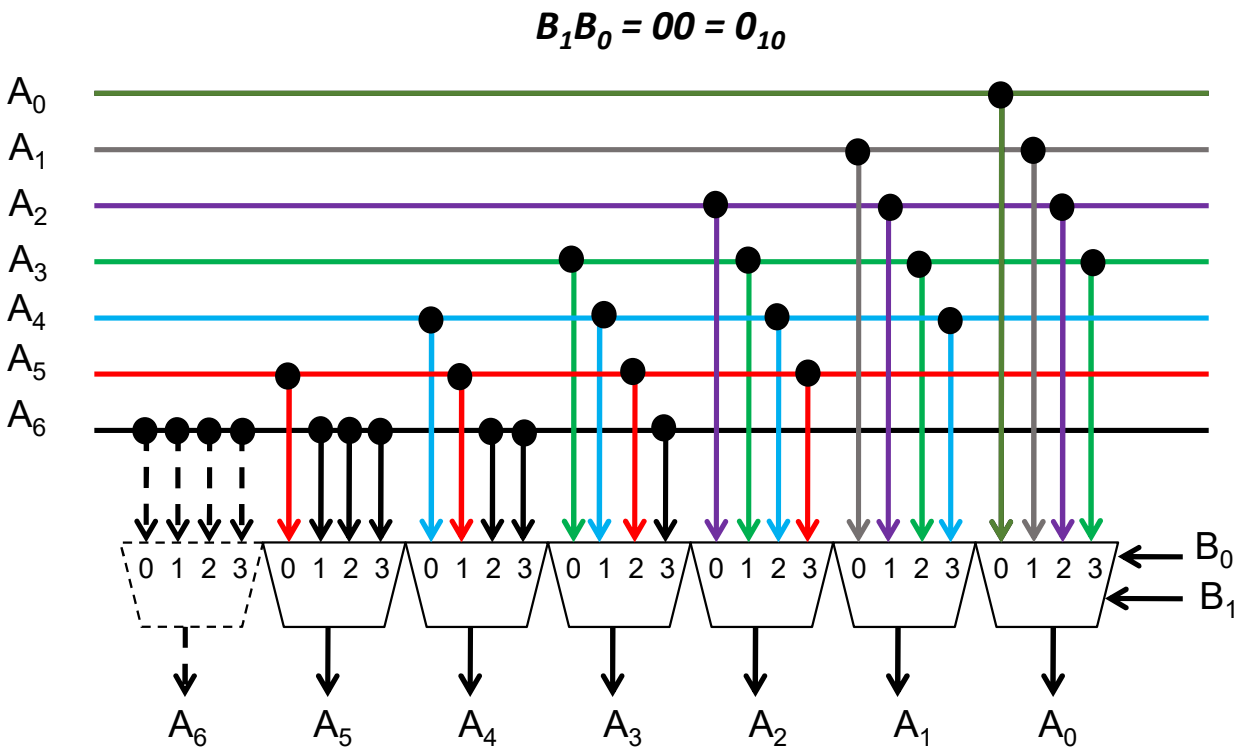


A_6 is the sign bit

It is assigned to multiple positions as we shift right

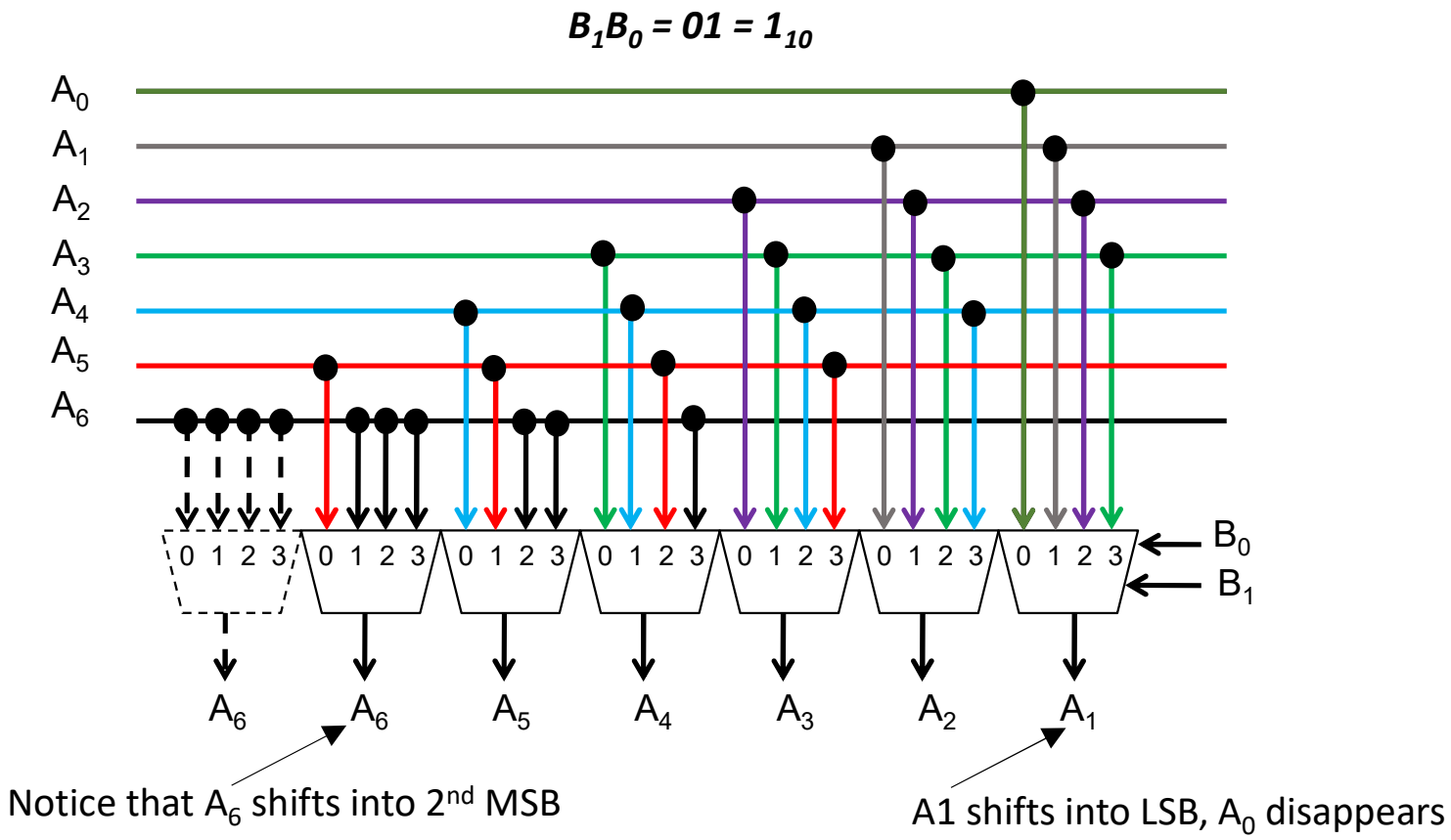
Arithmetic Shifter

Division by (2^0, 2^1, 2^2, 2^3) – 1, 2, 4, 8



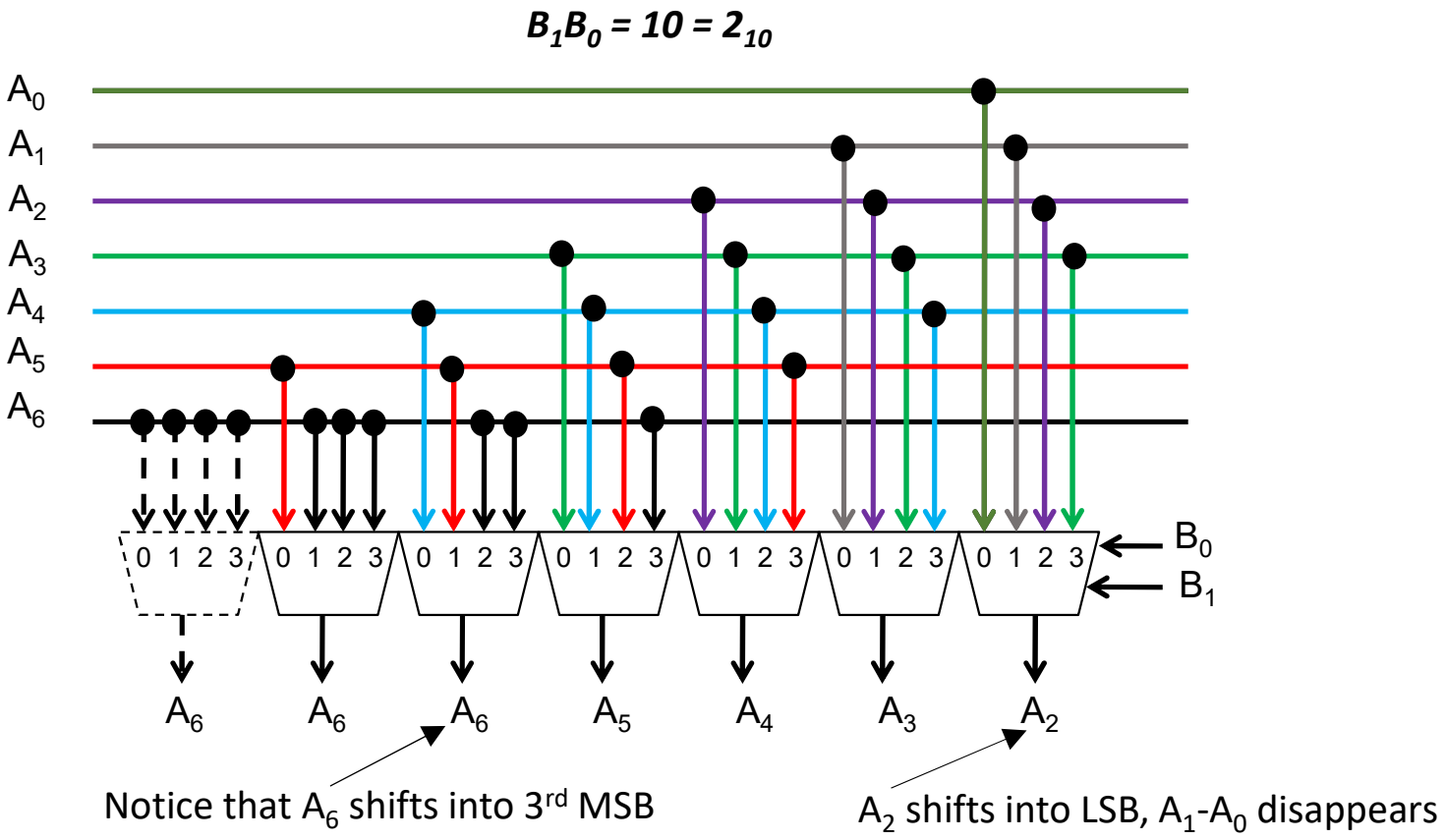
Arithmetic Right Shift

Division by (2^0, 2^1, 2^2, 2^3) – 1, 2, 4, 8



Arithmetic Right Shift

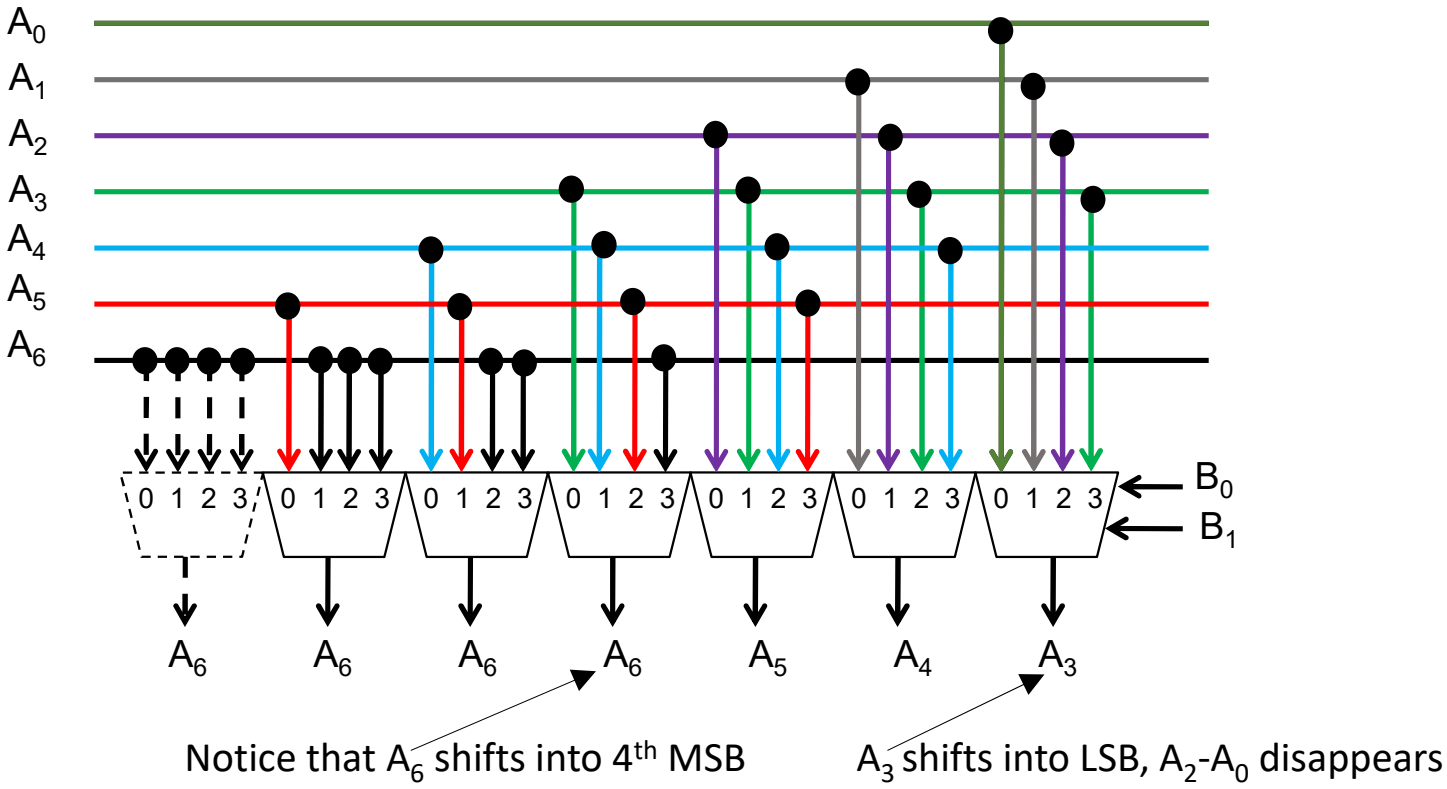
Division by (2^0, 2^1, 2^2, 2^3) – 1, 2, 4, 8



Arithmetic Right Shift

Division by (2^0, 2^1, 2^2, 2^3) – 1, 2, 4, 8

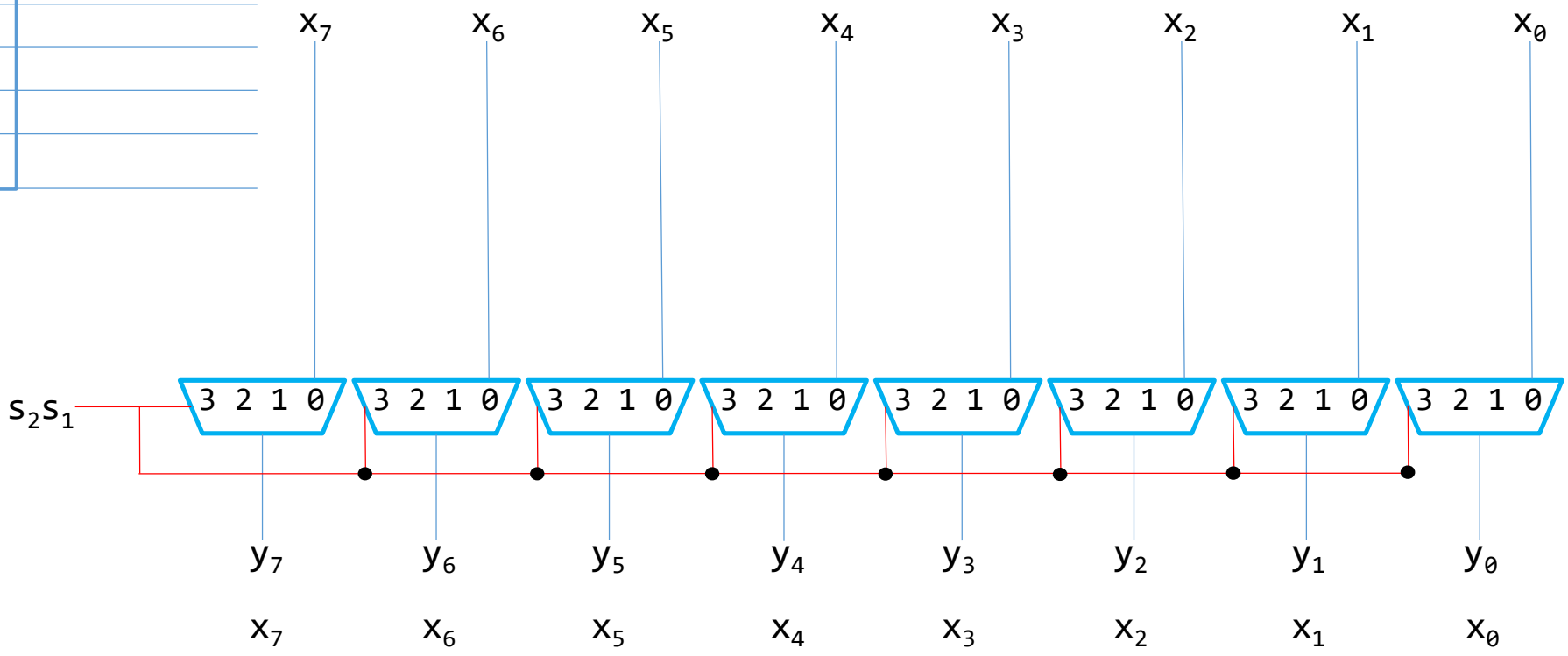
$B_1B_0 = 11 = 3_{10}$



Left Shifter and Rotator

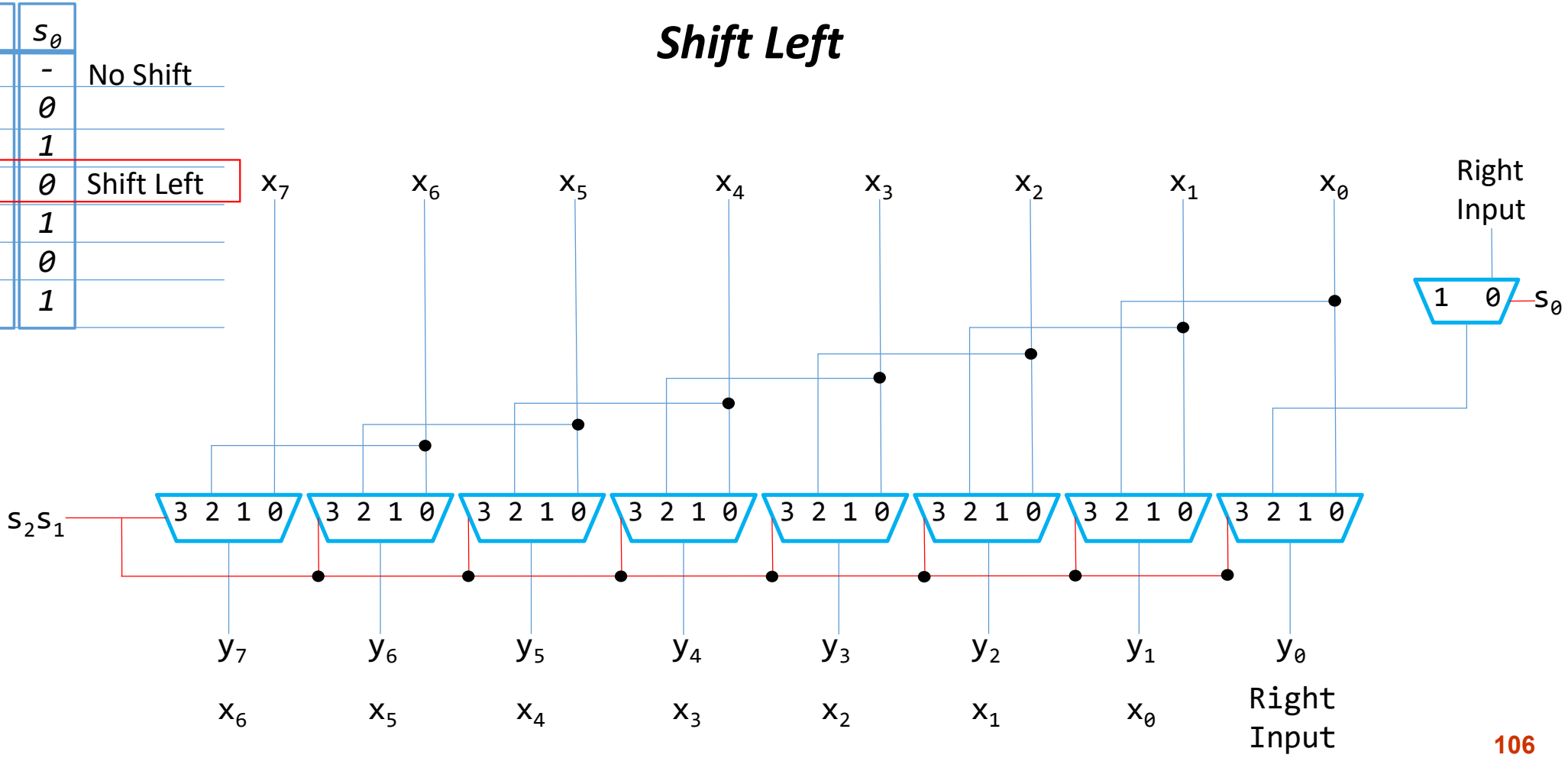
s_2	s_1	
0	0	No Shift
0	1	
0	1	
1	0	
1	0	
1	1	
1	1	

No Shift



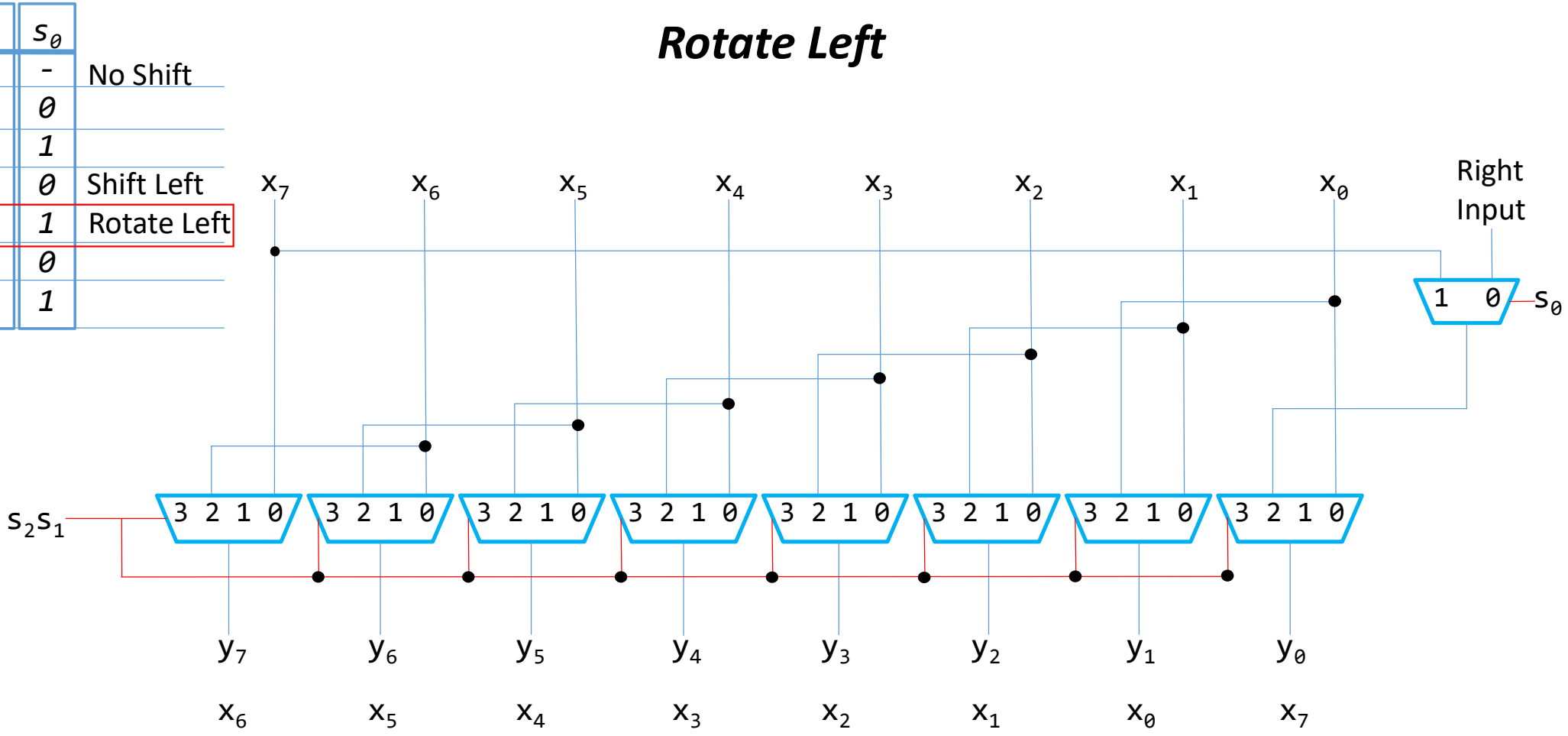
Left Shifter and Rotator

s_2	s_1	s_0	
0	0	-	No Shift
0	1	0	
0	1	1	
1	0	0	Shift Left
1	0	1	
1	1	0	
1	1	1	



Left Shifter and Rotator

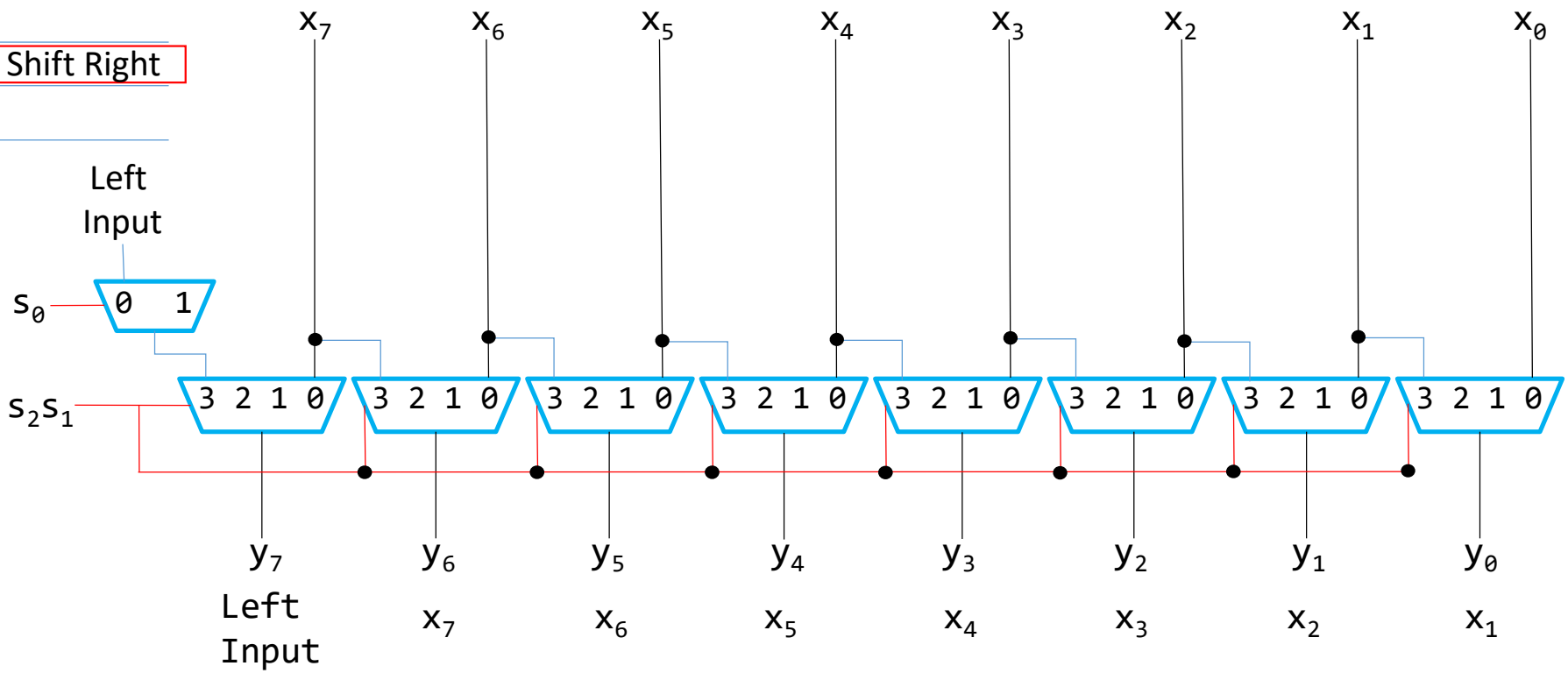
s_2	s_1	s_0	
0	0	-	No Shift
0	1	0	
0	1	1	
1	0	0	Shift Left
1	0	1	Rotate Left
1	1	0	
1	1	1	



Right Shifter and Rotator

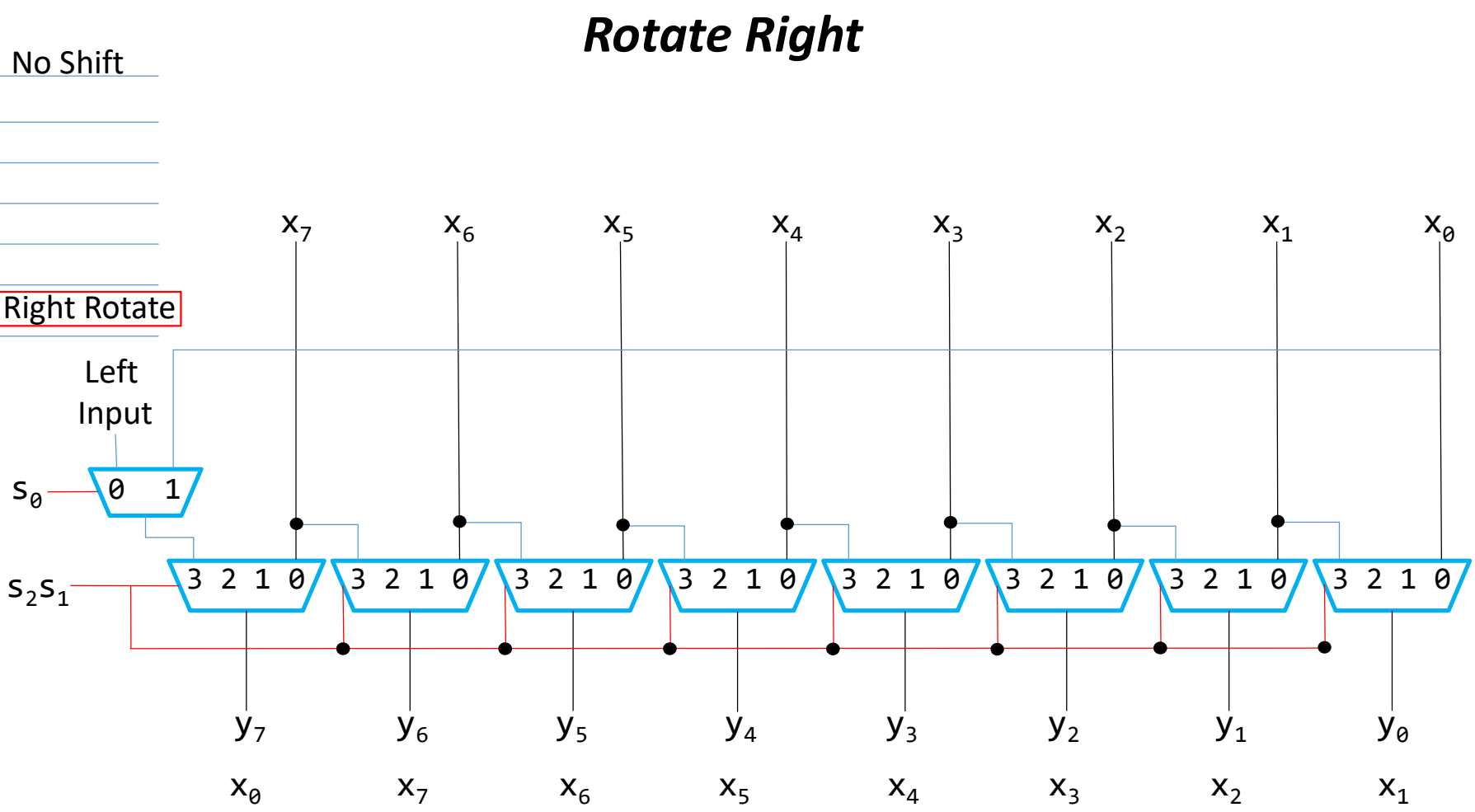
s_2	s_1	s_0	
0	0	-	No Shift
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	Shift Right
1	1	1	

Shift Right



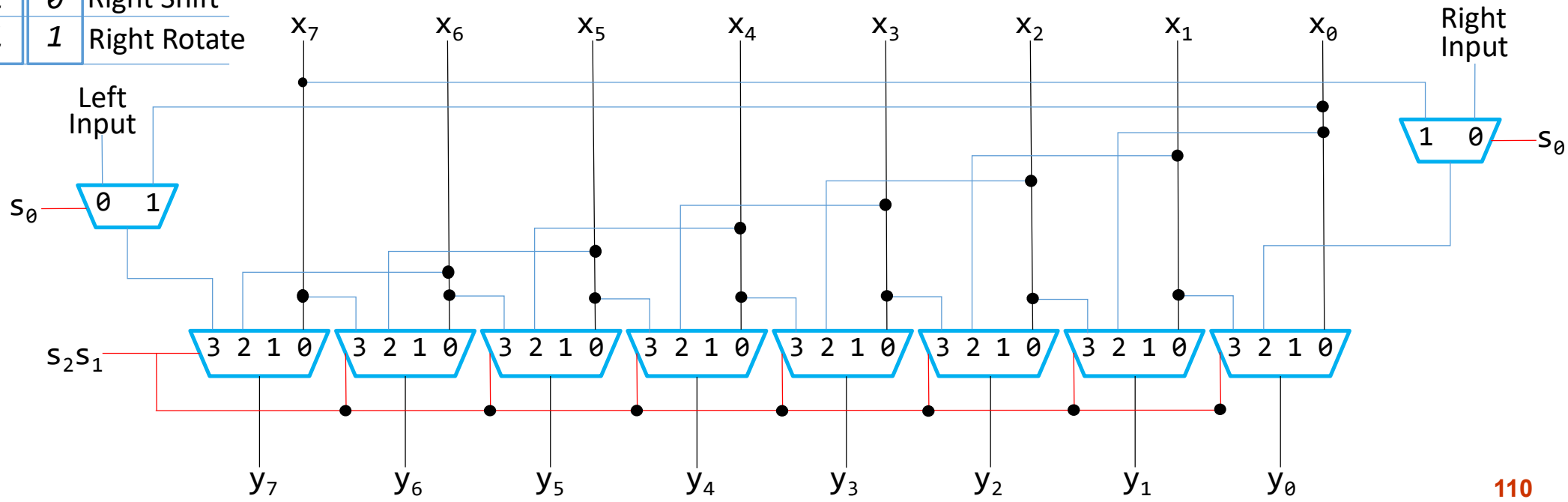
Right Shifter and Rotator

s_2	s_1	s_0	
0	0	-	No Shift
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	Right Rotate



Putting All Together

s_2	s_1	s_0	
0	0	-	No Shift
0	1	0	Not Used
0	1	1	Not Used
1	0	0	Left Shift
1	0	1	Left Rotate
1	1	0	Right Shift
1	1	1	Right Rotate



Quiz

s_2	s_1	s_0	
0	0	-	No Shift
0	1	0	Not Used
0	1	1	Not Used
1	0	0	Left Shift
1	0	1	Left Rotate
1	1	0	Right Shift
1	1	1	Right Rotate

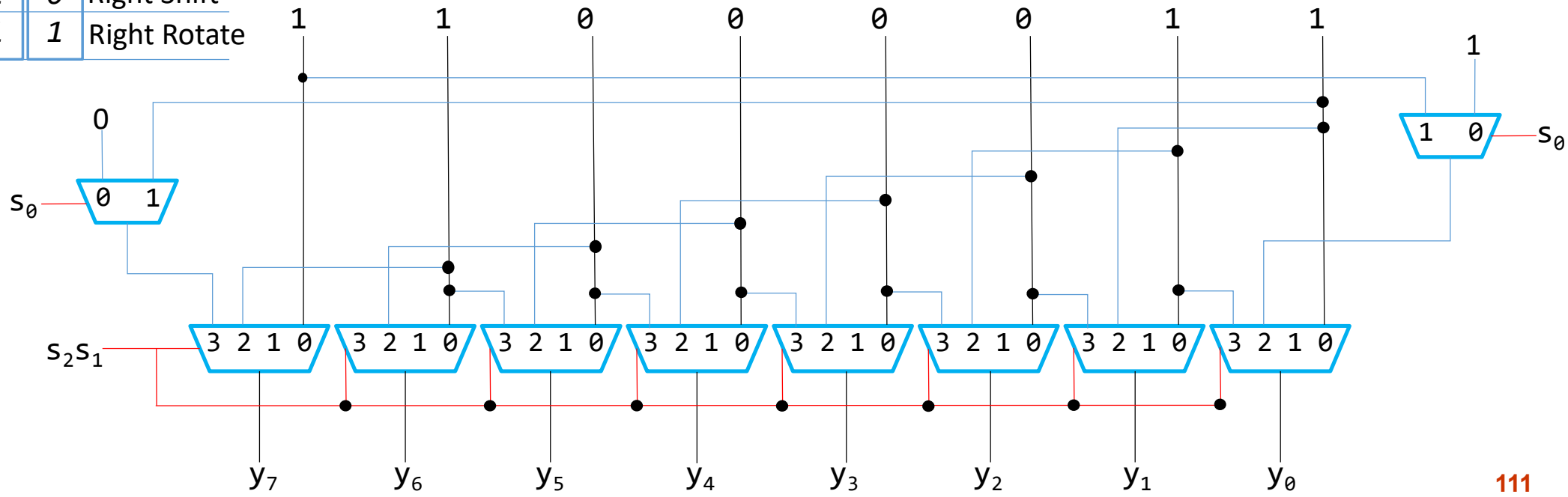
If $S_2S_1S_0$ = 100, what will be the value of y_7-y_0

A: 1000 0111

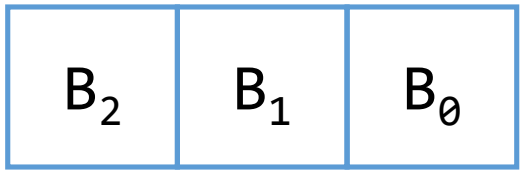
B: 1100 0011

C: 0110 0001

D: 1110 0001



The basis for Barrel Rotator / Shifter



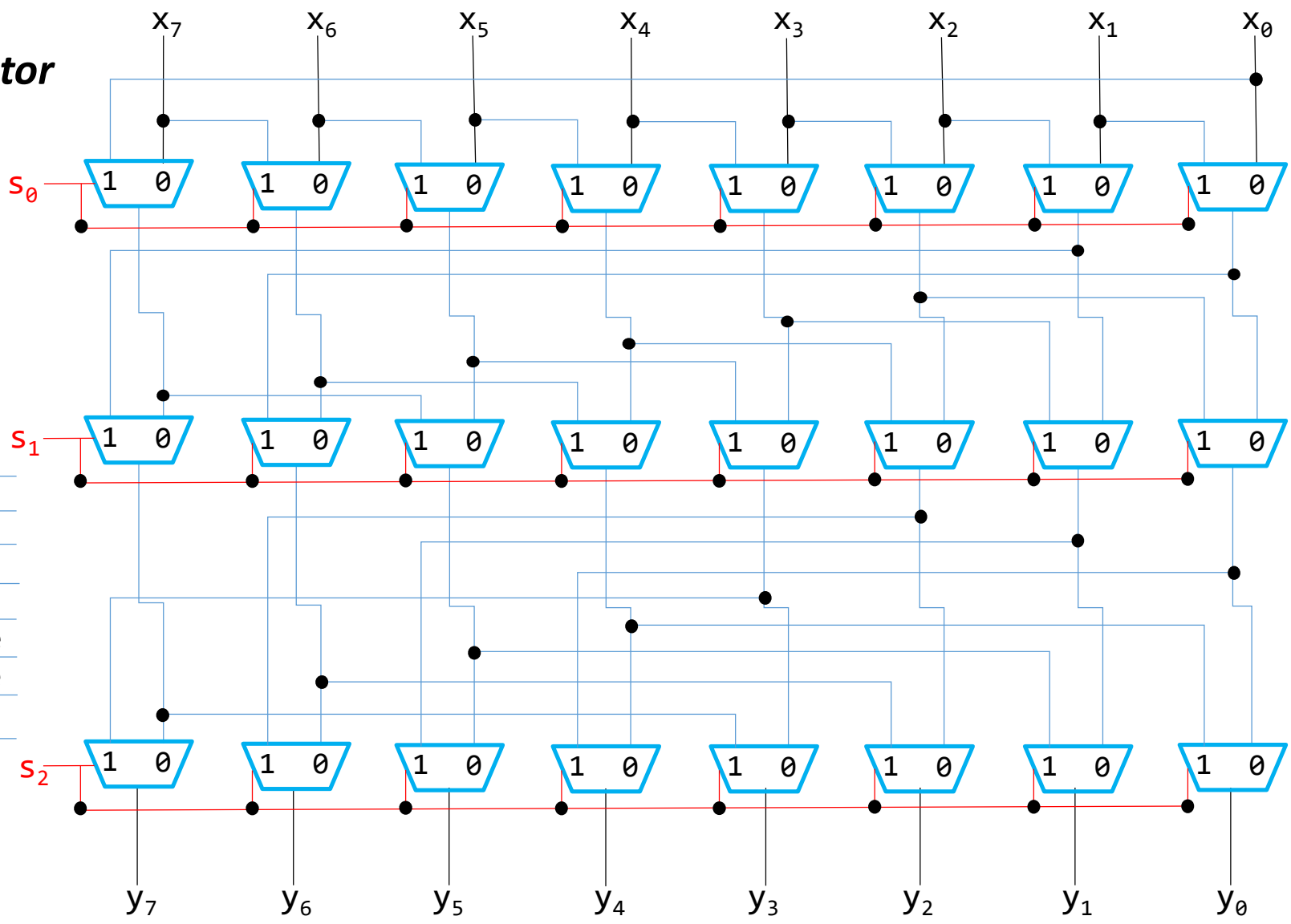
0	0	0
4	2	1

Recitation 4 has a question related Sticky Arithmetic Right Shift using barrel shifter

8 combinations that can represent numbers from 0 to 7

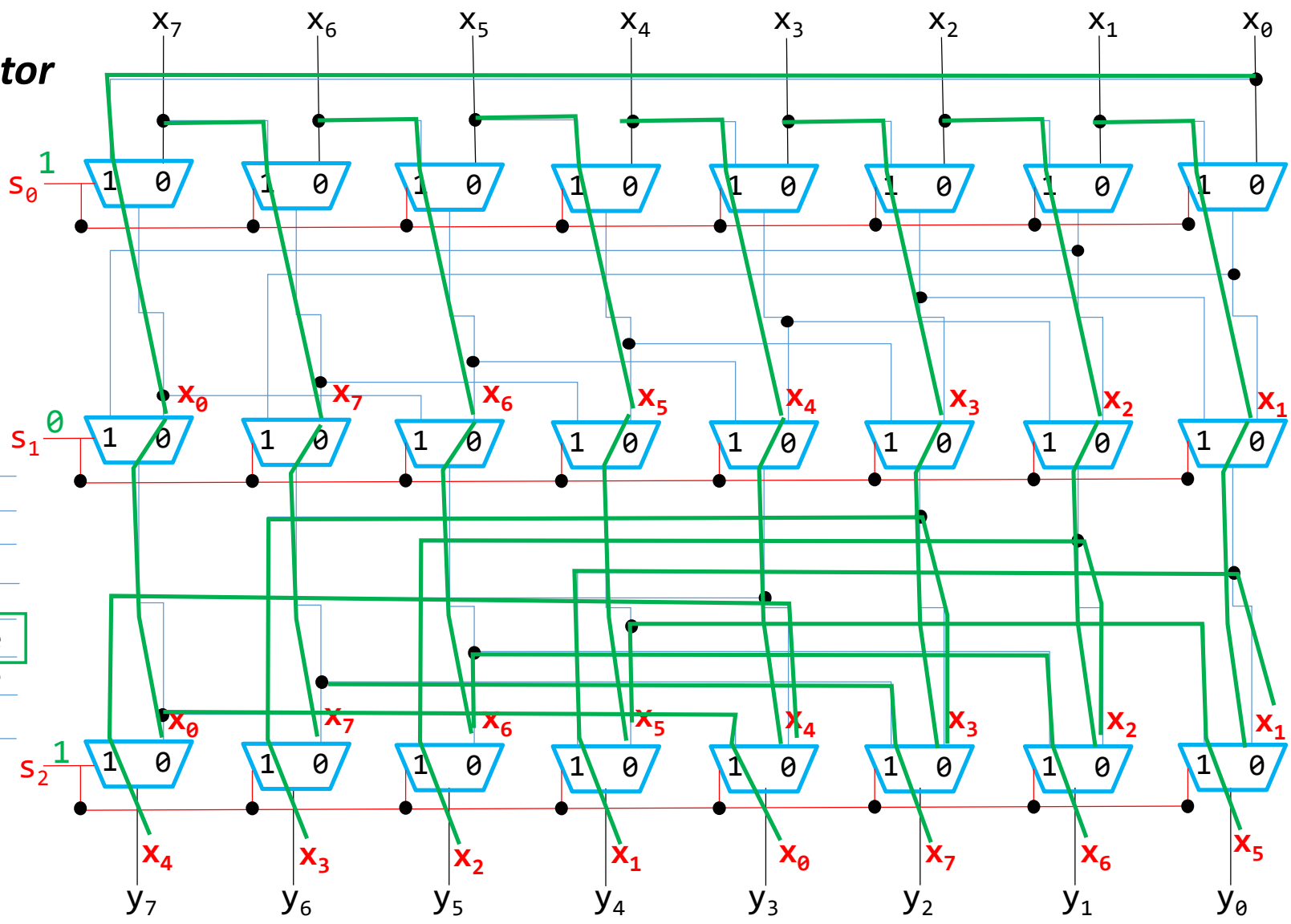
8-bit
Right Barrel Rotator

s_2	s_1	s_0	
0	0	0	0-bit rotate
0	0	1	1-bit rotate
0	1	0	2-bit rotate
0	1	1	3-bit rotate
1	0	0	4-bit rotate
1	0	1	5-bit rotate
1	1	0	6-bit rotate
1	1	1	7-bit rotate



8-bit
Right Barrel Rotator

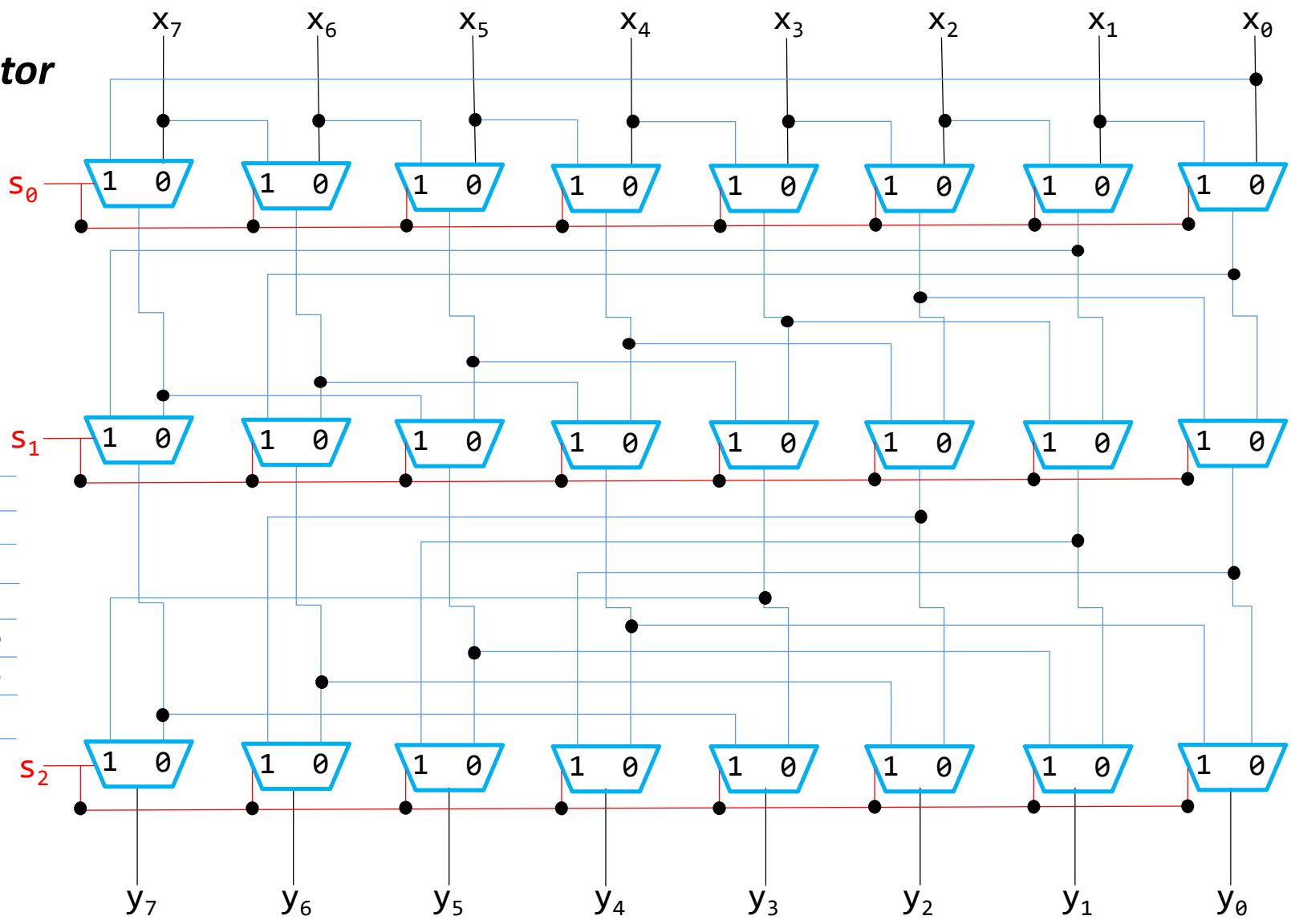
s_2	s_1	s_0	
0	0	0	0-bit rotate
0	0	1	1-bit rotate
0	1	0	2-bit rotate
0	1	1	3-bit rotate
1	0	0	4-bit rotate
1	0	1	5-bit rotate
1	1	0	6-bit rotate
1	1	1	7-bit rotate



8-bit
Right Barrel Rotator

1. Practice at home to rotate by 3 bits
2. This will reinforce your understanding

s_2	s_1	s_0	
0	0	0	0-bit rotate
0	0	1	1-bit rotate
0	1	0	2-bit rotate
0	1	1	3-bit rotate
1	0	0	4-bit rotate
1	0	1	5-bit rotate
1	1	0	6-bit rotate
1	1	1	7-bit rotate



ALU: Arithmetic Logic Unit

s_1	s_0	
0	0	Arithmetic Unit
0	1	Shifter/Rotator
1	0	Comparator
1	1	Unused

m_1	m_0		
0	0	Add	Rotate Right
0	1	Subtract	Rotate Left
1	0	Multiply	Shift Right
1	1	Unused	Shift Left

